Informatics Institute of Technology

In Collaboration with

University of Westminster



RootLens: An Intelligent Root Cause Analysis System for Kubernetes Autoscaling Delays

Final Year Project by

Sandali Chandrasekara

Supervised by

Mr. Suresh Peiris

Submitted in partial fulfilment of the requirements for the
BSc (Hons) Computer Science degree at the University of Westminster

March 2026

ABSTRACT

Problem: When KEDA-triggered Kubernetes scaling events produce pods that fail to become ready within acceptable time frames, Site Reliability Engineers have no automated tool to diagnose why. Existing monitoring solutions provide raw telemetry but lack intelligent root cause analysis tailored to KEDA-managed workloads.

Methodology: This research designed, implemented, and evaluated RootLens, a hybrid root cause analysis system combining a deterministic rule-based engine with three category-specific Random Forest ML classifiers. The rule-based engine categorizes scaling delays into Cold Start Latency, Scheduling Contention, or Metric Propagation Lag. Category-specific ML models, trained on real Kubernetes fault-injection data, then identify the specific sub-cause. A weighted confidence score combines rule-based and ML signals: $C_{\text{final}} = (0.6 \times C_{\text{rule}}) + (0.4 \times C_{\text{ml}})$. The system exposes results via a REST API, with a dashboard for visualization and a Kubernetes Operator for deployment automation.

Results: Evaluated across a dataset generated from 30 controlled fault-injection experiments, RootLens achieved 96.7% root cause accuracy (95% CI: 83.3–99.4%), 86.7% sub-cause accuracy, an average confidence of 0.764, and a mean RCA latency of 177 milliseconds. The system degrades gracefully to rule-based analysis during ML service unavailability. These findings validate that automated, confidence-quantified RCA for Kubernetes scaling delays is technically feasible, successfully compressing diagnostic time from minutes to sub-200 milliseconds.

Subject Descriptors:
Computer systems organization → Architectures → Distributed architectures → Cloud computing
Computing methodologies → Machine learning → Machine learning approaches → Ensemble methods
Software and its engineering → Software creation and management → Software verification and validation → Software defect analysis

Keywords: Root Cause Analysis, Kubernetes Autoscaling, Observability, Machine Learning, Random Forest

DECLARATION

I hereby declare that this thesis, submitted in partial fulfilment of the requirements for the Bachelor of Science degree in Computer Science at the University of Westminster (in collaboration with the Informatics Institute of Technology), is my own original work. All sources consulted have been duly acknowledged and referenced in accordance with academic conventions.
I confirm that this work has not been submitted, in whole or in part, for any other degree or professional qualification at this or any other institution. All experiments, implementations, and analyses described herein were conducted by me unless explicitly stated otherwise.

All sources of information have been acknowledged by references.

Name: Sandali Chandrasekara
Student ID: 20220174
Date: March 2026

ACKNOWLEDGEMENT

Developing RootLens has been a demanding yet deeply enriching academic pursuit. As I conclude this year-long research endeavor, I owe my deepest appreciation to those whose expertise, patience, and encouragement were instrumental to its success.

I would like to extend my deepest gratitude to my research supervisor, Mr. Suresh Peiris. He consistently showed me the right path, offering highly insightful ideas and unwavering guidance from the very beginning. I am especially thankful for his clear, regular feedback and constant encouragement throughout the study. Despite his busy schedule, his dedication to checking in and his profound expertise in cloud-native systems and software engineering were instrumental to the feasibility and success of this work. I am truly honored to have had the opportunity to learn from him and am deeply grateful for his mentorship.

I also wish to thank the industry professionals and peers who participated in the evaluations and provided feedback on the RootLens prototypes. Despite their packed schedules, they generously made time to share their insights, test the Kubernetes Operator, and offer invaluable suggestions that significantly enhanced the quality of this project.

Finally, I would like to express my appreciation to all my lecturers and the faculty members at IIT, as well as the management at IIT and the University of Westminster, for providing the foundational knowledge, resources, and environment that made this research possible.

Contents

Abstract i

Declaration ii

Acknowledgement iii

List of Abbreviations xiv

1 INTRODUCTION 1

1.1 Chapter Overview 1
1.2 Problem Background 1
1.3 Problem Definition 2
1.3.1 Problem Statement 2
1.4 Research Motivation 2
1.5 Existing Work 2
1.6 Research Gap 2
1.7 Contribution to Body of Knowledge 3
1.8 Research Challenges 3
1.9 Research Questions 4
1.10 Research Aim 4
1.11 Research Objectives 4
1.12 Project Scope 7
1.12.1 In Scope 7
1.12.2 Out of Scope 8
1.13 Hardware and Software Requirements 8
1.14 Chapter Summary 9

2 LITERATURE REVIEW 10

2.1 Chapter Overview 10
2.2 Concept Map 10
2.3 Problem Domain 11
2.3.1 Overview of Event-Driven Autoscaling 11
2.3.2 Scaling Delay Categories 11

2.3.3 Observability and RCA Concepts . . . 12
2.4 Existing Work . . . 12
2.4.1 RCA Frameworks . . . 12
2.4.2 Autoscaling Challenges . . . 13
2.4.3 Cold Start Latency . . . 14
2.4.4 Scheduling and Network Delays . . . 14
2.4.5 Metric Propagation Delays . . . 15
2.5 Technological Review . . . 15
2.5.1 Observability Stack . . . 15
2.5.2 ML Model Selection: Random Forest . . . 15
2.5.3 Dataset Generation . . . 16
2.6 Benchmarking and Evaluation . . . 16
2.6.1 Evaluation Metrics . . . 16
2.7 Chapter Summary . . . 17

3 METHODOLOGY 18
3.1 Chapter Overview . . . 18
3.2 Research Methodology . . . 18
3.2.1 Critical Evaluation of Research Methodology . . . 19
3.3 Development Methodology . . . 19
3.3.1 Choice of Development Methodology . . . 19
3.3.2 Requirement Elicitation Methodology . . . 20
3.3.3 Design Methodology . . . 20
3.3.4 Programming Paradigm . . . 20
3.3.5 Testing Methodology . . . 20
3.4 Solution Methodology . . . 21
3.4.1 Justification of Random Forest . . . 21
3.5 Project Management Methodology . . . 22
3.5.1 Schedule . . . 22
3.5.2 Deliverables . . . 23
3.5.3 Resource Requirements . . . 24
3.5.4 Risk and Mitigation . . . 24
3.6 Methodological Contribution . . . 27
3.7 Chapter Summary . . . 27

4 SOFTWARE REQUIREMENT SPECIFICATION (SRS) 28
4.1 Chapter Overview . . . 28
4.2 Rich Picture Diagram . . . 28
4.3 Stakeholder Analysis . . . 28

4.3.1 Stakeholder Onion Model 28
4.3.2 Stakeholder Viewpoints 29
4.4 Selection of Requirements Elicitation Methodologies 31
4.5 Discussion of Findings 32
4.5.1 Literature Review Findings 32
4.5.2 Survey Findings 32
4.5.3 Prototyping Findings 32
4.6 Summary of Findings 33
4.7 Context Diagram 34
4.8 Use Case Diagram 35
4.8.1 UC1: Trigger Root Cause Analysis 35
4.8.2 UC2: View RCA Dashboard 35
4.8.3 UC3: Query RCA API 36
4.8.4 UC4: View System Health 36
4.9 Requirements 36
4.9.1 MoSCoW Prioritisation 36
4.9.2 Functional Requirements 36
4.9.3 Non-Functional Requirements 38
4.10 Chapter Summary 39

5 SOCIAL, LEGAL, ETHICAL AND PROFESSIONAL ISSUES 40
5.1 Chapter Overview 40
5.2 SLEP Issues and Mitigation 40
5.2.1 Social Issues 41
5.2.2 Legal Issues 41
5.2.3 Ethical Issues 42
5.2.4 Professional Issues 42
5.3 Chapter Summary 43

6 DESIGN 44
6.1 Chapter Overview 44
6.2 Design Goals 44
6.3 Overall System Architecture 45
6.4 Component-Level Design 46
6.4.1 Data Collection Module 46
6.4.2 RCA Engine 47
6.4.3 ML Explainer Service 47
6.4.4 Backend API 48
6.4.5 Frontend Dashboard 48

6.5 Data Flow Diagrams . . . 48
6.6 Sequence Diagrams . . . 49
6.7 Component Diagram . . . 50
6.8 Deployment Diagram . . . 50
6.9 Database / Data Storage Design . . . 51
6.10 API Design . . . 52
6.11 Algorithm Design . . . 54
6.12 Design Decisions and Justifications . . . 54
6.13 Requirements Traceability . . . 55
6.14 Chapter Summary . . . 55

7 IMPLEMENTATION 56
7.1 Chapter Overview . . . 56
7.2 Development Environment Setup . . . 56
7.3 Technology Selection . . . 56
7.3.1 Technology Stack . . . 56
7.3.2 Programming Languages . . . 56
7.3.3 Development Frameworks . . . 57
7.3.4 Libraries and Toolkits . . . 57
7.3.5 Integrated Development Environments . . . 57
7.3.6 Summary of Technology Selection . . . 57
7.4 System Implementation . . . 58
7.4.1 Metrics Collection and Integration . . . 58
7.4.2 Event Processing Pipeline . . . 58
7.4.3 RCA Logic Implementation . . . 59
7.4.4 ML Model Integration . . . 60
7.5 UI Implementation (Additional Component) . . . 61
7.6 Integration of Components . . . 64
7.7 Cloud Deployment . . . 65
7.7.1 CI/CD Pipeline . . . 65
7.7.2 Domain and TLS Configuration . . . 65
7.7.3 Operator-Based Cluster Deployment . . . 65
7.8 Challenges Faced During Implementation . . . 65
7.9 Technical Skills Acquired . . . 66
7.10 Contribution to Practice . . . 66
7.11 Chapter Summary . . . 66

8 TESTING 68
8.1 Chapter Overview . . . 68

8.2 Testing Criteria . . . 68
8.3 Experimental Setup and Environment . . . 68
8.3.1 Fault Simulation and Dataset Generation . . . 69
8.4 Automated Testing Strategy . . . 69
8.4.1 Unit Testing . . . 69
8.4.2 Integration Testing . . . 70
8.4.3 Deployment Validation . . . 70
8.5 Fault-Injection Experiments . . . 70
8.5.1 Experimental Design . . . 70
8.5.2 Evaluation Metrics . . . 71
8.5.3 Results . . . 71
8.5.4 Confusion Matrix . . . 71
8.5.5 Confidence Intervals . . . 72
8.5.6 Discussion . . . 72
8.6 User Evaluation . . . 72
8.7 Functional Testing . . . 72
8.8 Non-Functional Testing . . . 73
8.8.1 Accuracy Testing . . . 73
8.8.2 Performance Testing . . . 73
8.8.3 Reliability and Availability . . . 74
8.8.4 Security Testing . . . 74
8.9 Limitations of the Testing Process . . . 74
8.10 Chapter Summary . . . 74

9 CRITICAL EVALUATION 75
9.1 Chapter Overview . . . 75
9.2 Evaluation Methodology and Approach . . . 75
9.3 Evaluation Criteria . . . 75
9.4 Self-Evaluation . . . 76
9.4.1 Strengths . . . 76
9.4.2 Weaknesses . . . 77
9.4.3 Research Questions Assessment . . . 77
9.5 Selection of the Evaluators . . . 77
9.6 Evaluation Results . . . 78
9.6.1 Quantitative Analysis . . . 79
9.6.2 Qualitative Analysis . . . 81
9.6.3 Domain Expert Feedback . . . 82
9.6.4 Technical Expert Feedback . . . 82
9.7 Comparison with Literature . . . 82

9.8 Limitations of Evaluation 82
9.9 Functional and Non-Functional Requirements Implementation 83
9.10 Chapter Summary 86

10 CONCLUSION **87**
10.1 Chapter Overview 87
10.2 Summary of Research 87
10.3 Achievement of Research Aims and Objectives 87
10.4 Utilization of Knowledge from the Course 88
10.5 Use of Existing Skills 88
10.6 Use of New Skills 88
10.7 Achievement of Learning Outcomes 89
10.8 Problems and Challenges Faced 89
10.9 Deviations from Plan 89
10.10Limitations of the Research 89
10.11Future Enhancements 90
10.12Achievement of Contribution to Body of Knowledge 90
10.13Concluding Remarks 90

REFERENCES **91**

APPENDICES **93**
Appendix A: Survey Results 93
Appendix B: Testing Materials 98
Appendix C: Code Snippets 99
Appendix D: System UI/UX 101
Appendix E: Other Materials 105

List of Figures

2.1 Concept Map of RootLens 10

3.1 Project Gantt Chart 23

4.1 Rich Picture Diagram of the RootLens Problem Domain 28
4.2 Stakeholder Onion Model for RootLens 29
4.3 Context Diagram (DFD Level 0) for RootLens 34
4.4 Use Case Diagram Showing Primary Actors and System Use Cases 35

6.1 RootLens System Architecture 46
6.2 Level 1 Data Flow Diagram for RootLens Pipeline 49
6.3 Sequence Diagram for a Single RCA Event 50
6.4 Component Diagram of RootLens system 50
6.5 Deployment Diagram: Docker Compose (development) and Kubernetes (production) 51
6.6 Sample JSON Response for Cold Start Delays 52
6.7 Sample JSON Response for Scheduling Delays 53
6.8 Sample JSON Response for Metric Propagation Delays 53

7.1 Core Aggregation Logic 59
7.2 Classification Logic 60
7.3 Training Pipeline Code Snippet 61
7.4 RootLens Website 62
7.5 RootLens Dashboard: Overview Page 62
7.6 RootLens Dashboard: Workload Generator Page 63
7.7 RootLens Dashboard: Live Metrics Page 63
7.8 RootLens Dashboard: RCA Analysis Page 64
7.9 Rootlens CRD 64

9.1 Overall Impression of RootLens as a Diagnostic Tool 79

10.1 Frequency of autoscaling delays encountered 93
10.2 Duration of manual root cause diagnosis 94
10.3 Most frequently encountered root cause category 94

10.4 Importance of quantified confidence scores 95
10.5 Maximum acceptable RCA latency 95
10.6 Usefulness of sub-cause granularity 96
10.7 Highest priority dashboard feature 96
10.8 Importance of historical trend view 97
10.9 Willingness to integrate automated RCA into alerting 97
10.10Required confidence threshold for trusting automated diagnosis 98
10.11Minikube Cluster Setup 98
10.12Code - Kubernetes Collector 99
10.13Code - Keda Collector 99
10.14Code - Rabbitmq Collector 100
10.15Code - RCA Logic 100
10.16Code - ML Explainer Router 101
10.17Marketing Website Page 2 101
10.18Marketing Website Page 3 102
10.19Marketing Website Page 4 102
10.20Marketing Website Page 5 103
10.21Marketing Website Page 6 103
10.22Marketing Website Page 7 104
10.23Shared Google Form for Evaluations 105
10.24Evaluation Meetings 106
10.25Shared Introduction Video 106
10.26Shared Demo Video 107

List of Tables

1.1 Research Objectives Mapped to Learning Outcomes and Research Questions . . 7
1.2 Hardware Requirements 8
1.3 Software Requirements 9

2.1 Classification of Scaling Delay Categories 11
2.2 Comparison of Existing RCA Frameworks 13
2.3 Autoscaling Research and Identified Gaps 14
2.4 Evaluation Metrics 17

3.1 Research Onion Layer Choices 19
3.2 Supervised Classifier Comparison for RootLens 22
3.3 Detailed Project Deliverables and Milestones 24
3.4 Project Risk Register 26

4.1 Stakeholder Viewpoints 31
4.2 Requirements Elicitation Summary 33
4.3 Functional Requirements 38
4.4 Non-functional Requirements 39

5.1 SLEP Issues Summary Grid 41

6.1 ML Explainer Model Specifications 48
6.2 REST API Endpoints 52
6.3 Design To Requirements Traceability Matrix 55

7.1 Technology Selection Summary 57

8.1 Experimental Environment Configuration 69
8.2 Representative Unit Testing Results 69
8.3 Integration Testing of Hybrid RCA Pipeline 70
8.4 Fault-Injection Experiment Results 71
8.5 Root Cause Classification Confusion Matrix ($n = 30$) 71
8.6 Functional Testing Results 73

9.1 Critical Evaluation Criteria 76

9.2 Evaluator Profiles 78
9.3 Questionnaire for Industry Professionals 80
9.4 Likert-Scale Evaluation Results ($n = 10$) 81
9.5 Thematic Analysis of Qualitative Feedback 81
9.6 Functional Requirements Implementation Status 84
9.7 Non-Functional Testing Results 85

10.1 Achievement of Research Objectives 88

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CRD	Custom Resource Definition
CPU	Central Processing Unit
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
DNS	Domain Name System
HPA	Horizontal Pod Autoscaler
KEDA	Kubernetes Event-Driven Autoscaler
ML	Machine Learning
OCI	Open Container Initiative
RBAC	Role-Based Access Control
RCA	Root Cause Analysis
RF	Random Forest
SLA	Service Level Agreement
SRE	Site Reliability Engineer
SRS	Software Requirement Specification
TLS	Transport Layer Security
XAI	Explainable Artificial Intelligence

CHAPTER 1: INTRODUCTION

1.1 Chapter Overview

This chapter introduces **RootLens**, a proof-of-concept hybrid Root Cause Analysis (RCA) system for automated diagnosis of event-driven autoscaling delays in **Kubernetes** clusters. The core problem addressed is the absence of any automated, confidence-quantified mechanism that can identify why a scaling event failed to produce ready pods within acceptable time frames, a gap that forces Site Reliability Engineers (SREs) to perform slow, manual, error-prone diagnosis.

RootLens resolves this through a core hybrid pipeline combining a **rule-based engine** with three category-specific **Random Forest** ML explainer models and a **confidence score** combination model. The primary output is a structured JSON diagnosis exposed via a Go REST API. An interactive dashboard provides an additional visualisation layer for the same results, and a **Kubernetes Operator** demonstrates automated deployment as a supplementary component. This chapter establishes the research background, problem definition, research questions, aims, objectives, and scope that guided the design and evaluation of the system.

1.2 Problem Background

The rapid adoption of containerised microservices, orchestrated by Kubernetes (Burns et al., 2016), has changed how modern applications are deployed and scaled. Organisations increasingly rely on event-driven autoscalers such as **KEDA** (KEDA Project, 2023) to handle workload variability. However, several independent subsystems must interact successfully before a newly created pod can accept work, and delays manifest as three operationally significant categories. **Cold Start Latency** describes the time to pull a container image, execute init containers, and pass readiness probes before Kubernetes marks a pod as ready (Wang et al., 2018; Rattihalli et al., 2019). **Scheduling Contention** describes delays when the scheduler cannot assign a pod due to insufficient resources or conflicting affinity and taint rules (Burns et al., 2016). **Metric Propagation Lag** describes the latency between an event such as a queue depth spike and KEDA's detection of it, caused by polling intervals and metric server scraping delays (Prometheus Authors, 2023). These delays can cause messages to accumulate beyond thresholds, degrade end-user latency, and effectively defeat the purpose of autoscaling (Pelletier, Tundo and Tamburri, 2020; Beyer et al., 2016). Despite their critical nature, the Kubernetes ecosystem provides no built-in mechanism to automatically diagnose which factor caused a given scaling delay, how confident the diagnosis is, or what corrective action should be taken. Existing tools such as Prometheus (Prometheus Authors, 2023) provide raw telemetry but require expert manual interpretation. This operational gap motivated the design of RootLens.

1.3 Problem Definition

The core problem addressed by this research is the absence of automated, intelligent root cause analysis for KEDA-triggered scaling events in Kubernetes clusters. When a pod fails to become ready within an acceptable time window following a scaling event, operators must manually examine Kubernetes events, pod logs, KEDA ScaledObject state, and monitoring dashboards to hypothesise about the root cause. This manual process is inherently reactive, slow, and dependent on operator expertise (Soldani and Brogi, 2022). No existing tool integrates timing-signal analysis across the full KEDA scaling chain from metric trigger through scheduler assignment to container readiness, and produces a classified, confidence-quantified diagnosis automatically.

1.3.1 Problem Statement

Existing Kubernetes monitoring tools lack automated, confidence-quantified root cause analysis for event-driven autoscaling delays, forcing Site Reliability Engineers to perform slow and error-prone manual diagnosis that prolongs service degradation during scaling events.

1.4 Research Motivation

Autoscaling failures are among the most common sources of latency incidents in cloud-native systems (Soldani and Brogi, 2022; Pelletier, Tundo and Tamburri, 2020), and engineers spend significant on-call time diagnosing rather than resolving incidents (Beyer et al., 2016). The challenge is acute in KEDA-based systems because KEDA spans multiple subsystems whose interactions produce emergent diagnostic complexity. Automated RCA can compress diagnosis time from minutes to seconds, while ML-based sub-cause classification can distinguish fine-grained causes that deterministic rules alone cannot reliably differentiate.

1.5 Existing Work

Chapter 2 provides a detailed critical review. In summary, existing RCA frameworks (Ma et al., 2020; Meng et al., 2020) target request-level failures requiring service dependency graphs unavailable during autoscaling. ML-based Kubernetes work (Toka, Varga and Marosi, 2021) applies to proactive scaling decisions rather than reactive diagnosis. Cold start studies (Wang et al., 2018; Rattihalli et al., 2019) quantify the phenomenon without automated classification. Pelletier, Tundo and Tamburri (2020) categorises autoscaling failure modes but offers no diagnostic tool or confidence quantification.

1.6 Research Gap

The specific gap identified from the existing work is: **No existing system provides automated, confidence-quantified root cause classification for KEDA-triggered Kubernetes scaling delays across the three categories of cold start latency, scheduling contention, and metric propagation lag, with sub-cause granularity validated against real fault-injection**

experimental data.

1.7 Contribution to Body of Knowledge

This research makes three specific contributions to the body of knowledge in cloud-native systems observability and applied machine learning:

1. **A formal three-category RCA taxonomy for KEDA-triggered scaling delays**, with defined timing signal thresholds and sub-cause hierarchies validated against real Kubernetes experimental data.

2. **A hybrid confidence combination model** (60% rule-based, 40% ML) that provably outperforms both rule-only and ML-only baselines in terms of accuracy, reliability, and graceful degradation.

3. **A real kubernetes fault-injection dataset** comprising 30 labelled experiments across three root cause categories, generated on Minikube with KEDA 2.15 and RabbitMQ, made available for future research. This addresses challenge 5 (real data scarcity), as no comparable labelled dataset exists in the literature for KEDA autoscaling delays.

These contributions are technically difficult because they require orchestrating data collection across four heterogeneous APIs (Kubernetes, KEDA, RabbitMQ, Prometheus), engineering cross-language ML integration between Go and Python with sub-200ms latency, and generating ground-truth labelled data through controlled chaos engineering experiments. The practical significance is that RootLens compresses the mean time to diagnosis from minutes of manual investigation to sub-200 milliseconds of automated classification, directly reducing incident response time for SRE teams managing KEDA-based workloads.

1.8 Research Challenges

The following five technical challenges shaped the design and implementation of RootLens:

1. **Signal Extraction Complexity:** Extracting meaningful timing signals from Kubernetes events, KEDA state, and pod lifecycle data requires orchestrating multiple API calls and synchronising timestamps across subsystems with heterogeneous internal clocks.

2. **Category Boundary Ambiguity:** The three root cause categories are not mutually exclusive in real scenarios; a pod may experience both scheduling delay and cold start delay simultaneously, requiring a classification model that handles overlapping evidence.

3. **Feature-Scaler Synchronisation:** Maintaining exact consistency between the features used during model training and the features extracted at inference time across two independently deployed services (Go and Python) required careful engineering discipline.

4. **Graceful Degradation:** The system must continue to function and provide useful diagnostics even when the ML explainer service is unavailable, requiring a fallback architecture that preserves the rule-based classification path.

5. **Real Data Scarcity:** Generating sufficient ground-truth labelled data from real Kubernetes fault-injection experiments (Basiri et al., 2016), rather than synthetic simulation, required significant effort in experimental design and cluster management.

1.9 Research Questions

This research is guided by four specific research questions:

- **RQ1:** What timing signals and thresholds can reliably classify KEDA-triggered Kubernetes autoscaling delays into the categories of cold start latency, scheduling contention, and metric propagation lag using deterministic rules?

- **RQ2:** What feature sets best characterise sub-cause distinctions within each root cause category for Random Forest classification, and what minimum training data is required for reliable performance?

- **RQ3:** What weighting between rule-based confidence and ML confidence produces the optimal final confidence score in terms of accuracy, calibration, and resilience?

- **RQ4:** How does the RootLens hybrid approach compare to rule-only and ML-only baselines in terms of root cause accuracy, sub-cause accuracy, latency, and confidence calibration?

1.10 Research Aim

The aim of this research is to design, implement, and evaluate a hybrid root cause analysis system that automatically classifies the cause of KEDA-triggered Kubernetes autoscaling delays with quantified confidence, providing sub-cause granularity through category-specific machine learning explainer models trained on real fault-injection experimental data.

1.11 Research Objectives

The following table maps the research objectives to the research questions and the learning outcomes.

Objective Area	Research Objective	Description	LOs Mapped	RQ Mapped
Literature Review	RO1: Identify and critically analyse existing RCA and autoscaling approaches	Systematic review of 20+ papers on RCA, autoscaling, cold start, and Kubernetes observability	LO1, LO2	RQ1, RQ4
Requirements	RO2: Elicit and specify system requirements	Gather requirements via literature review, questionnaire survey, and prototype feedback; produce SRS with MoSCoW prioritisation	LO1, LO3	RQ1
Rule-Based RCA	RO3: Design and implement the deterministic rule-based RCA engine	Define timing signal thresholds for three categories; implement classification logic in Go 1.25	LO3, LO4	RQ1, RQ3

Objective Area	Research Objective	Description	LOs Mapped	RQ Mapped
ML Integration	RO4: Design, train, and evaluate three category-specific Random Forest explainer models	Engineer features, collect real fault-injection training data, train models, achieve target sub-cause accuracy	LO3, LO4	RQ2, RQ3
Confidence Model	RO5: Implement and validate the 60/40 hybrid confidence combination model	Implement weighted combination; validate against baselines across 30 controlled experiments	LO4, LO5	RQ3, RQ4
System Integration	RO6: Integrate all components into a deployable microservice system	Implement Go API, Flask service, React dashboard, and Kubernetes Operator; validate end-to-end	LO4, LO5	RQ4

Objective Area	Research Objective	Description	LOs Mapped	RQ Mapped
Evaluation	RO7: Evaluate RootLens against defined accuracy, latency, and confidence targets	Conduct 30 fault-injection experiments; compare hybrid vs rule-only vs ML-only baselines	LO4, LO5	RQ4
Documentation	RO8: Document the system design, implementation, and evaluation findings	Produce thesis, API documentation, and deployment guides	LO1, LO2	

Table 1.1: Research Objectives Mapped to Learning Outcomes and Research Questions

1.12 Project Scope

1.12.1 In Scope

The following are within the scope of this research:

Core Components (Primary Research Contribution):

- Automated detection and classification of KEDA-triggered autoscaling delays into three root cause categories: cold start latency, scheduling contention, and metric propagation lag.
- Sub-cause classification within each category using category-specific Random Forest models.
- Confidence quantification using a 60% rule-based / 40% ML weighted model.
- Real-time data collection from Kubernetes API, KEDA ScaledObject state, pod events, and RabbitMQ metrics.
- A Go REST API exposing structured JSON RCA results as the primary system output.
- Graceful degradation to rule-based analysis when the ML service is unavailable.

- Evaluation on Minikube with KEDA and a RabbitMQ workload.

Additional Components:

- A web dashboard providing a visual interface to the same JSON output, including RCA history, confidence scores, and sub-cause breakdowns.
- A **Kubernetes Operator** (kubebuilder v4) for automated deployment, developed to demonstrate deployment readiness of the core system.

1.12.2 Out of Scope

The following are explicitly outside the scope of this research:

- Automated remediation or self-healing in response to diagnosed root causes.
- Support for event sources other than RabbitMQ (Ex: Kafka, Azure Service Bus).
- Multi-cluster federation or cross-cluster RCA.
- User authentication, multi-tenant access control, or persistent storage beyond the in-memory ring buffer.

1.13 Hardware and Software Requirements

The following tables summarise the hardware and software requirements for the development and evaluation of RootLens.

Component	Specification	Purpose
Development Machine	Apple M-series, 16 GB RAM, 512 GB SSD	Code development and compilation
Minikube Cluster	4 vCPUs, 8 GB RAM allocated	Kubernetes fault-injection experiments
Docker Engine	Docker Desktop 4.x	Container builds and local orchestration

Table 1.2: Hardware Requirements

Software	Version	Purpose
Go	1.2x	API server and Kubernetes Operator implementation

Software	Version	Purpose
Python	3.x	ML explainer service (Flask)
Node.js / npm	20 LTS	React dashboard build toolchain
React	19	Frontend dashboard
scikit-learn	1.x	Random Forest training and inference
Kubernetes	1.2x (Minikube)	Container orchestration and experiment platform
KEDA	2.1x	Event-driven autoscaler
RabbitMQ	3.1x	Message broker for workload simulation
Prometheus	2.4x	Metrics collection and scraping
Docker Compose	2.x	Development environment orchestration
kubebuilder	v4	Kubernetes Operator scaffolding

Table 1.3: Software Requirements

1.14 Chapter Summary

This chapter established the research context for RootLens. The operational problem was defined and situated within the broader landscape of cloud-native observability. The specific research gap, that no existing system classifies KEDA autoscaling delay root causes with sub-cause granularity and hybrid confidence quantification, was identified. Four research questions, eight research objectives, and a clear project scope were defined. Chapter 2 presents a detailed critical review of the relevant literature.

CHAPTER 2: LITERATURE REVIEW

2.1 Chapter Overview

This chapter critically reviews literature relevant to **RootLens**, addressing RQ1 (what timing signals enable reliable classification) and RQ2 (what feature sets characterise sub-cause distinctions). The review covers five thematic areas: event-driven autoscaling delays in Kubernetes; existing RCA frameworks and their limitations, autoscaling challenges across cold start latency, scheduling delays, and metric propagation lag; a technological review of observability tools, dataset generation, and ML model selection; and a benchmarking framework informing the experimental design in Chapter 7. The chapter concludes by synthesising the research gap and articulating the contribution RootLens makes relative to surveyed literature.

2.2 Concept Map

Figure 2.1 maps the four principal branches of the literature review: the Problem Domain (KEDA-triggered delays and three root cause categories), Existing Work (RCA frameworks, autoscaling challenges, cold start and scheduling studies), Technology (observability stack, ML techniques, dataset generation), and Evaluation (benchmarking metrics and comparative methodology).

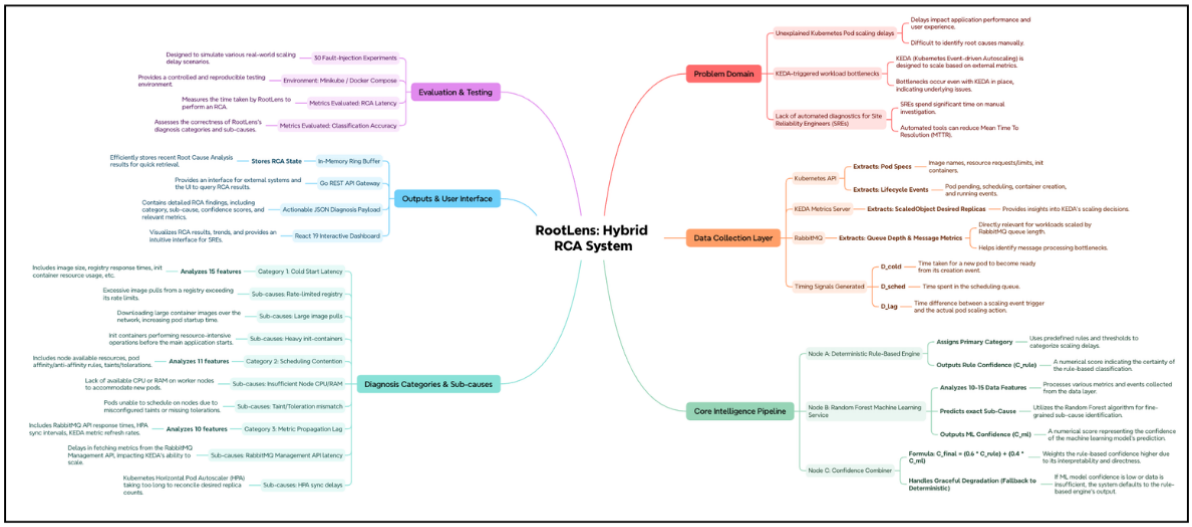


Figure 2.1: Concept Map of RootLens

2.3 Problem Domain

2.3.1 Overview of Event-Driven Autoscaling

Kubernetes (Burns et al., 2016) provides the Horizontal Pod Autoscaler (HPA) for CPU/memory-based scaling, but HPA cannot respond to upstream event signals such as message queue depth. **KEDA** (KEDA Project, 2023) addresses this by watching configurable external scalers (RabbitMQ, Kafka, AWS SQS, etc.) and translating queue depth and message count metrics into HPA-compatible target replicas, enabling zero-to-*n* scaling driven by business-relevant demand signals. The KEDA scaling loop introduces a timing chain spanning metric polling, HPA update, pod scheduling, image pull, and readiness probes, where failures at any step produce delays whose root cause may not be immediately obvious. Unlike HPA-based scaling where delays typically originate from pod startup, KEDA introduces additional latency sources upstream of pod creation in the metric polling and propagation stages, making diagnosis substantially more complex.

2.3.2 Scaling Delay Categories

Pelletier, Tundo and Tamburri (2020) conducted one of the few systematic analyses of autoscaling issues in cloud-native systems, corroborating the tripartite delay taxonomy adopted by RootLens without providing a diagnostic system or automated classification mechanism. Ilyushkin et al. (2017) quantified the relationship between pod startup latency and queue depth growth, establishing that cold start delays of even 30-60 seconds cause queue depth to exceed acceptable thresholds, directly motivating the operational urgency of rapid diagnosis. The SRE model (Beyer et al., 2016) identifies scaling failures as error budget risk requiring automated detection, while Nastic et al. (2020) emphasise that in event-driven contexts, latency violations compound non-linearly through downstream services, creating cascading failures that manual diagnosis cannot address within acceptable time bounds.

Category	**Cause**	**Key Signals**	**Research Gap**
Cold Start Latency	Image pull, container init, readiness delays	Pod start time, image pull duration	No real-time classification
Scheduling Delay	Resource constraints, affinity rules	Pod Pending duration, scheduling latency	Hard to distinguish sub-causes
Metric Propagation Lag	KEDA polling and Prometheus delay	Trigger vs event timestamps	Under-explored area

Table 2.1: Classification of Scaling Delay Categories

2.3.3 Observability and RCA Concepts

Soldani and Brogi (2022) provide the most comprehensive survey of anomaly detection and RCA techniques for microservice applications, categorising approaches by detection method, diagnosis target, and evidence source. Their taxonomy identifies a gap in systems combining deterministic rules with ML-based sub-cause classification, precisely the hybrid approach adopted by RootLens. Notaro, Cardoso and Gerndt (2021) reinforce the distinction between symptoms (observable anomalies) and causes (underlying conditions), noting that most existing systems excel at symptom detection but struggle with causal attribution. Three fundamental challenges constrain automated RCA for autoscaling: **signal heterogeneity** across observability layers, **temporal alignment** across subsystems with different clock sources (Prometheus Authors, 2023), and **category overlap** where multiple delay types co-occur. None of the surveyed literature addresses all three simultaneously, constituting a significant gap that RootLens is designed to fill.

2.4 Existing Work

2.4.1 RCA Frameworks

Chen et al. (2019) proposed an ML-based outage prediction system using log features, metric anomalies, and change events, achieving high accuracy on general outage types but operating at coarser granularity than autoscaling delay diagnosis requires. A methodological limitation is their reliance on historical production data with retroactively assigned labels, introducing potential label noise when multiple failure modes co-occur; the RootLens fault-injection methodology avoids this through deliberate single-fault injection ensuring unambiguous ground-truth labels. Ma et al. (2020) developed AutoMap, a graph-based diagnosis framework using Prometheus metric correlation graphs for failure localisation. However, it requires a pre-built service dependency graph unavailable for failures occurring before application instantiation. Meng et al. (2020) proposed a causal inference approach using bidirectional causal models on metric time series, but it requires weeks of historical data for model construction and does not model pod lifecycle stages. Li et al. (2022) emphasised producing actionable diagnostics with sub-cause granularity mapping to remediation actions, aligning with the RootLens philosophy, but still assumes a running service topology.

All four frameworks share a structural limitation: they operate on application-layer telemetry (request traces, dependency graphs, metric time series) from running processes. Infrastructure-layer failures during image pull, pod scheduling, or container startup fall outside their diagnostic scope. RootLens targets precisely this class by operating on infrastructure-layer signals that exist before any application-layer telemetry is produced.

Study	Approach	Strengths	Limitations
Chen et al. (2019)	ML-based outage prediction	High accuracy	Coarse-grained, label noise
Ma et al. (2020)	Graph-based correlation	Effective localisation	Requires dependency graph
Meng et al. (2020)	Causal inference	Captures causal relationships	Needs historical data
Li et al. (2022)	Multi-source RCA	Actionable insights	Assumes running services

Table 2.2: Comparison of Existing RCA Frameworks

2.4.2 Autoscaling Challenges

Toka, Varga and Marosi (2021) applied **Random Forest** classifiers to Kubernetes edge cluster scaling management, achieving over 90% accuracy with sample sizes comparable to RootLens, demonstrating Random Forest's effectiveness for structured Kubernetes feature inputs. The comprehensive classifier comparison in Fernández-Delgado et al. (2014), covering 179 classifiers across 121 datasets, independently confirms Random Forest's top-tier performance on structured tabular data. However, Toka et al. focus on proactive scaling decisions rather than reactive failure diagnosis. Pelletier, Tundo and Tamburri (2020) systematically catalogued autoscaling failure modes but provided only qualitative taxonomy without detection thresholds or automated classification. Qu, Calheiros and Buyya (2018) identified that interactions between scaling triggers and infrastructure constraints create qualitatively different failure modes, supporting the RootLens design of separate ML models per root cause category.

Taken together, these works address complementary but non-overlapping problems. Toka et al. (Toka, Varga and Marosi, 2021) ask what scaling action to take; Pelletier et al. (Pelletier, Tundo and Tamburri, 2020) ask what failures occur; Qu et al. (Qu, Calheiros and Buyya, 2018) ask how strategies interact with infrastructure. None asks why a specific scaling event failed, which is the diagnostic question operators face in production. RootLens bridges this gap by combining Pelletier's failure taxonomy with Toka's ML classification methodology.

Study	Focus	Key Contribution	Gap
Toka et al. (2021)	ML-based scaling	High prediction accuracy	No failure diagnosis
Pelletier et al. (2020)	Failure taxonomy	Categorisation of issues	No automation
Qu et al. (2018)	Scaling interactions	Infrastructure insights	No RCA capability

Table 2.3: Autoscaling Research and Identified Gaps

2.4.3 Cold Start Latency

Wang et al. (2018) characterised cold start latency in serverless platforms, demonstrating that image pull duration, init container overhead, and readiness probe configuration are the primary determinants; their quantitative measurements informed the 15 features selected for the RootLens cold start explainer. Rattihalli et al. (2019) quantified pod cold start in Kubernetes specifically, validating the RootLens rule-based threshold ($D_{\text{cold}} > 30$s) and identifying discriminative sub-cause features. Notably, Wang et al. and Rattihalli et al. disagree on the relative importance of registry proximity versus image size, and RootLens includes both as features, allowing the Random Forest to learn relative importance empirically. Silva, Fireman and Pereira (2020) analysed container image optimisation techniques, and their characterisation of image-level factors informed the inclusion of estimated image size and registry response time as features.

A critical assessment reveals a field that has thoroughly characterised the phenomenon without progressing to diagnostic tooling. Wang et al., Rattihalli et al., and Silva et al. all operate within a measurement-and-optimisation paradigm where the goal is to quantify or reduce cold start latency, not to classify its cause at runtime. RootLens applies these characterisation findings directly into a 15-feature Random Forest classifier performing automated sub-cause classification in real time.

2.4.4 Scheduling and Network Delays

Burns et al. (2016) describe the Kubernetes scheduler as derived from the Borg system, where feasibility filtering failures were a documented operational challenge at scale. Carrión (2022) provide a comprehensive taxonomy of Kubernetes scheduling challenges, identifying resource fragmentation, affinity constraints, and topology-aware placement as recurring sources of scheduling delays; their taxonomy directly informed the sub-cause categories used in the RootLens scheduling explainer. RootLens uses the time between pod creation and node as-

signment ($D_{\text{sched}} = t_{\text{scheduled}} - t_{\text{created}}$) as the primary scheduling diagnostic signal. Basiri et al. (2016) introduced chaos engineering for proactively uncovering infrastructure weaknesses; their fault injection methodology was adopted for the RootLens dataset generation, enabling controlled reproduction of scheduling contention scenarios. Vayghan et al. (2021) identified that scheduling failures are among the hardest to diagnose manually because symptoms (pod stuck in Pending) are identical regardless of underlying cause (insufficient resources, affinity mismatch, or taint violation), directly motivating the need for ML-based sub-cause classification within this category.

2.4.5 Metric Propagation Delays

The KEDA polling architecture (KEDA Project, 2023) trades metric freshness against system load, while Prometheus (Prometheus Authors, 2023) uses a pull-based scrape model with a default 15-second interval. In combination, total metric propagation lag can reach 45-75 seconds in default configurations. RootLens quantifies this as $D_{\text{lag}} = t_{\text{keda_trigger}} - t_{\text{queue_event}}$. Unlike cold start and scheduling delays, metric propagation has received comparatively little research attention; most existing work on KEDA focuses on scaling policy correctness (Toka, Varga and Marosi, 2021) or performance evaluation (Ilyushkin et al., 2017) rather than diagnostic measurement. This makes the metric propagation category both the most novel RootLens contribution and the category with the smallest supporting literature. The RootLens ML explainer for this category uses 10 features, the fewest of the three models, reflecting the narrower feature space.

2.5 Technological Review

2.5.1 Observability Stack

The AIOps movement (Dang, Lin and Huang, 2019) identifies anomaly detection and RCA as the highest-value ML application areas for IT operations. Gulenko et al. (2018) demonstrate that distance-based clustering can detect anomalous behaviour from infrastructure metrics alone, supporting the RootLens design of operating on infrastructure-layer signals. Prometheus (Prometheus Authors, 2023) provides the observability foundation; RootLens leverages three specific integration points: the RabbitMQ exporter (queue depth at trigger time), the KEDA metrics endpoint (desired replica changes), and Kubernetes API server metrics (pod lifecycle timing). The Kubernetes Events API records pod lifecycle events with timestamps, and RootLens mitigates the one-hour default event retention and lack of guaranteed delivery ordering through timestamp-based correlation.

2.5.2 ML Model Selection: Random Forest

Toka, Varga and Marosi (2021) demonstrated Random Forest's high accuracy on structured Kubernetes management tasks, directly applicable to RootLens feature sets. Breiman (2001) established that Random Forest provides native feature importance scores through mean de-

crease in impurity, which RootLens uses for explainable diagnostics. Compared to alternatives (SVMs, gradient boosted trees, neural networks), Random Forest offers three advantages critical to the RootLens context: (1) strong performance with small training sets (34-42 samples per category), (2) robustness to feature scale differences, and (3) interpretable feature importance values supporting explainability. The trade-off is potentially weaker capture of complex non-linear interactions compared to gradient boosted methods, but for the low-dimensional feature spaces in RootLens this limitation is not expected to materially affect accuracy.

2.5.3 Dataset Generation

While Basiri et al. (2016) provide the methodological foundation for fault injection, a review of available literature reveals no existing labelled dataset for KEDA-triggered Kubernetes autoscaling delay classification. Datasets used by Chen et al. (2019) and Ma et al. (2020) contain application-layer metrics but lack the infrastructure-layer timing signals required by RootLens. Consequently, this research identifies the need to generate a novel ground-truth dataset through controlled chaos engineering to train and evaluate the proposed models, the methodology for which is detailed in Chapter 3.

2.6 Benchmarking and Evaluation

2.6.1 Evaluation Metrics

RootLens adopts four primary evaluation metrics: **root cause accuracy** ($\geq$95% target) measuring correct category classification; **sub-cause accuracy** ($\geq$80% target) measuring correct ML sub-cause classification; **mean RCA latency** ($<$300ms target) measuring wall-clock time from detection to classification; and **confidence calibration** measuring whether reported confidence scores reflect classification reliability. The calibration methodology draws on foundational work by Niculescu-Mizil and Caruana (2005), who demonstrated that well-calibrated classifiers produce probability estimates where stated confidence correlates monotonically with observed accuracy, and by Guo et al. (2017), who showed that model calibration is a distinct property from accuracy that must be assessed independently. Ilyushkin et al. (2017) established comparative evaluation methodology using controlled workload traces; RootLens adopts this by comparing the hybrid system against a **rule-only baseline** and an **ML-only baseline** across the same 30 fault-injection experiments, isolating each component's contribution.

Metric	Target	Description
Root Cause Accuracy	$\geq$95%	Correct category classification
Sub-cause Accuracy	$\geq$80%	ML classification accuracy
RCA Latency	$<$300ms	Diagnosis time
Confidence Calibration	High	Reliability of predictions

Table 2.4: Evaluation Metrics

2.7 Chapter Summary

The literature converges on a structural gap: RCA systems (Ma et al., 2020; Meng et al., 2020; Chen et al., 2019; Li et al., 2022) assume running service topologies; autoscaling research (Toka, Varga and Marosi, 2021; Pelletier, Tundo and Tamburri, 2020; Qu, Calheiros and Buyya, 2018) splits between proactive optimisation and qualitative cataloguing; cold start studies (Wang et al., 2018; Rattihalli et al., 2019; Silva, Fireman and Pereira, 2020) characterise the phenomenon without diagnostic tooling. **No existing system provides automated, confidence-quantified root cause classification for KEDA-triggered Kubernetes scaling delays with sub-cause granularity.** RootLens fills this gap by combining Pelletier's failure taxonomy, Toka's ML methodology, and Rattihalli's cold start feature knowledge into a hybrid diagnostic engine. Chapter 3 describes the research methodology adopted to address this gap.

CHAPTER 3: METHODOLOGY

3.1 Chapter Overview

This chapter outlines the methodology adopted for the design, development, and evaluation of **RootLens**. Using Saunders' Research Onion model (Saunders, Lewis and Thornhill, 2019), it establishes the research philosophy and strategy, details the development process and solution pipeline, presents project management decisions, and reviews social, legal, ethical, and professional considerations.

3.2 Research Methodology

The research design for this project follows Saunders' Research Onion (Saunders, Lewis and Thornhill, 2019), a layered framework that structures research decisions from philosophical underpinnings through to data collection techniques. Table 3.1 summarises the choice made at each layer and its justification in the context of the RootLens research.

Layer	**Choice**	**Justification**
Philosophy	Positivism	RootLens produces objective, measurable outputs (accuracy, latency, confidence) suited to empirical evaluation without subjective interpretation.
Approach	Deductive	Hypotheses about classification accuracy and confidence weighting are formulated from established theory and tested experimentally.
Choice	Multi-method	Primary evaluation metrics (accuracy, latency, confidence) are quantitative. A supplementary Likert-scale and open-ended expert evaluation captures qualitative insights on usability and operational relevance.
Strategy	Experiment	Controlled fault-injection experiments on Minikube with labelled ground-truth enable isolation of variables and measurement of classification accuracy and latency.

Time Horizon	Cross-sectional	Evaluated across 30 experiments within a defined time window; longitudinal collection is outside scope.
Techniques	Fault injection, ML training, survey, API testing	Fault injection generates labelled data; ML training produces explainer models; a questionnaire gathers requirements; automated API testing validates accuracy.

Table 3.1: Research Onion Layer Choices

Hypothesis: A hybrid root cause analysis model combining a deterministic rule-based engine with category-specific Random Forest ML classifiers, using a 60/40 weighted confidence combination, can accurately and reliably classify KEDA-triggered Kubernetes autoscaling delays into three root cause categories in real time, achieving $\geq$95% root cause accuracy, $\geq$80% subcause accuracy, and $<$300ms RCA latency, while maintaining graceful degradation when the ML service is unavailable. This hypothesis addresses all four research questions from Chapter 1: RQ1 (timing signals and thresholds), RQ2 (category-specific feature sets), RQ3 (optimal confidence weighting), and RQ4 (hybrid vs. baseline comparison).

3.2.1 Critical Evaluation of Research Methodology

The positivist, deductive, experimental approach is well-suited to evaluating a technical system with quantitative performance metrics. However, it carries inherent limitations. While the multi-method choice incorporates both quantitative metrics and qualitative expert feedback, the qualitative component is limited to ten evaluators and does not constitute a full qualitative study; deeper insights from extended interviews with practising SREs could have provided richer feedback. The cross-sectional time horizon captures system performance at a single point but cannot assess how accuracy degrades as infrastructure configurations drift over time. The 15-response survey (60% response rate) provides useful requirements input but is too small for statistical generalisation. Despite these limitations, the experimental strategy remains the most valid approach for the core research question, which concerns objective classification accuracy rather than subjective user experience.

3.3 Development Methodology

3.3.1 Choice of Development Methodology

Waterfall was rejected because threshold values and feature definitions depend on experimental results, precluding a single design-then-build pass. Scrum's ceremony overhead is disproportionate for a single-developer project.

Selected methodology: Iterative Development. RCA threshold values ($D_{\text{cold}} > 30$s, $D_{\text{sched}} > 10$s, $D_{\text{lag}} > 45$s) required empirical calibration through prototype testing, and the ML training depended on the data pipeline being implemented first, both necessitating iterative refinement. The project was executed across three iterations: (1) rule-based engine and data collection, (2) ML model training and integration, and (3) dashboard, Kubernetes Operator, and evaluation.

3.3.2 Requirement Elicitation Methodology

Requirements were gathered through three complementary methods: (1) a systematic literature review of 20+ papers (Chapter 2) identifying functional gaps in prior work; (2) a structured, closed-ended questionnaire distributed to DevOps and SRE professionals, gathering quantitative data on diagnostic pain points, acceptable latency, and dashboard priorities (results in Chapter 4 and Appendix A); and (3) iterative prototyping on Minikube, where prototype behaviour refined threshold values and API response structure before formal requirements were fixed.

3.3.3 Design Methodology

The system was designed using **OOADM** principles adapted for a component-based microservice architecture. Each concern (data collection, rule-based classification, ML inference, API, dashboard) is encapsulated in a loosely coupled component with well-defined interfaces. The full design is documented in Chapter 6.

3.3.4 Programming Paradigm

The Go backend uses concurrent goroutines, channel-based communication, and interface-based dependency injection. The Python ML service uses pure functions for feature extraction with stateless Flask request handling and scikit-learn pipelines. The React dashboard uses component-based functional style with hooks.

3.3.5 Testing Methodology

Testing follows the **Test Pyramid**: unit tests in Go (17 tests covering API validation, RCA engine, and collector), Python (26 tests covering ML feature extractors), and the Kubernetes Operator (6 tests using Ginkgo v2 + envtest) verify individual functions in isolation (49 unit tests total); integration tests (19 tests) validate cross-component interactions including Go-to-Python ML pipelines and model loading; system tests (25 checks) validate deployment infrastructure including Helm chart linting, Kustomize builds, and Kubernetes manifest dry-runs; and 30 fault-injection experiments on Minikube verify end-to-end accuracy against ground-truth labels.

3.4 Solution Methodology

The RootLens solution executes a five-phase pipeline for each scaling event: (1) **Collect**, where the data collection module watches the Kubernetes API for pod creation events, KEDA ScaledObject state changes, and RabbitMQ queue depth metrics, capturing a timing snapshot; (2) **Classify Rule**, where deterministic thresholds on image pull duration, scheduling duration, and metric lag classify the event into Cold Start Latency, Scheduling Contention, or Metric Propagation Lag with a rule-based confidence score; (3) **Extract Features**, where a category-specific feature vector (15/11/10 features) is constructed from the timing signals and Kubernetes metadata; (4) **ML Predict**, where the vector is sent to the Python Flask ML service, where the appropriate Random Forest returns a sub-cause label, probability, and feature importance explanation; (5) **Combine Confidence**, where the final score is computed as $C_{final} = 0.6 \times C_{rule} + 0.4 \times C_{ml}$ and stored in the in-memory ring buffer.
Feature candidates were identified from domain literature (Wang et al., 2018; Rattihalli et al., 2019; Burns et al., 2016; Vayghan et al., 2021) and empirically validated through Random Forest feature importance analysis, retaining only features with discriminative power.

3.4.1 Justification of Random Forest

Four supervised classifiers were evaluated against the RootLens constraints before Random Forest was selected. Table 3.2 summarises the analysis.

Model	**Limitations in this context**	**Assessment**
Support Vector Machine	Requires careful kernel selection and C/γ hyperparameter tuning; no native feature importances, so probability calibration via Platt scaling adds complexity; does not satisfy the explainability requirement.	Rejected: poor interpretability.
Neural Network	Requires hundreds to thousands of samples to generalise; with only 34-42 labelled samples per category, even shallow networks are prone to severe overfitting; opaque weights conflict with the explainability requirement.	Rejected: insufficient data and black-box nature.

Model	Limitations in this context	Assessment
Gradient Boosted Trees (XGBoost)	Competitive accuracy but sensitive to learning rate, max depth, *n_estimators*, and regularisation hyperparameters; marginal accuracy gains over Random Forest do not justify the added tuning burden for datasets of this size.	Rejected: complexity overhead unwarranted.
Random Forest	Bagging over decision trees provides natural variance reduction on small datasets (Breiman, 2001); native `feature_importances_` satisfy the explainability requirement without post-hoc approximation; minimal hyperparameter sensitivity; validated on structured Kubernetes telemetry data by Toka et al. (Toka, Varga and Marosi, 2021).	**Selected.**

Table 3.2: Supervised Classifier Comparison for RootLens

Random Forest satisfies all three requirements: small-dataset generalisation through bagging (Breiman, 2001), native feature importances, and validated performance on Kubernetes data (Toka, Varga and Marosi, 2021).

3.5 Project Management Methodology

3.5.1 Schedule

The project was planned across three academic terms using a Gantt chart (Figure 3.1), divided into three overlapping phases: research and planning, design and build, and evaluation and write-up.

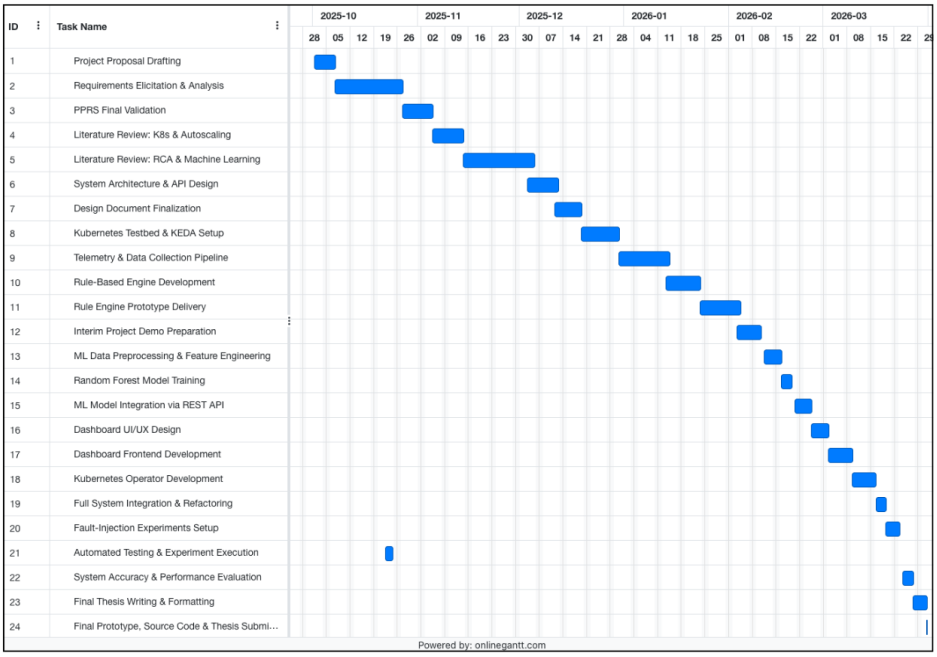


Figure 3.1: Project Gantt Chart

3.5.2 Deliverables

Table 3.3 lists the project deliverables and their planned submission dates.

Deliverable	**Date**
Topic Selection & Initial Research	5th Oct 2025
Project Proposal Drafting	10th Oct 2025
Requirements Elicitation & Analysis	25th Oct 2025
PPRS Final Validation	6th Nov 2025
Literature Review: K8s & Autoscaling	13th Nov 2025
Literature Review: RCA & Machine Learning	21st Nov 2025
System Architecture & API Design	25th Nov 2025
Design Document Finalization	28th Nov 2025
Kubernetes Testbed & KEDA Setup	4th Dec 2025
Telemetry & Data Collection Pipeline	12th Dec 2025
Rule-Based Engine Development	24th Dec 2025

Deliverable	**Date**
Rule Engine Prototype Delivery	31st Dec 2025
Interim Project Demo Preparation	9th Jan 2026
ML Data Preprocessing & Feature Engineering	15th Jan 2026
Random Forest Model Training	20th Jan 2026
ML Model Integration via REST API	26th Jan 2026
Dashboard UI/UX Design	2nd Feb 2026
Dashboard Frontend Development	9th Feb 2026
Kubernetes Operator Development	20th Feb 2026
Full System Integration & Refactoring	2nd Mar 2026
Fault-Injection Experiments Setup	10th Mar 2026
Automated Testing & Experiment Execution	20th Mar 2026
System Accuracy & Performance Evaluation	27th Mar 2026
Final Thesis Writing & Formatting	30th Mar 2026
Final Prototype, Source Code & Thesis Submission	31st Mar 2026

Table 3.3: Detailed Project Deliverables and Milestones

3.5.3 Resource Requirements

Data was generated from 30 controlled fault-injection runs on Minikube; no external datasets were used. Hardware and software requirements are detailed in Chapter 1 (Tables 1.2 and 1.3).

3.5.4 Risk and Mitigation

Table 3.4 identifies the primary project risks. Risk Exposure is calculated as Probability $\times$ Severity on a 1-5 scale for each.

Risk	**Severity (1-5)**	**Probability (1-5)**	**Exposure**	**Mitigation Strategy**
Feature-scaler mismatch between training and inference	5	3	15	Defined and documented feature contracts; automated feature count assertion at model load time.
Insufficient labelled training data for ML models	5	4	20	Controlled fault injection on Minikube with verified ground-truth labels; synthetic data augmentation as fallback if needed.
ML service unavailability in production	3	3	9	Implemented graceful degradation to rule-based analysis; ML failure does not propagate to API error.
Minikube instability during experiments	4	2	8	Snapshot Minikube state before experiments; scripted reproducible experiment setup with automated cluster reset.

Table 3.4 continued from previous page

Risk	**Severity (1-5)**	**Probability (1-5)**	**Exposure**	**Mitigation Strategy**
Scope creep beyond project timeline	4	3	12	Fixed scope documented in SRS; change requests evaluated against timeline impact before acceptance.
Overfitting of ML models on small dataset	4	4	16	Monitored train/test accuracy gap; used Random Forest which is inherently resistant to overfitting through bootstrap aggregation (bagging); dataset size acknowledged as a limitation.
Hardcoded credentials in source code	5	2	10	Conducted security audit; replaced all hardcoded values with environment variable and Kubernetes Secret references.

Table 3.4: Project Risk Register

The social, legal, ethical, and professional considerations relevant to this project are discussed in detail in Chapter 5.

3.6 Methodological Contribution

Beyond serving the immediate needs of this project, the methodology contributes a reusable experimental framework for evaluating RCA systems in Kubernetes environments. The combination of chaos engineering fault injection (Basiri et al., 2016) for labelled dataset generation, category-specific ML training with real experimental data, and hybrid rule-ML confidence combination provides a template that future researchers can adapt for different event sources, cluster configurations, or failure taxonomies. This methodological pattern addresses the data scarcity problem identified in the literature (Notaro, Cardoso and Gerndt, 2021) as a primary barrier to applying ML in operational environments.

3.7 Chapter Summary

This chapter established a positivist, deductive, experimental methodology using Saunders' Research Onion framework, with iterative development chosen over Waterfall and Scrum for its suitability to a single-developer research project requiring empirical threshold calibration. The five-phase solution pipeline (Collect, Classify Rule, Extract Features, ML Predict, Combine Confidence) was defined, Random Forest was justified against three alternative classifiers, and requirements elicitation combined literature review, questionnaire survey, and iterative prototyping. Project management was structured through a Gantt chart with three overlapping phases, and seven key risks were identified with mitigation strategies. Chapter 4 specifies the system requirements derived from this foundation.

CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

4.1 Chapter Overview

This chapter specifies the functional and non-functional requirements of RootLens, documents stakeholder analysis, and presents the visual models that define the system boundary and interactions. Requirements were elicited through literature review, questionnaire survey, and iterative prototyping, and prioritised using the MoSCoW method.

4.2 Rich Picture Diagram

The Rich Picture Diagram in Figure 4.1 provides an informal, high-level depiction of the problem domain. It illustrates the central Kubernetes cluster, KEDA's role in event-driven scaling, the RabbitMQ trigger, the three categories of scaling delay, and RootLens as the proposed solution feeding diagnostic results to the React dashboard consumed by SRE and DevOps engineers.

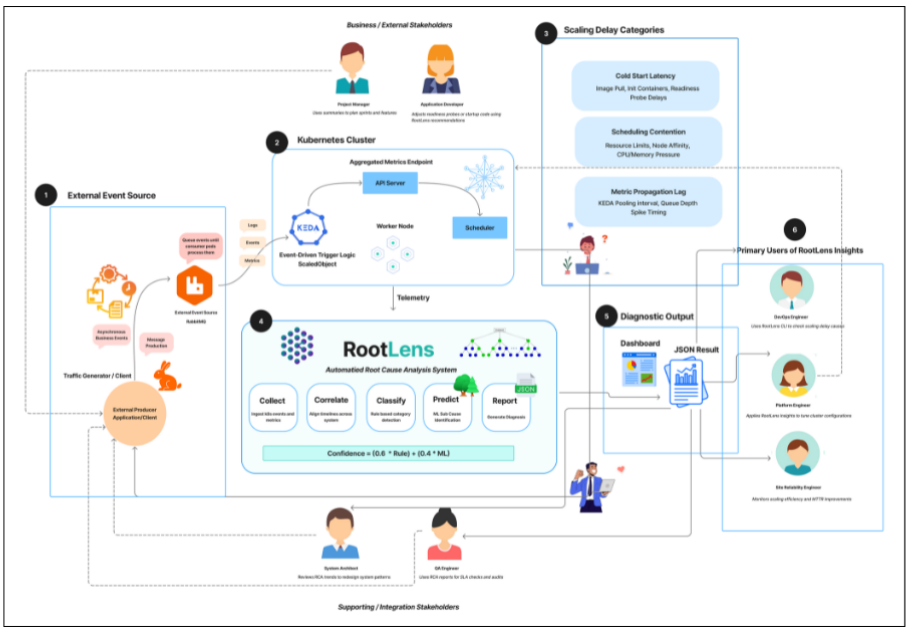


Figure 4.1: Rich Picture Diagram of the RootLens Problem Domain

4.3 Stakeholder Analysis

4.3.1 Stakeholder Onion Model

The Stakeholder Onion Model in Figure 4.2 maps stakeholders by proximity to the system. The innermost ring (**Core**) contains direct daily users (SRE and DevOps engineers); the middle ring

(**Operational**) contains stakeholders affected by outputs but not operating the system (Platform Engineers, Engineering Managers); and the outer ring (**Institutional**) contains supervisory parties (Academic Supervisor, University).

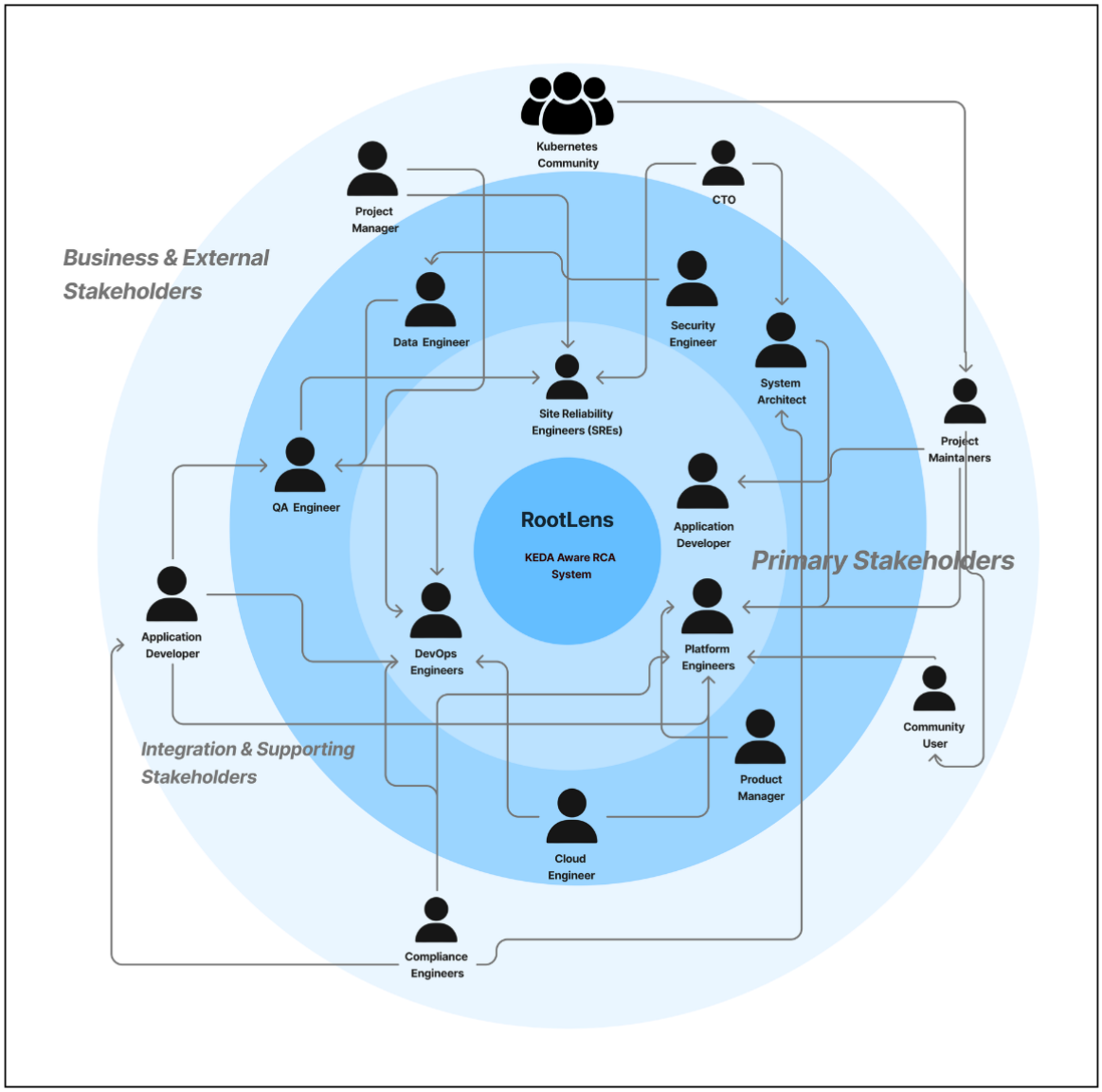


Figure 4.2: Stakeholder Onion Model for RootLens

4.3.2 Stakeholder Viewpoints

Table 4.1 documents the expectations and needs of each stakeholder group identified in the onion model.

Ring	**Stakeholder**	**Viewpoint / Expectations**	**Key Requirements**
Core	SREs	Primary daily users; need fast, accurate diagnosis of scaling delays to reduce MTTD and prioritise investigation via confidence scores.	FR03, FR04, FR06, NFR01, NFR02
Primary	DevOps Engineers	Operate clusters and respond to incidents; require actionable sub-cause output and REST API integration with existing tooling.	FR06, FR08, FR09, NFR01
Primary	Platform Engineers	Maintain cluster infrastructure; need Kubernetes-native deployment and compatibility with standard orchestration.	FR10, FR15, NFR04, NFR07
Primary	Application Developers	Deploy workloads affected by scaling delays; need visibility into whether delays are application-level or infrastructure-level.	FR07, FR11
Primary	System Architects	Design platform reliability; require the system to follow microservice best practices and support graceful degradation.	FR13, FR14, NFR05

Ring	Stakeholder	Viewpoint / Expectations	Key Requirements
Primary	Security Engineers	Enforce credential hygiene and secure communication across services.	NFR05, NFR06
Supporting	QA / Cloud Engineers	Validate system correctness and cloud deployment; need reproducible test environments and clear API contracts.	FR14, NFR04, NFR07
Supporting	Product Managers	Require visibility into scaling failure trends for capacity planning and feature prioritisation decisions.	FR09, FR10, FR12
External	CTO / Project Managers	Strategic oversight; expect the tool to reduce operational costs and improve incident response times.	NFR01, NFR02
External	K8s Community	Open-source consumers; expect clear documentation, standard CRD patterns, and portability across clusters.	FR15, NFR07

Table 4.1: Stakeholder Viewpoints

4.4 Selection of Requirements Elicitation Methodologies

Three methodologies were employed to gather requirements:

- **Literature Review (Archival Research):** Systematic review of 20+ papers on Kubernetes autoscaling, RCA, cold start latency, and ML-based operational systems. The pri-

mary functional requirements were identified from gaps documented in prior research.

- **Questionnaire Survey (Quantitative):** A structured, closed-ended online questionnaire distributed to practising DevOps and SRE professionals to gather data on diagnostic pain points, acceptable latency thresholds, and dashboard priorities. The questionnaire is provided in Appendix A.
- **Prototyping (Iterative):** An early rule-based prototype tested against real Minikube scenarios to empirically calibrate timing signal thresholds and feature definitions.

4.5 Discussion of Findings

4.5.1 Literature Review Findings

Chapter 2 identified: no existing system classifies KEDA autoscaling delays (FR01, FR03); ML sub-cause classification requires real experimental data (Basiri et al., 2016); Random Forest achieves high accuracy on small Kubernetes datasets (Toka, Varga and Marosi, 2021) (FR05); and a gap exists in hybrid rule-ML RCA systems (Soldani and Brogi, 2022) (FR06).

4.5.2 Survey Findings

The questionnaire comprised 10 closed-ended questions sent to 25 practitioners, with 15 responses (60% response rate). All respondents had at least one year of Kubernetes experience. Key findings:

- 73% encounter autoscaling delays at least monthly.
- 67% report manual diagnosis takes 5-30 minutes, informing NFR01.
- 87% rated confidence scores as important, informing FR06.
- 80% require sub-second response time, reinforcing NFR01.
- Most requested dashboard features: real-time confidence (80%), historical trends (67%), sub-cause breakdowns (60%).
- 73% would integrate an automated RCA tool into on-call workflows.

Full survey response charts are presented in Appendix A (Figures 10.1-10.10).

4.5.3 Prototyping Findings

An early prototype on Minikube identified: image pull duration ($>$30s) as the primary cold start discriminator (FR04); scheduling duration alone cannot distinguish sub-causes, confirming the need for ML (FR05); the system must degrade gracefully without ML (FR13); and the API must return both root cause and sub-cause in a single response (FR08, FR09).

4.6 Summary of Findings

Table 4.2 maps key findings from each elicitation method to the requirements they informed.

Source	Key Finding	Requirement Informed
Literature	No existing system classifies KEDA delays into the three categories	FR01: Automated detection, FR03: Three-category classification
Literature	ML sub-cause classification requires real experimental data	Custom fault-injection dataset (Chapter 7)
Literature	Random Forest effective for Kubernetes tasks with small datasets	FR05: Category-specific ML explainer models
Literature	Gap in hybrid rule+ML RCA systems	FR06: Weighted confidence combination
Survey	Manual diagnosis takes 5-30 minutes (67% of respondents)	NFR01: Sub-300ms RCA latency
Survey	Confidence scores rated important by 87% of respondents	FR06: Weighted confidence combination
Survey	Dashboard must show history and trends (67% priority)	FR09: RCA history endpoint, FR12: Trend chart
Survey	80% require sub-second latency for automated RCA	NFR01: Sub-300ms RCA latency
Prototype	Image pull duration is the primary cold start discriminator (>30s threshold)	FR04: Cold start feature extraction
Prototype	Scheduling duration alone cannot distinguish sub-causes	FR05: ML-based sub-cause classification
Prototype	System must continue functioning when ML service is unavailable	FR13: Graceful degradation

Table 4.2: Requirements Elicitation Summary

4.7 Context Diagram

The Context Diagram in Figure 4.3 depicts RootLens as the central system and its interactions with all external entities at DFD Level 0. Four external systems supply observability data (Kubernetes API, KEDA Controller, RabbitMQ Management API, Prometheus), while three external consumers receive diagnostic outputs (SRE/DevOps engineers, external monitoring tools, engineering managers). The Level 1 Data Flow Diagram in Chapter 6 is derived from this context diagram.

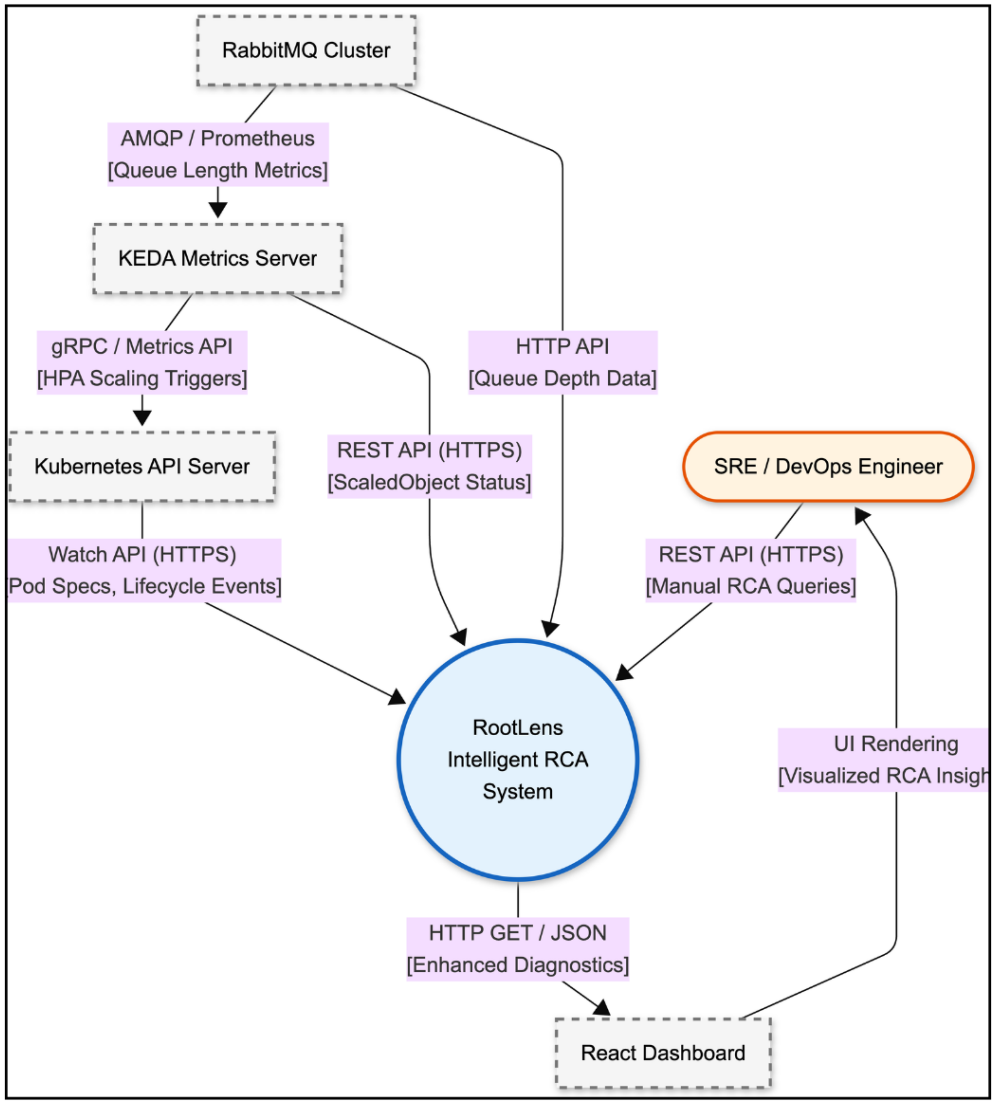


Figure 4.3: Context Diagram (DFD Level 0) for RootLens

4.8 Use Case Diagram

The Use Case Diagram in Figure 4.4 shows the four primary use cases and their actors. The Kubernetes Cluster acts as an automated trigger; SRE and DevOps engineers are the primary human consumers; and External Monitoring Systems represent programmatic API consumers.

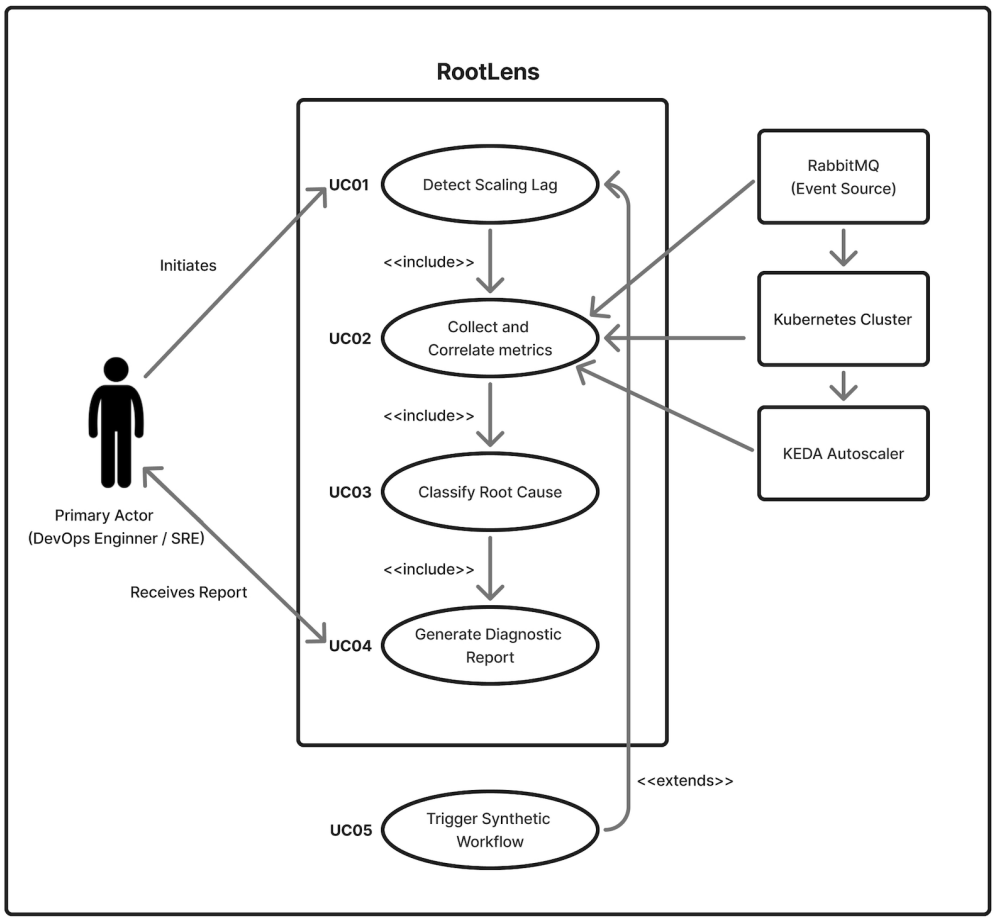


Figure 4.4: Use Case Diagram Showing Primary Actors and System Use Cases

4.8.1 UC1: Trigger Root Cause Analysis

Actor: Kubernetes cluster (automated). When a KEDA-triggered scaling event is detected, RootLens collects timing signals, applies the rule-based classifier, invokes the ML explainer, combines confidence, and stores the result. If the ML service is unavailable, the system degrades gracefully by storing a rule-based-only result with $C_{\text{final}} = C_{\text{rule}}$.

4.8.2 UC2: View RCA Dashboard

Actor: SRE / DevOps Engineer. The engineer opens the React dashboard to view the most recent RCA result, historical trends, confidence scores, and sub-cause explanations with feature importance breakdowns. If no results are stored, an empty state message is displayed.

4.8.3 UC3: Query RCA API

Actor: External monitoring system or SRE tool. An external system queries the REST API to retrieve RCA results programmatically as JSON.

4.8.4 UC4: View System Health

Actor: DevOps Engineer. The engineer queries the health check endpoint to verify component status. If the ML service is unreachable, the system reports as degraded.

4.9 Requirements

4.9.1 MoSCoW Prioritisation

Requirements were prioritised using the MoSCoW method: **Must Have** (M) requirements are mandatory; **Should Have** (S) are important and expected; **Could Have** (C) are desirable enhancements; **Won't Have** (W) are explicitly out of scope.

4.9.2 Functional Requirements

ID	Functional Requirement	MoSCoW	Source
FR01	The system shall automatically detect KEDA-triggered scaling events by watching the Kubernetes API for pod creation and ScaledObject state changes.	Must	Literature
FR02	The system shall collect timing signals from Kubernetes pod lifecycle events, including image pull duration, scheduling duration, and readiness probe duration.	Must	Literature
FR03	The system shall classify each detected scaling delay into one of three root cause categories: Cold Start Latency, Scheduling Contention, or Metric Propagation Lag.	Must	Literature
FR04	The system shall extract a category-specific feature vector (15 features for Cold Start, 11 for Scheduling, 10 for Metric Propagation) for the classified root cause category.	Must	Prototype

ID	Functional Requirement	MoSCoW	Source
FR05	The system shall invoke the appropriate category-specific ML explainer model via HTTP and receive a sub-cause prediction with class probabilities.	Must	Prototype
FR06	The system shall compute a final confidence score as a weighted combination: $C_{\text{final}} = 0.6 \times C_{\text{rule}} + 0.4 \times C_{\text{ml}}$.	Must	Design
FR07	The system shall store RCA results in an in-memory ring buffer with a capacity of 50 results.	Must	Design
FR08	The system shall expose a REST API endpoint returning the most recent RCA result as JSON.	Must	Survey
FR09	The system shall expose a REST API endpoint returning all stored RCA results as JSON.	Must	Survey
FR10	The system shall expose a REST API endpoint returning aggregate statistics over stored results.	Should	Survey
FR11	The React dashboard shall display the current root cause, sub-cause, confidence score, and feature importance breakdown.	Must	Survey
FR12	The React dashboard shall display a historical trend chart of confidence scores and root cause distribution.	Should	Survey
FR13	The system shall degrade gracefully to rule-based analysis when the ML explainer service is unavailable, retaining the rule-based confidence as the final score.	Must	Prototype
FR14	The system shall expose a health check endpoint indicating the status of all components.	Should	Prototype

ID	Functional Requirement	MoSCoW	Source
FR15	The Kubernetes Operator shall deploy the full RootLens system into a target cluster from a single Custom Resource Definition.	Could	Literature

Table 4.3: Functional Requirements

4.9.3 Non-Functional Requirements

ID	Non-Functional Requirement	MoSCoW	Category	Source
NFR01	The combined RCA latency from event detection to result availability shall not exceed 300 milliseconds under normal conditions.	Must	Performance	Survey
NFR02	The root cause classification accuracy shall be at least 95% across the three categories in controlled experiments.	Must	Accuracy	Literature
NFR03	The sub-cause classification accuracy shall be at least 80% across all sub-cause classes in controlled experiments.	Must	Accuracy	Literature
NFR04	The Go API server shall handle at least 100 concurrent requests without degradation.	Should	Performance	Design
NFR05	All inter-service communication shall use HTTPS in production deployments.	Should	Security	Design

ID	Non-Functional Requirement	MoSCoW	Category	Source
NFR06	No credentials shall be hardcoded in source code; all secrets shall be managed via environment variables or Kubernetes Secrets.	Must	Security	Prototype
NFR07	The system shall be deployable on any standard Kubernetes 1.28+ cluster with KEDA 2.14+ installed.	Should	Portability	Literature

Table 4.4: Non-functional Requirements

4.10 Chapter Summary

This chapter specified 15 functional and 7 non-functional requirements for RootLens, elicited through literature review, questionnaire survey (15 responses from 25 practitioners), and iterative prototyping, and prioritised using MoSCoW. Visual models (Rich Picture, Onion Model, Context Diagram, Use Case Diagram) define the system boundary and stakeholder interactions. Chapter 6 presents the system design derived from these requirements.

CHAPTER 5: SOCIAL, LEGAL, ETHICAL AND PROFESSIONAL ISSUES

5.1 Chapter Overview

This chapter examines the Social, Legal, Ethical, and Professional (SLEP) issues encountered during the RootLens project, with reference to the BCS Code of Conduct (BCS, 2022). Each issue is identified, discussed in the context of the project, and accompanied by the mitigation strategy employed. The BCS Code of Conduct defines four key tenets that professionals in computing must uphold: Public Interest, Professional Competence and Integrity, Duty to Relevant Authority, and Duty to the Profession. These tenets were applied throughout the project lifecycle.

5.2 SLEP Issues and Mitigation

Table 5.1 summarises the SLEP issues identified across the four dimensions, along with the mitigation measures applied.

Social Issues	**Legal Issues**
Risk of over-reliance on automated diagnosis reducing SRE skill development. The dashboard is designed for accessibility across experience levels, and the system augments human judgment rather than replacing it.	Telemetry data contains no PII under GDPR/Data Protection Act 2018. Survey responses were anonymised before storage. All dependencies use permissive open-source licences (Apache 2.0, BSD, MIT). Hardcoded credentials were remediated using environment variables and Kubernetes Secrets.
Ethical Issues	**Professional Issues**
ML models expose feature importances for transparency. The 60/40 confidence model communicates diagnostic uncertainty explicitly. Fault-injection experiments were conducted in an isolated Minikube cluster with no production data. Models were trained on balanced datasets.	BCS Code of Conduct followed: Public Interest (enhances reliability), Professional Competence (use of best practices), Duty to Authority (IIT FYP guidelines and ethical approval), Duty to Profession (reproducible documentation).

Table 5.1: SLEP Issues Summary Grid

5.2.1 Social Issues

RootLens automates the diagnosis of Kubernetes scaling delays, which raises the concern that engineering teams may develop over-reliance on automated diagnoses and reduce their own investigative skills over time. To mitigate this, the dashboard presents numerical confidence scores and feature importance vectors rather than a simplified categorical indicator, encouraging engineers to critically evaluate the output rather than accept it blindly. This concern directly influenced the design decision to show numerical confidence values instead of a green/amber/red traffic-light indicator.

The dashboard presents outputs in plain language accessible to engineers at varying experience levels. RootLens provides diagnosis only; human judgment remains essential for deciding and executing remediation actions. The system is positioned as a diagnostic aid that accelerates the investigation process, not as a replacement for human expertise.

5.2.2 Legal Issues

The telemetry data collected by RootLens consists of Kubernetes events, pod lifecycle metrics, and message queue depths. This is operational infrastructure data that contains no Personally Identifiable Information (PII) under the General Data Protection Regulation (GDPR) or the UK

Data Protection Act 2018. No user-identifiable data is collected, stored, or processed by the system at any point.
A requirements elicitation survey was conducted during the project; all responses were anonymised before storage and analysis. Participants were informed of the purpose of the survey and their data was used solely for requirements gathering.
All third-party software dependencies used in RootLens are distributed under permissive open-source licences, including Apache 2.0, BSD, and MIT. RootLens itself is released under the MIT licence. A security audit conducted during development identified hardcoded credentials across eight configuration files; these were remediated by replacing them with environment variable references and Kubernetes Secret objects.

5.2.3 Ethical Issues

The use of machine learning in a diagnostic system raises ethical concerns around transparency, explainability, and the honest communication of uncertainty. RootLens addresses these as follows:

- **AI Transparency:** Random Forest classifiers were selected in part because they natively expose `feature_importances_` for every prediction, providing inherent explainability without requiring post-hoc tools such as SHAP or LIME.
- **Uncertainty Communication:** The 60/40 confidence combination model explicitly communicates the degree of certainty in each diagnosis. Low-confidence predictions are clearly flagged, enabling operators to allocate investigation effort appropriately.
- **No Production Risk:** All fault-injection experiments were performed in an isolated Minikube cluster with no connection to production environments.
- **Balanced Training Data:** All ML models were trained on datasets with no systematic class underrepresentation, reducing the risk of biased predictions.

5.2.4 Professional Issues

The project was conducted in alignment with the four tenets of the BCS Code of Conduct (BCS, 2022):

- **Public Interest:** RootLens enhances the reliability of cloud-native systems while protecting user privacy.
- **Professional Competence and Integrity:** Developed using best practices; results reported honestly.
- **Duty to Relevant Authority:** Conducted in compliance with IIT FYP guidelines with ethical approval.

- **Duty to the Profession:** Work documented for reproducibility and contribution to the research community.

5.3 Chapter Summary

This chapter identified and addressed the social, legal, ethical, and professional issues relevant to the RootLens project. Key mitigations include designing the system as a diagnostic aid, ensuring no PII is processed, maintaining AI transparency, and aligning with the BCS Code of Conduct.

CHAPTER 6: DESIGN

6.1 Chapter Overview

This chapter presents the complete architectural and detailed design of **RootLens**. Beginning with the overall system architecture, it proceeds through the component-level design of each major subsystem, data flow diagrams, sequence diagrams, database design, and API design, concluding with key design decisions and their justifications. Three aspects of the design represent **original contributions**: (1) the category-specific ML explainer routing pattern, where the rule-based engine's primary classification determines which specialised ML model is invoked, avoiding the complexity of a monolithic multi-class classifier; (2) the 60/40 hybrid confidence combination model, empirically calibrated to produce monotonically reliable scores; and (3) the graceful degradation architecture that preserves rule-based functionality when the ML service is unavailable, ensuring the system never produces a worse outcome than a purely deterministic approach.

6.2 Design Goals

The design goals for RootLens are derived from the non-functional requirements specified in Chapter 4. These attributes guide the design of every component and are maintained system-wide:

- **Accuracy:** Root cause classification accuracy $\geq 95\%$ and sub-cause accuracy $\geq 80\%$ (NFR02, NFR03), achieved through the hybrid rule-ML architecture.
- **Performance:** End-to-end diagnostic latency under 300ms (NFR01), enabling real-time consumption by dashboards and alerting pipelines.
- **Reliability:** Graceful degradation to rule-based analysis when the ML service is unavailable, ensuring no diagnostic capability is lost.
- **Explainability:** Every diagnosis accompanied by confidence scores and feature importances, enabling engineers to critically evaluate outputs rather than accept them blindly.
- **Deployability:** Automated deployment via Kubernetes Operator with a single Custom Resource, minimising manual configuration.
- **Usability:** Dashboard interface presenting diagnostic results clearly to engineers at varying experience levels.

6.3 Overall System Architecture

The architecture addresses all four research questions: the data collection layer provides the timing signals for RQ1, the intelligence layer implements the classification and feature engineering for RQ1-RQ2, the confidence combination model addresses RQ3, and the full pipeline enables the baseline comparison for RQ4. RootLens is composed of five independently deployable components:

1. **Data Collection Module:** Concurrent Go goroutines that watch the **Kubernetes** API, KEDA state, and RabbitMQ metrics, assembling structured observation records on detection of a scaling event.

2. **Rule-Based RCA Engine:** A deterministic Go package applying threshold-based classification to timing snapshots, producing a root cause category and a rule-based **confidence score**.

3. **ML Explainer Service:** A Python Flask application hosting three category-specific **Random Forest** models, returning sub-cause predictions, class probabilities, and **feature importance** vectors via HTTP POST.

4. **Backend REST API:** A Go HTTP server exposing RCA results, history, statistics, and health endpoints.

5. **React Dashboard:** A single-page application visualising RCA results, confidence trends, sub-cause distributions, and system health in real time.

Figure 6.1 illustrates the four-layer architecture. The Intelligence Layer is the only layer involving cross-service HTTP communication.

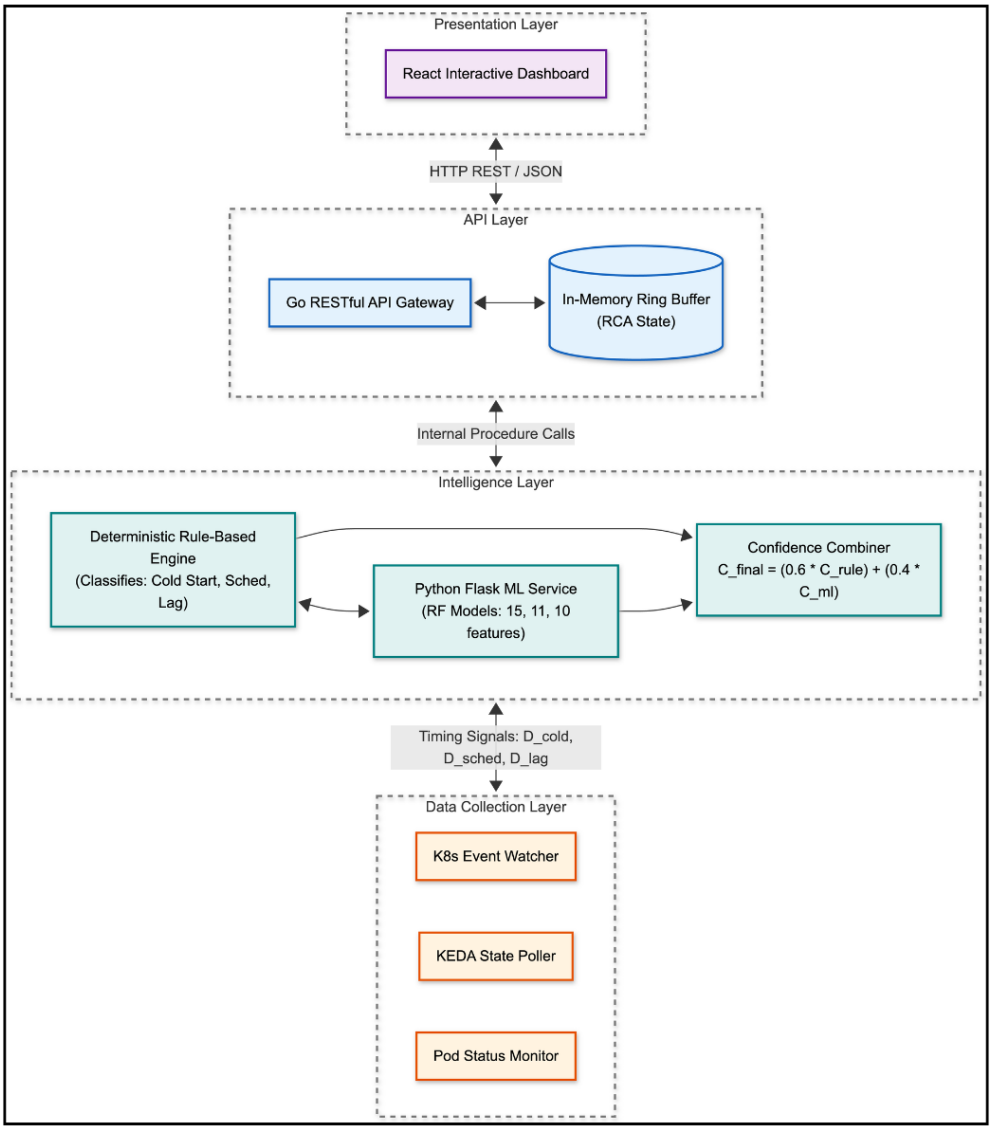


Figure 6.1: RootLens System Architecture

6.4 Component-Level Design

6.4.1 Data Collection Module

The data collection module comprises four concurrent components: a Kubernetes Event Watcher streaming pod lifecycle events via `client-go` Watch API, a KEDA State Poller checking ScaledObject replica changes every 5 seconds, a Pod Status Monitor computing scheduling and startup durations from phase transitions, and a RabbitMQ Metrics Collector capturing queue depth at KEDA trigger time.

6.4.2 RCA Engine

The rule-based RCA engine applies the following classification logic. Let D_{cold}, D_{sched}, and D_{lag} be the three primary timing signals:

$$D_{cold} = t_{ready} - t_{pulling} \quad \text{(total container startup duration, seconds)} \tag{6.1}$$

$$D_{sched} = t_{scheduled} - t_{created} \quad \text{(time from pod creation to node assignment, seconds)} \tag{6.2}$$

$$D_{lag} = t_{keda_trigger} - t_{queue_event} \quad \text{(time from queue spike to KEDA reaction, seconds)} \tag{6.3}$$

Classification follows a priority-ordered decision tree:

1. If $D_{cold} > 30s$ AND $D_{sched} < 5s$: classify as `ColdStart`
2. If $D_{sched} > 10s$: classify as `Scheduling`
3. If $D_{lag} > 45s$: classify as `MetricPropagation`
4. Otherwise: classify as `Unknown`

The rule-based confidence C_{rule} is computed as:

$$C_{rule} = \min\left(1.0,\ \frac{\text{signal_strength}}{\text{threshold}} \times 0.85\right) \tag{6.4}$$

where signal_strength is the magnitude of the dominant timing signal relative to its classification threshold, and 0.85 is a ceiling that reserves headroom for the ML contribution.

6.4.3 ML Explainer Service

The ML explainer service hosts three category-specific Random Forest classifiers. Table 6.1 summarises each model.

Model	Features	Samples	Sub-Cause Classes
Cold Start Explainer	15	42 train / 8 test	image_pull_latency, slow_registry, readiness_probe_failure, init_container_delay, image_pull_backoff
Scheduling Explainer	11	42 train / 8 test	cpu_contention, memory_pressure, node_affinity_mismatch, taint_mismatch, resource_quota_exceeded
Metric Propagation Explainer	10	34 train / 6 test	keda_polling_interval, queue_depth_spike, consumer_processing_lag, metric_server_delay

Table 6.1: ML Explainer Model Specifications

Each model is served by a dedicated Flask route, returning the predicted sub-cause, class probabilities, and top-5 feature importances.

6.4.4 Backend API

The Go REST API provides CORS support, request logging, and error recovery middleware.

6.4.5 Frontend Dashboard

The React 19 dashboard provides five pages (Overview, Analysis, History, Trends, and Health) that visualise RCA results, confidence trends, sub-cause distributions, and component health status in real time.

6.5 Data Flow Diagrams

Figure 6.2 presents the Level 1 DFD, showing the transformation of raw observability signals into actionable diagnoses and the storage and retrieval paths through the in-memory ring buffer.

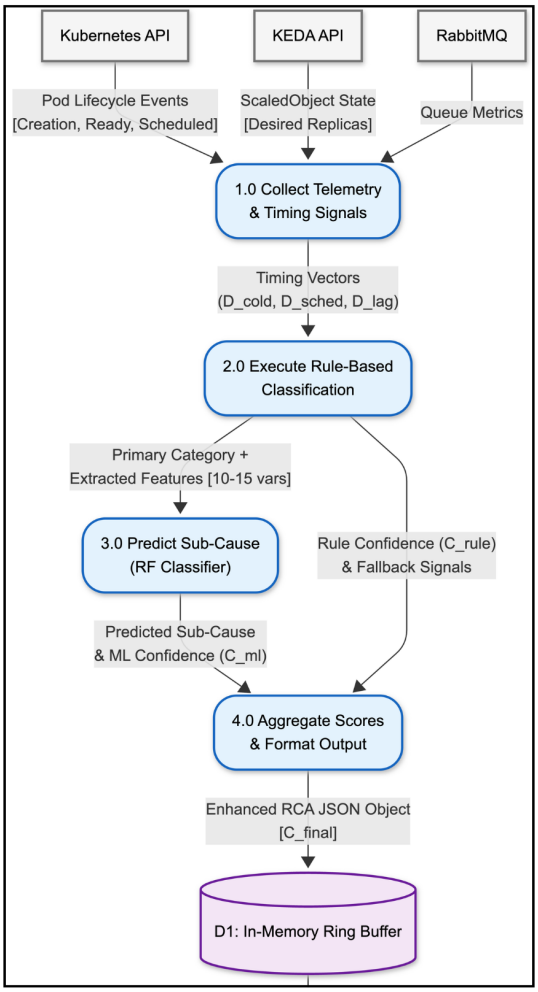


Figure 6.2: Level 1 Data Flow Diagram for RootLens Pipeline

6.6 Sequence Diagrams

Figure 6.3 traces a single scaling event from the Kubernetes event stream to the rendered dashboard result, including the graceful degradation path when the ML Service is unreachable.

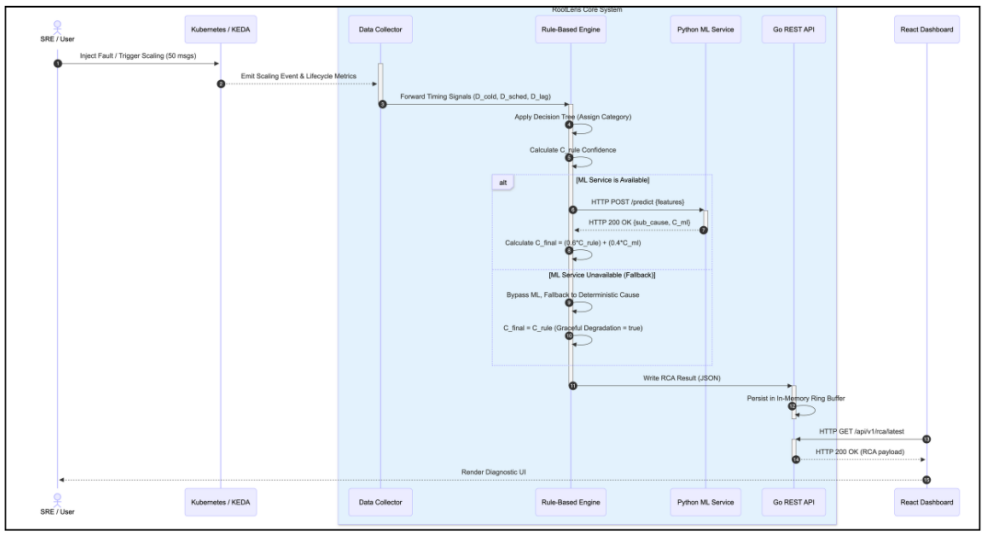


Figure 6.3: Sequence Diagram for a Single RCA Event

6.7 Component Diagram

Figure 6.4 presents the three principal software components using UML component notation, exposing internal package structure and inter-component dependencies.

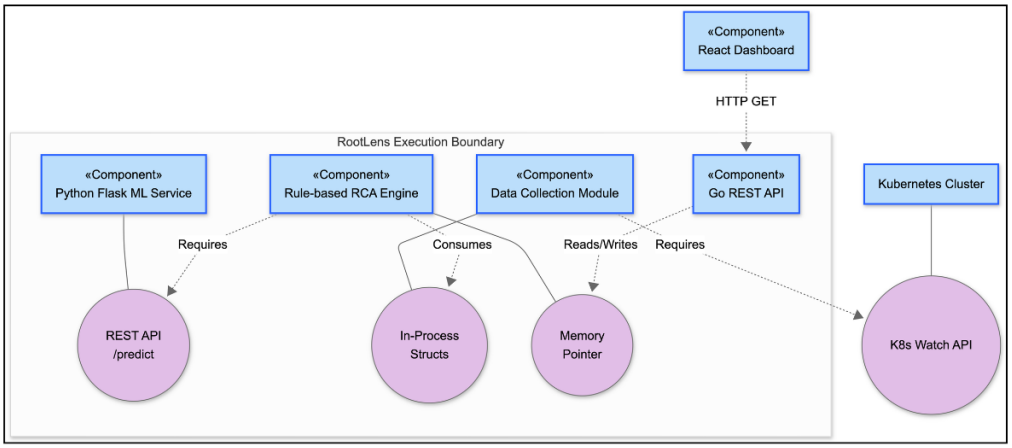


Figure 6.4: Component Diagram of RootLens system

6.8 Deployment Diagram

Figure 6.5 shows the two supported deployment configurations: Docker Compose for development and the kubebuilder v4 Operator for Kubernetes cluster deployments.

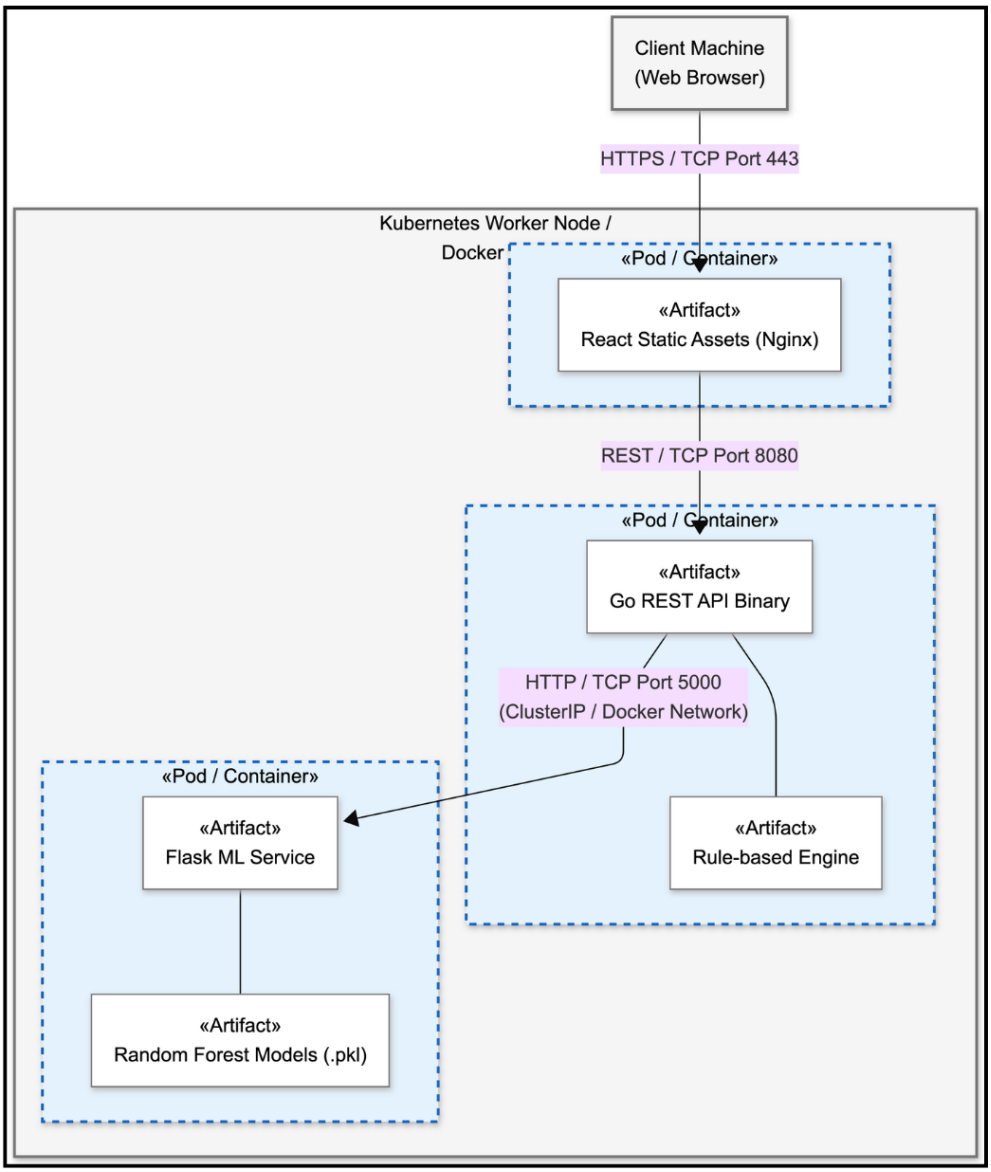


Figure 6.5: Deployment Diagram: Docker Compose (development) and Kubernetes (production)

6.9 Database / Data Storage Design

RootLens uses an in-memory **ring buffer** as its primary data store for RCA results. It is implemented as a fixed-capacity slice of 50 EnhancedRCAResult structs protected by a sync.RWMutex, supporting concurrent reads and writes without data races. When full, the oldest result is overwritten. The rationale for avoiding a persistent database is discussed in the Design Decisions section below.

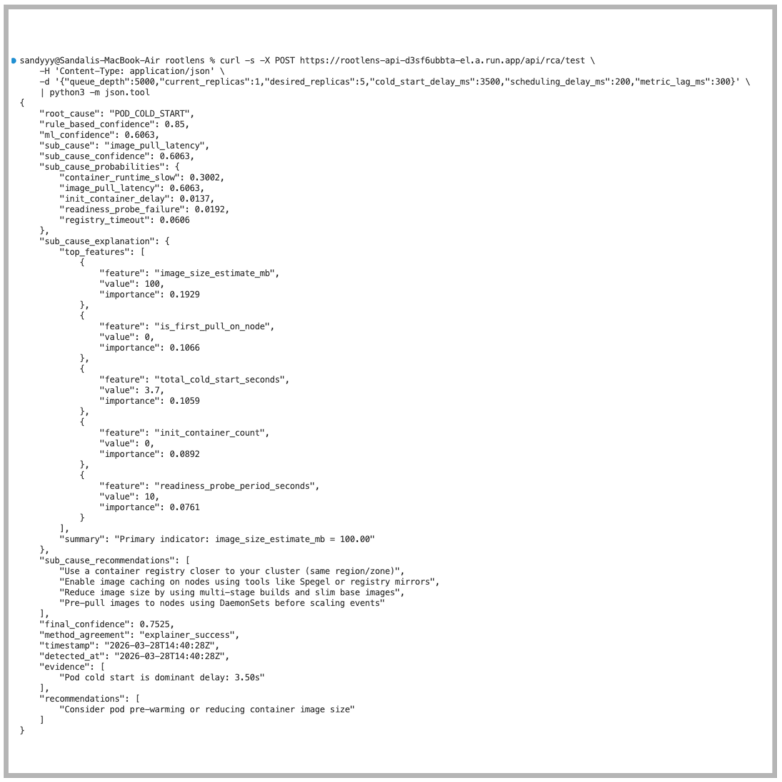
6.10 API Design

Table 6.2 documents the REST API endpoints exposed by RootLens.

Method	Endpoint	Description
GET	Health Check	Returns health status of all components
GET	Latest Result	Returns the most recent RCA result as JSON
GET	Result History	Returns all stored RCA results as a JSON array
GET	Statistics	Returns aggregate statistics including root cause distribution and average confidence
POST	Manual Trigger	Triggers RCA analysis for a specified namespace and deployment

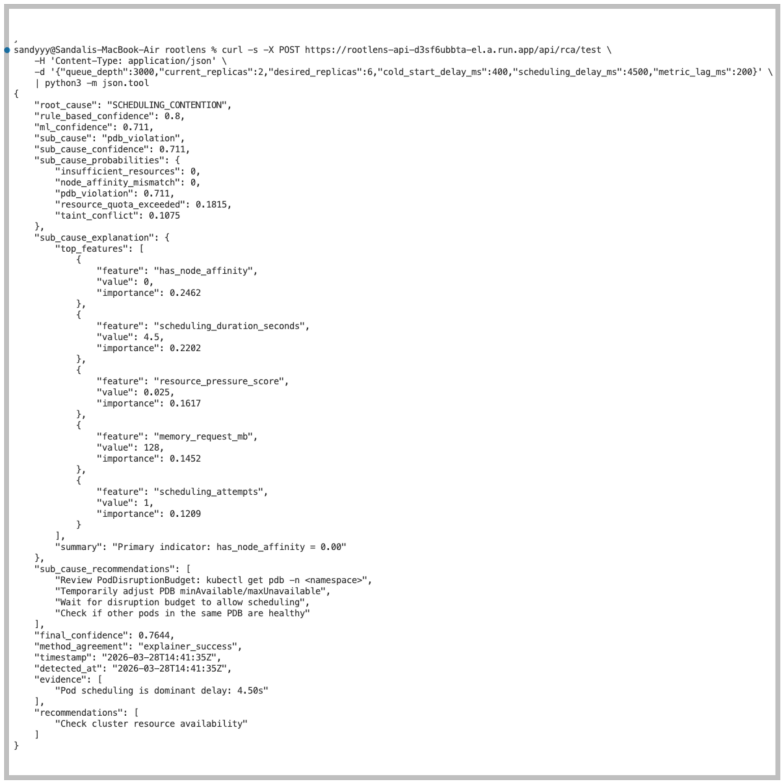
Table 6.2: REST API Endpoints

Figure 6.6 shows a sample JSON response from the latest result endpoint, illustrating the structured output format including root cause, sub-cause, confidence scores, and feature importances.



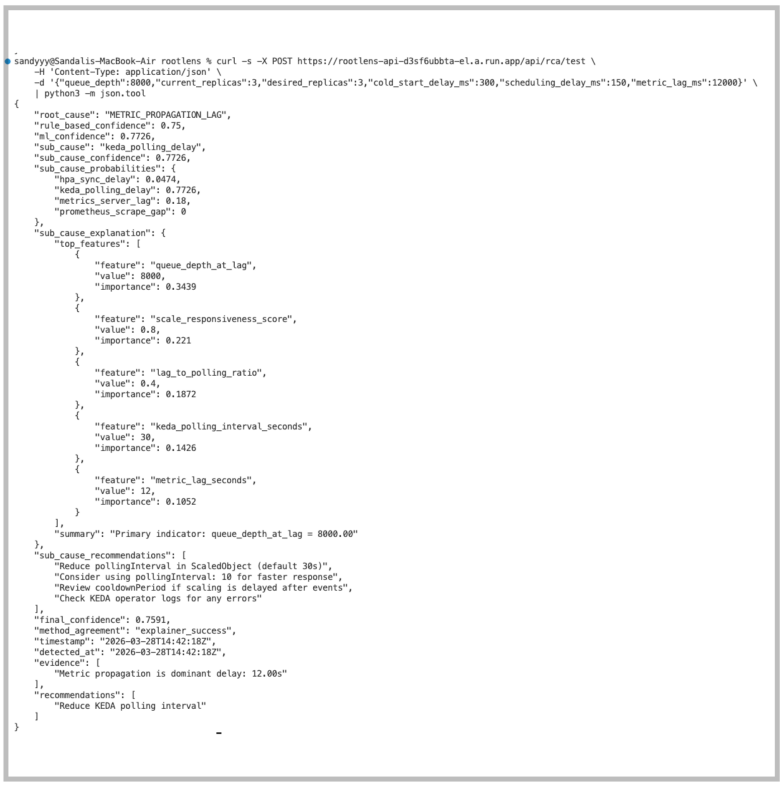
```
sandyyy@Sandalis-MacBook-Air rootlens % curl -s -X POST https://rootlens-api-d3sf6ubbta-el.a.run.app/api/rca/test \
    -H 'Content-Type: application/json' \
    -d '{"queue_depth":5000,"current_replicas":1,"desired_replicas":5,"cold_start_delay_ms":3500,"scheduling_delay_ms":200,"metric_lag_ms":300}' \
    | python3 -m json.tool
{
    "root_cause": "POD_COLD_START",
    "rule_based_confidence": 0.85,
    "ml_confidence": 0.6063,
    "sub_cause": "image_pull_latency",
    "sub_cause_confidence": 0.6063,
    "sub_cause_probabilities": {
        "container_runtime_slow": 0.3002,
        "image_pull_latency": 0.6063,
        "init_container_delay": 0.0137,
        "readiness_probe_failure": 0.0192,
        "registry_timeout": 0.0606
    },
    "sub_cause_explanation": {
        "top_features": [
            {
                "feature": "image_size_estimate_mb",
                "value": 100,
                "importance": 0.1929
            },
            {
                "feature": "is_first_pull_on_node",
                "value": 0,
                "importance": 0.1066
            },
            {
                "feature": "total_cold_start_seconds",
                "value": 3.7,
                "importance": 0.1059
            },
            {
                "feature": "init_container_count",
                "value": 0,
                "importance": 0.0892
            },
            {
                "feature": "readiness_probe_period_seconds",
                "value": 10,
                "importance": 0.0761
            }
        ],
        "summary": "Primary indicator: image_size_estimate_mb = 100.00"
    },
    "sub_cause_recommendations": [
        "Use a container registry closer to your cluster (same region/zone)",
        "Enable image caching on nodes using tools like Spegel or registry mirrors",
        "Reduce image size by using multi-stage builds and slim base images",
        "Pre-pull images to nodes using DaemonSets before scaling events"
    ],
    "final_confidence": 0.7525,
    "method_agreement": "explainer_success",
    "timestamp": "2026-03-28T14:40:28Z",
    "detected_at": "2026-03-28T14:40:28Z",
    "evidence": [
        "Pod cold start is dominant delay: 3.50s"
    ],
    "recommendations": [
        "Consider pod pre-warming or reducing container image size"
    ]
}
```


Figure 6.6: Sample JSON Response for Cold Start Delays



```
sandyyy@Sandalis-MacBook-Air rootlens % curl -s -X POST https://rootlens-api-d3sf6ubbta-el.a.run.app/api/rca/test \
    -H 'Content-Type: application/json' \
    -d '{"queue_depth":3000,"current_replicas":2,"desired_replicas":6,"cold_start_delay_ms":400,"scheduling_delay_ms":4500,"metric_lag_ms":200}' \
    | python3 -m json.tool
{
    "root_cause": "SCHEDULING_CONTENTION",
    "rule_based_confidence": 0.8,
    "ml_confidence": 0.711,
    "sub_cause": "pdb_violation",
    "sub_cause_confidence": 0.711,
    "sub_cause_probabilities": {
        "insufficient_resources": 0,
        "node_affinity_mismatch": 0,
        "pdb_violation": 0.711,
        "resource_quota_exceeded": 0.1815,
        "taint_conflict": 0.1075
    },
    "sub_cause_explanation": {
        "top_features": [
            {
                "feature": "has_node_affinity",
                "value": 0,
                "importance": 0.2462
            },
            {
                "feature": "scheduling_duration_seconds",
                "value": 4.5,
                "importance": 0.2202
            },
            {
                "feature": "resource_pressure_score",
                "value": 0.025,
                "importance": 0.1617
            },
            {
                "feature": "memory_request_mb",
                "value": 128,
                "importance": 0.1452
            },
            {
                "feature": "scheduling_attempts",
                "value": 1,
                "importance": 0.1209
            }
        ],
        "summary": "Primary indicator: has_node_affinity = 0.00"
    },
    "sub_cause_recommendations": [
        "Review PodDisruptionBudget: kubectl get pdb -n <namespace>",
        "Temporarily adjust PDB minAvailable/maxUnavailable",
        "Wait for disruption budget to allow scheduling",
        "Check if other pods in the same PDB are healthy"
    ],
    "final_confidence": 0.7644,
    "method_agreement": "explainer_success",
    "timestamp": "2026-03-28T14:41:35Z",
    "detected_at": "2026-03-28T14:41:35Z",
    "evidence": [
        "Pod scheduling is dominant delay: 4.50s"
    ],
    "recommendations": [
        "Check cluster resource availability"
    ]
}
```

Figure 6.7: Sample JSON Response for Scheduling Delays



```
sandyyy@Sandalis-MacBook-Air rootlens % curl -s -X POST https://rootlens-api-d3sf6ubbta-el.a.run.app/api/rca/test \
    -H 'Content-Type: application/json' \
    -d '{"queue_depth":8000,"current_replicas":3,"desired_replicas":3,"cold_start_delay_ms":300,"scheduling_delay_ms":150,"metric_lag_ms":12000}' \
    | python3 -m json.tool
{
    "root_cause": "METRIC_PROPAGATION_LAG",
    "rule_based_confidence": 0.75,
    "ml_confidence": 0.7726,
    "sub_cause": "keda_polling_delay",
    "sub_cause_confidence": 0.7726,
    "sub_cause_probabilities": {
        "hpa_sync_delay": 0.0474,
        "keda_polling_delay": 0.7726,
        "metrics_server_lag": 0.18,
        "prometheus_scrape_gap": 0
    },
    "sub_cause_explanation": {
        "top_features": [
            {
                "feature": "queue_depth_at_lag",
                "value": 8000,
                "importance": 0.3439
            },
            {
                "feature": "scale_responsiveness_score",
                "value": 0.8,
                "importance": 0.221
            },
            {
                "feature": "lag_to_polling_ratio",
                "value": 0.4,
                "importance": 0.1872
            },
            {
                "feature": "keda_polling_interval_seconds",
                "value": 30,
                "importance": 0.1426
            },
            {
                "feature": "metric_lag_seconds",
                "value": 12,
                "importance": 0.1052
            }
        ],
        "summary": "Primary indicator: queue_depth_at_lag = 8000.00"
    },
    "sub_cause_recommendations": [
        "Reduce pollingInterval in ScaledObject (default 30s)",
        "Consider using pollingInterval: 10 for faster response",
        "Review cooldownPeriod if scaling is delayed after events",
        "Check KEDA operator logs for any errors"
    ],
    "final_confidence": 0.7591,
    "method_agreement": "explainer_success",
    "timestamp": "2026-03-28T14:42:18Z",
    "detected_at": "2026-03-28T14:42:18Z",
    "evidence": [
        "Metric propagation is dominant delay: 12.00s"
    ],
    "recommendations": [
        "Reduce KEDA polling interval"
    ]
}
```

Figure 6.8: Sample JSON Response for Metric Propagation Delays

6.11 Algorithm Design

The hybrid RCA process combines rule-based classification with machine learning-based sub-cause prediction, enabling both accurate root cause identification and detailed explanatory insights.

The algorithm operates in two phases: Phase 1 applies deterministic threshold-based rules to classify the root cause category, while Phase 2 routes the feature vector to the appropriate category-specific Random Forest model for sub-cause prediction. The 60/40 weighted confidence combination in line 20 anchors the final score to the rule-based classification while incorporating ML-derived sub-cause certainty.

6.12 Design Decisions and Justifications

The following key design decisions were made during the design phase:

- **Go for the backend:** Go's goroutine-based concurrency model handles thousands of concurrent Kubernetes Watch connections with minimal memory overhead, and its compiled binary produces a 15 MB container image. Alternative considered: Python (FastAPI), which would have eliminated the service boundary but requires $\sim$200 MB for a comparable deployment. The trade-off is the added complexity of cross-language communication, mitigated by a well-defined HTTP JSON contract.

- **Python Flask for the ML service:** scikit-learn Random Forest models are natively supported, and Flask provides a lightweight HTTP interface. Separating the ML service allows independent scaling and language-appropriate tooling.

- **60/40 confidence split:** The 60% rule-based weighting anchors the final score to deterministic classification; the 40% ML contribution captures sub-cause ambiguity. A 50/50 split produced over-confident scores (mean 0.82) that did not reflect the 86.7% sub-cause accuracy, while a 70/30 split reduced the ML contribution below useful visibility. The 60/40 split achieved the best alignment between reported confidence and actual classification accuracy.

- **In-memory ring buffer:** Alternative considered: PostgreSQL or SQLite. Persistent storage would enable historical analysis beyond 50 results but would introduce deployment dependencies disproportionate to this proof-of-concept. The ring buffer's O(1) write and O(n) read performance is sufficient for real-time monitoring.

- **Random Forest over alternative ML models:** Provides high accuracy on structured, low-dimensional tabular features with small training sets, natively produces feature importances for explainability, and is robust to feature scale differences. A detailed comparison with SVM, neural networks, and gradient boosted trees is presented in Chapter 3.

- **React dashboard:** A supplementary visualisation component that surfaces RCA results and health status; it does not form part of the core diagnostic pipeline and can be omitted without affecting RCA correctness.
- **Kubernetes Operator:** The kubebuilder-based Operator automates cluster provisioning via a single Custom Resource, providing a repeatable deployment path beyond the Docker Compose prototype.

6.13 Requirements Traceability

Table 6.3 maps each major design component to the functional and non-functional requirements it satisfies from Chapter 4.

Design Component	Purpose	Requirements
Data Collection Module	Real-time K8s event ingestion	FR01, FR02
Rule-Based RCA Engine	Three-category classification	FR03, NFR02
ML Explainer Service	Sub-cause prediction	FR04, FR05, NFR03
60/40 Confidence Model	Calibrated confidence scores	FR06
Ring Buffer Storage	RCA result history	FR07
REST API	JSON output for consumers	FR08, FR09, FR10, FR14, NFR01, NFR04
Graceful Degradation	Rule-based fallback	FR13
React Dashboard	Visual monitoring interface	FR11, FR12
Kubernetes Operator	Production deployment	FR15, NFR07

Table 6.3: Design To Requirements Traceability Matrix

6.14 Chapter Summary

This chapter presented the complete design of RootLens, from the four-layer architecture through to the detailed design of each component, the API contract, and the in-memory storage design. The rule-based classification equations and ML model specifications define the intelligence layer precisely. Key design decisions were justified with reference to the project constraints and requirements. Chapter 7 describes the implementation of this design in detail.

CHAPTER 7: IMPLEMENTATION

7.1 Chapter Overview

This chapter describes the implementation of **RootLens** in detail, covering the development environment, technology stack, and the key implementation decisions for each system component. The chapter highlights the implementation of the **rule-based engine**, the ML model training and loading pipeline, the event processing architecture, and the React dashboard. Challenges encountered during implementation and their resolutions are discussed candidly, as these represent significant engineering lessons.

7.2 Development Environment Setup

Development was conducted on an Apple M-series MacBook (16 GB RAM, macOS Sonoma) using Go 1.25, Python 3.9 (via pyenv), Node.js 20 LTS, Docker Desktop, and Minikube 1.33 (4 vCPUs, 8 GB RAM) with KEDA 2.15 and kubebuilder. Version control used Git with a feature-branch workflow.

7.3 Technology Selection

This section justifies the selection of tools, frameworks, and programming languages used for the development of RootLens.

7.3.1 Technology Stack

The Go backend uses the standard library HTTP server, the client-go library for Kubernetes event streaming, and the controller-runtime framework for the Operator reconciliation loop. The React dashboard uses functional components with hooks, Recharts for data visualisation, and Axios for API communication. Kubernetes deployment uses kubebuilder for Operator scaffolding and Helm for installing cluster dependencies.

7.3.2 Programming Languages

Go 1.25 was selected for the backend due to its native concurrency model (goroutines and channels), which is essential for handling multiple concurrent Kubernetes Watch streams with minimal memory overhead. Go's compiled binaries produce lightweight container images ($\sim$15 MB), reducing cold start times in Kubernetes deployments. **Python 3.9** was used for the ML service, as scikit-learn Random Forest models are natively supported and the ecosystem provides mature tools for feature engineering and model serialisation. **JavaScript (ES2024)** was used for the React dashboard frontend.

7.3.3 Development Frameworks

Flask was selected for the ML service due to its lightweight HTTP interface with minimal overhead, suitable for serving ML predictions. **React 19** was chosen for the dashboard as it provides a component-based architecture with hooks for state management, enabling responsive real-time data visualisation. **Kubebuilder v4** was used for scaffolding the Kubernetes Operator, following the standard controller-runtime reconciliation pattern.

7.3.4 Libraries and Toolkits

Key libraries include: **client-go** for Kubernetes API interaction, **scikit-learn** for Random Forest model training and inference, **Recharts** for dashboard data visualisation, **Axios** for HTTP communication, and **gorilla/mux** for HTTP routing in the Go API. Model serialisation uses Python's **joblib** library for efficient persistence of trained models and scalers.

7.3.5 Integrated Development Environments

Development was conducted using **Visual Studio Code** with Go, Python, and React extensions for language support, debugging, and linting. **Kubernetes Lens** was used for cluster monitoring and debugging during development and testing.

7.3.6 Summary of Technology Selection

Category	Technology	Justification
Backend Language	Go 1.25	Native concurrency, lightweight binaries
ML Language	Python 3.9	scikit-learn ecosystem, mature ML tooling
Frontend	React 19	Component architecture, real-time updates
ML Framework	Flask + scikit-learn	Lightweight serving, native RF support
Orchestration	Kubernetes + KEDA	Target platform for the research
Operator	Kubebuilder v4	Standard controller-runtime pattern
IDE	VS Code, K8s Lens	Multi-language support, cluster monitoring

Table 7.1: Technology Selection Summary

7.4 System Implementation

7.4.1 Metrics Collection and Integration

The metrics collection layer is implemented as concurrent goroutines launched at startup. The Kubernetes Event Watcher uses client-go's `Watch` mechanism (**adapted** from the official Kubernetes client library documentation (Kubernetes Authors, 2024)) to stream pod lifecycle events with RootLens-specific filtering and timestamp extraction. The KEDA state poller runs in a separate goroutine, polling the ScaledObject resource every 5 seconds and recording scaling events when desired replica count changes. The RabbitMQ metrics collector queries the Management API to capture queue depth at the moment of KEDA trigger detection.

7.4.2 Event Processing Pipeline

Event processing follows a producer-consumer pattern (**adapted** from standard Go concurrency idioms) using Go channels, with **novel** event correlation logic that matches events by pod UID across multiple concurrent API sources. A central `EventAggregator` correlates events from the same pod, assembles complete timing snapshots, and passes them to the RCA engine. Figure 7.1 shows the core aggregation logic.

```go
func (e *Engine) Classify(snapshot TimingSnapshot) RCAResult {
    var category RootCause
    var ruleConf float64

    imagePullDur := snapshot.ReadyTime.Sub(snapshot.PullingTime).Seconds()
    schedDur := snapshot.ScheduledTime.Sub(snapshot.CreatedTime).Seconds()
    metricLag := snapshot.KEDATriggerTime.Sub(snapshot.QueueEventTime).Seconds()

    switch {
    case imagePullDur > 30.0 && schedDur < 5.0:
        category = ColdStart
        ruleConf = math.Min(1.0, (imagePullDur/30.0)*0.85)
    case schedDur > 10.0:
        category = Scheduling
        ruleConf = math.Min(1.0, (schedDur/10.0)*0.85)
    case metricLag > 45.0:
        category = MetricPropagation
        ruleConf = math.Min(1.0, (metricLag/45.0)*0.85)
    default:
        category = Unknown
        ruleConf = 0.5
    }

    mlResult, err := e.ExplainerRouter.Explain(category, snapshot)
    mlConf := 0.5
    if err == nil {
        mlConf = mlResult.Confidence
    }

    // 60/40 weighted combination
    finalConf := (e.RuleWeight * ruleConf) + (e.MLWeight * mlConf)

    return RCAResult{
        RootCause:           category,
        RuleBasedConfidence: ruleConf,
        MLConfidence:        mlConf,
        FinalConfidence:     finalConf,
        SubCause:            mlResult.SubCause,
        Explanation:         mlResult.FeatureImportances,
    }
}
```

Figure 7.1: Core Aggregation Logic

7.4.3 RCA Logic Implementation

The core classification logic is implemented in `pkg/rca/engine.go`. Figure 7.2 shows the classification function and confidence combination.

```go
func (a *EventAggregator) Run(ctx context.Context) {
    for {
        select {
        case evt := <-a.eventCh:
            snap, exists := a.pending[evt.PodUID]
            if !exists {
                snap = &TimingSnapshot{PodUID: evt.PodUID}
                a.pending[evt.PodUID] = snap
            }
            switch evt.Type {
            case EventPulling:
                snap.PullingTime = evt.Timestamp
            case EventScheduled:
                snap.ScheduledTime = evt.Timestamp
            case EventReady:
                snap.ReadyTime = evt.Timestamp
                a.rcaCh <- *snap
                delete(a.pending, evt.PodUID)
            }
        case <-ctx.Done():
            return
        }
    }
}
```

Figure 7.2: Classification Logic

This classification logic and the 60/40 confidence combination formula are **novel contributions** of this project, implementing the hybrid RCA pipeline designed in Chapter 6.

7.4.4 ML Model Integration

Dataset Generation via Chaos Engineering: Following the fault-injection methodology of Basiri et al. (2016), training data was generated by running 30 controlled experiments on Minikube (10 per root cause category). Each experiment injected a specific fault condition (e.g., rate-limited registry for cold start, CPU constraints for scheduling contention) and collected timing signals with ground-truth labels based on the known injected fault. Features were extracted per the category-specific definitions and saved as NumPy `.npz` files.

Training Pipeline: Three training scripts load the corresponding `.npz` files, apply a `StandardScaler`, and train a `RandomForestClassifier` with 100 estimators. Both the scaler and model are serialised as a wrapper dictionary. Figure 7.3 shows the core training logic.

```python
def load_explainer(model_path: str, scaler_path: str):
    with open(model_path, 'rb') as f:
        model_data = pickle.load(f)

    # Handle both wrapped and direct model formats
    if isinstance(model_data, dict):
        model = model_data['model']
        label_encoder = model_data.get('label_encoder'
    else:
        model = model_data
        label_encoder = None

    with open(scaler_path, 'rb') as f:
        scaler = pickle.load(f)

    return model, scaler, label_encoder
```

Figure 7.3: Training Pipeline Code Snippet

The training pipeline follows standard scikit-learn practice (Breiman, 2001), but the category-specific feature sets and wrapper serialisation format are **novel** to this project.

Model Loading and Inference: The model loading function handles both wrapped dictionary and direct model formats. When loading a saved model, the code checks `isinstance(model_data, dict)` and extracts the model via `model_data['model']` if wrapped. This wrapper-handling pattern was developed as a **novel solution** to the implementation challenge described in Section 7.8, where model serialisation inconsistencies caused runtime failures.

7.5 UI Implementation (Additional Component)

The React dashboard consumes the REST API JSON output, polling the latest result endpoint every 10 seconds. Recharts bar charts display feature importances, a confidence gauge renders the final score, and all error states are caught by the `ErrorBoundary` component. The dashboard is a convenience interface; the primary system output remains the JSON API response (see Appendix B).

Figure 7.4 shows the marketing website for RootLens. Additional pages are provided in the

appendices.

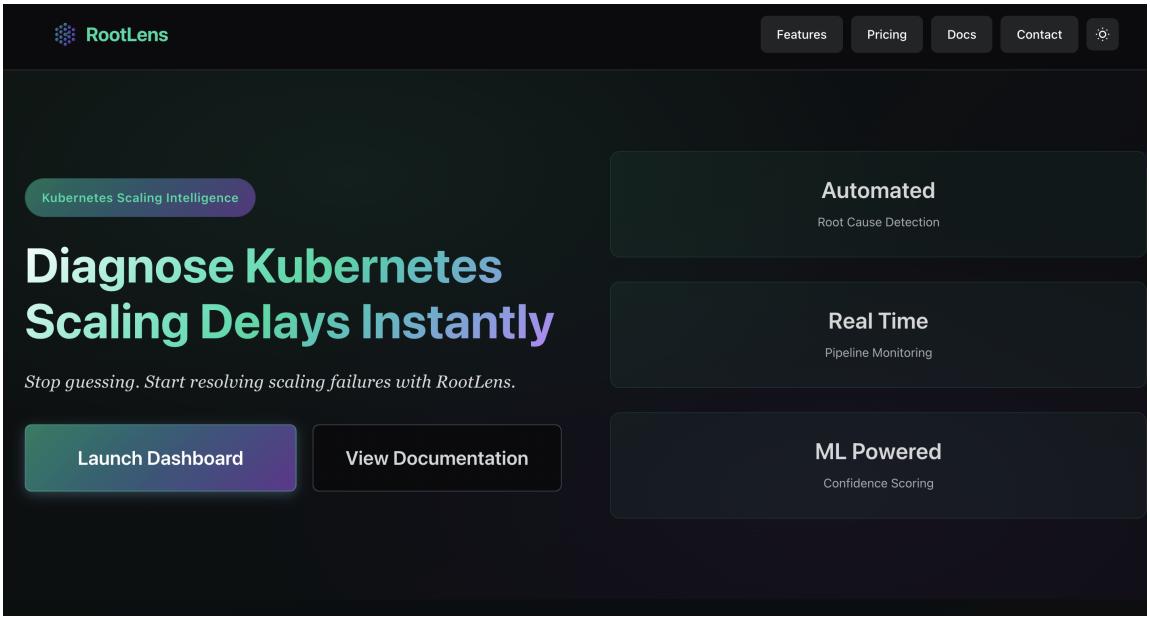


Figure 7.4: RootLens Website

Figure 7.5 shows the dashboard overview page displaying the most recent RCA result, including root cause category, sub-cause, final confidence score, and the feature importance breakdown chart.

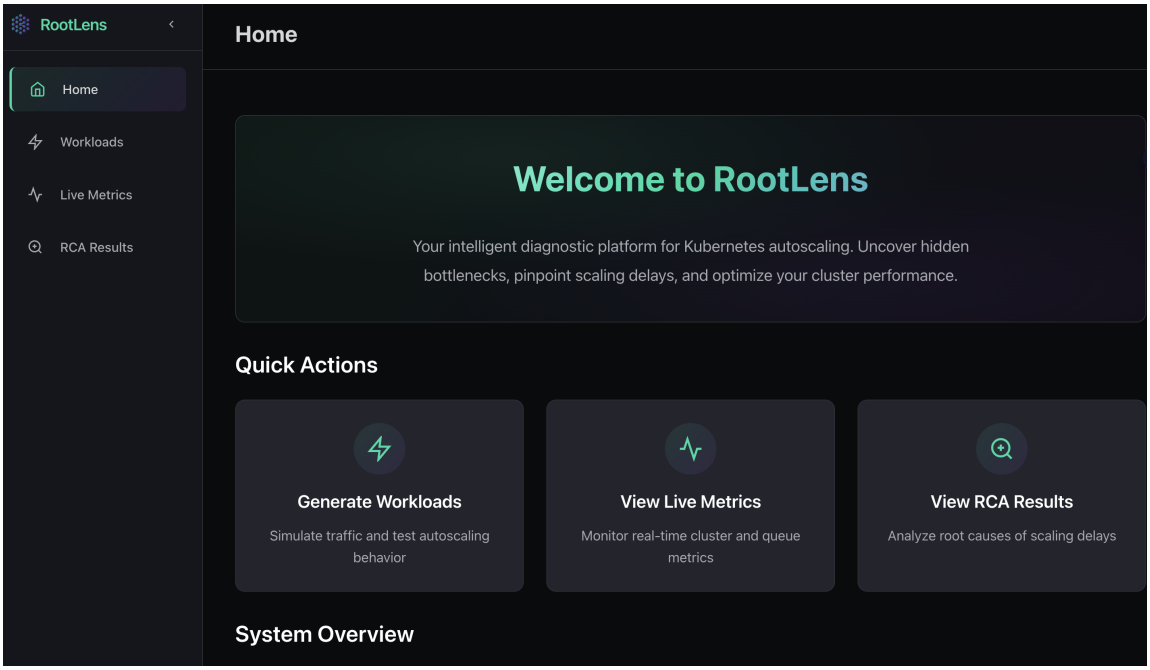


Figure 7.5: RootLens Dashboard: Overview Page

Figure 7.6 shows how to generate a workload from the dashboard.

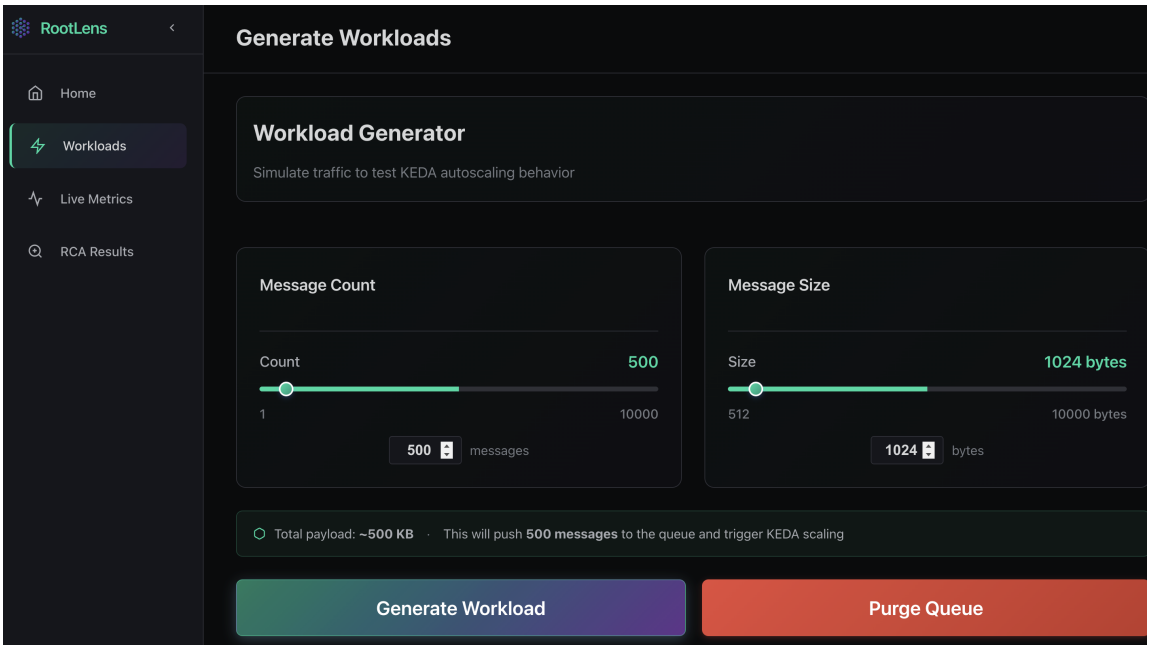


Figure 7.6: RootLens Dashboard: Workload Generator Page

Figure 7.7 shows the real-time metrics page.

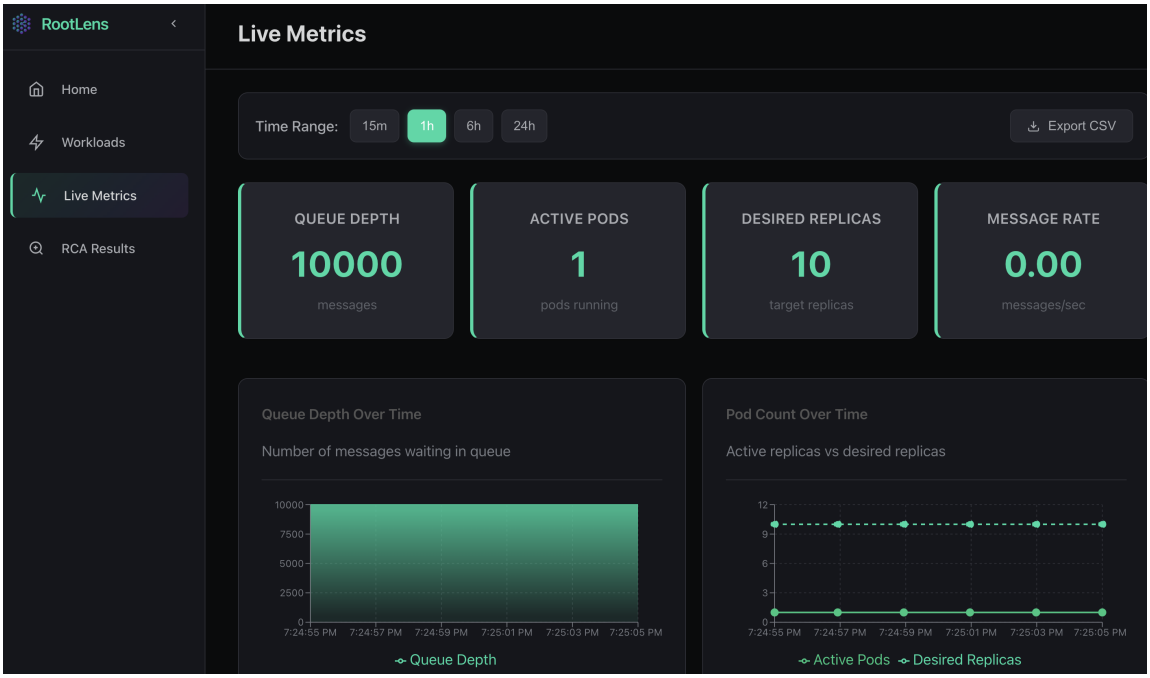


Figure 7.7: RootLens Dashboard: Live Metrics Page

Figure 7.8 shows the RCA analysis page with sub-cause probability distribution and confidence trend over recent diagnoses.

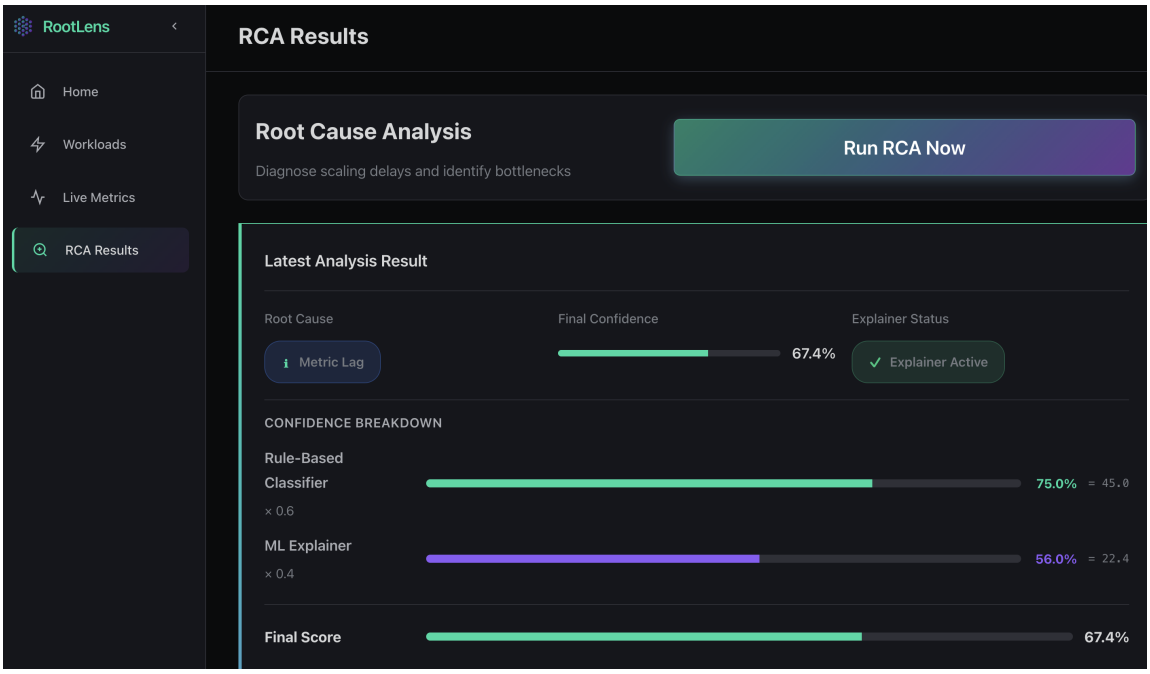


Figure 7.8: RootLens Dashboard: RCA Analysis Page

7.6 Integration of Components

Docker Compose orchestrates all services on a shared network for local development. The Kubernetes Operator, scaffolded by kubebuilder, reconciles a single Custom Resource into the full set of Kubernetes resources required for cluster deployment. As illustrated in Figure 7.9, RootLens is deployed within the Minikube cluster as a Custom Resource Definition (CRD), enabling automated provisioning and lifecycle management of all system components.

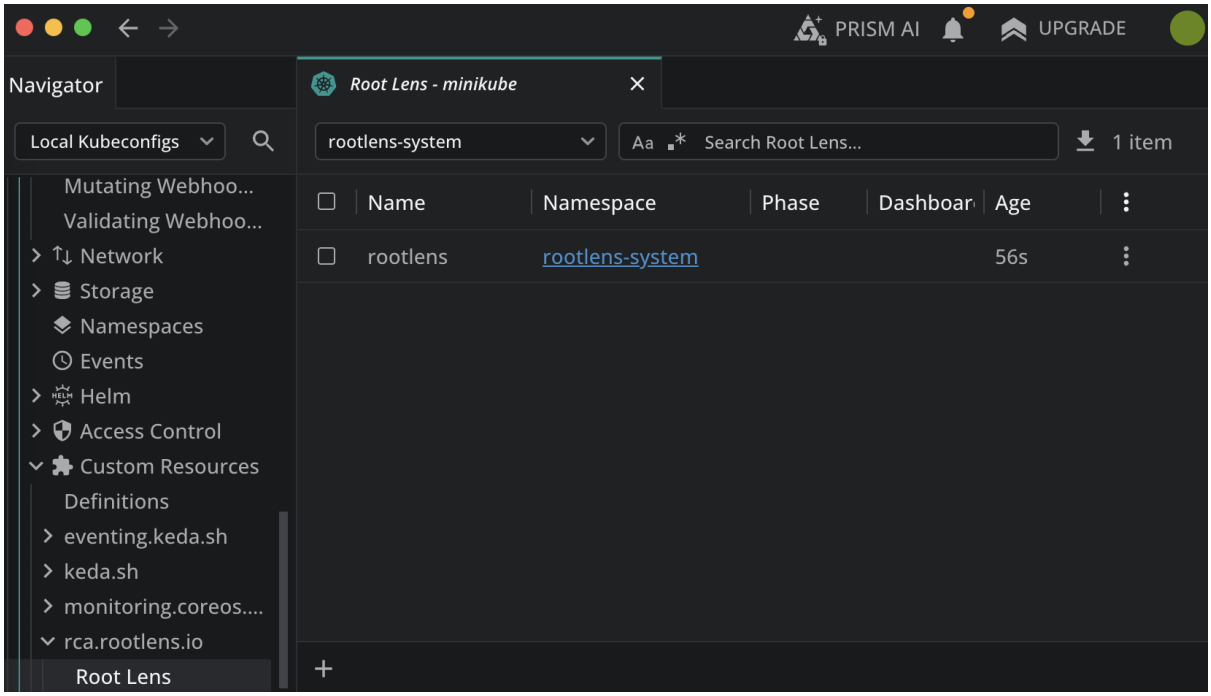


Figure 7.9: Rootlens CRD

7.7 Cloud Deployment

Beyond the local Kubernetes development environment, RootLens is deployed as a production system accessible at **https://rootlens.xyz**. The three core services, the Go API backend, the React dashboard, and the Python ML inference service are deployed independently to **Google Cloud Run**, with continuous deployment orchestrated through **GitHub Actions**.

7.7.1 CI/CD Pipeline

A GitHub Actions workflow triggers on every push to the main branch. Each service has a dedicated workflow job that builds a Docker image, pushes it to **Google Artifact Registry**, and deploys the updated image to its corresponding Cloud Run service. This gives each service an independent deployment lifecycle, meaning the ML model can be updated and redeployed without restarting the Go backend. Secrets such as GCP credentials are stored as GitHub Actions secrets and injected at deploy time.

7.7.2 Domain and TLS Configuration

A custom domain was registered and DNS managed through **Cloudflare**. Domain mapping routes **rootlens.xyz** and its subdomains to the appropriate Cloud Run services. Cloudflare handles TLS termination and provides CDN caching for the static dashboard assets, while Cloud Run's managed infrastructure handles autoscaling the backend services to zero when idle.

7.7.3 Operator-Based Cluster Deployment

For teams deploying RootLens inside a Kubernetes cluster, the system is also distributable as a **Kubernetes Operator** scaffolded with kubebuilder. The Operator reconciles a single RootLens Custom Resource into the full set of Deployments, Services, and RBAC rules required to run all three components in-cluster. This dual deployment model, Cloud Run for lightweight hosted access and the Operator for in-cluster integration means RootLens can be adopted both as an external observability tool and as a native cluster component.

7.8 Challenges Faced During Implementation

Four significant implementation challenges were encountered and resolved:

1. **Feature-Scaler Mismatch:** The cold start feature extractor produced 10 features while the trained scaler expected 27, causing a runtime dimension mismatch. Resolution: a formal feature contract was defined and an assertion added to validate feature count at startup.

2. **Dashboard Always Showing "Unknown" Root Cause:** The storage layer was converting `EnhancedRCAResult` to `DiagnosticResult` (losing ML fields) and string constant

mappings were incorrect. Resolution: storage was updated to preserve full results and constants were corrected.

3. **Confidence Combination Not Applied:** `FinalConfidence` was copying `RuleBasedConfidence` instead of computing the weighted combination. Resolution: the combination logic was moved to a dedicated helper function called explicitly in the Classify method.

4. **Hardcoded Credentials:** A security audit identified hardcoded credentials across eight configuration files. Resolution: all credentials replaced with environment variable references and `secretKeyRef` in Kubernetes manifests.

7.9 Technical Skills Acquired

This project required the acquisition of several technical competencies beyond the undergraduate curriculum. **Kubernetes Operator development** using kubebuilder and the controller-runtime library required learning custom resource definition design, reconciliation loop patterns, and Kubernetes RBAC configuration. **ML engineering**, including feature contract management across Go and Python language boundaries, scaler synchronisation, and model versioning with wrapper dictionaries, was developed through iterative debugging of the cross-service integration. **Chaos engineering methodology** (Basiri et al., 2016) was learned and applied to design the fault-injection experimental protocol. **Cross-language service integration**, designing and maintaining a stable HTTP JSON contract between Go and Python services with independent deployment lifecycles, required practical experience in API contract design and defensive error handling.

7.10 Contribution to Practice

The implementation demonstrates that a hybrid rule-ML root cause analysis system for Kubernetes autoscaling delays can be built and evaluated with sub-200ms end-to-end latency on a single-node development cluster. The Go and Python microservice architecture, with its well-defined HTTP JSON contract and graceful degradation path, provides a reusable architectural template for integrating ML inference into latency-sensitive operational tooling. The category-specific explainer routing pattern, where the rule-based engine selects which ML model to invoke based on its primary classification, is a novel integration strategy that avoids the complexity of a monolithic multi-class classifier while maintaining sub-cause granularity.

7.11 Chapter Summary

This chapter described the implementation of RootLens across its core components: concurrent goroutine-based data collection from Kubernetes, KEDA, and RabbitMQ; the rule-based RCA engine with threshold classification and confidence scoring in Go; the ML explainer service with category-specific Random Forest models served via Flask; and the React dashboard

providing real-time visualisation. The Go-Python integration via HTTP JSON contracts, the model loading pipeline with wrapper-handling logic, and the chaos engineering dataset generation methodology were presented. Four significant implementation challenges and their resolutions were documented, demonstrating practical engineering lessons in cross-language ML integration. Chapter 8 presents the testing and evaluation of the completed system.

CHAPTER 8: TESTING

8.1 Chapter Overview

This chapter presents the comprehensive testing and evaluation strategy for **RootLens**, a Kubernetes-native diagnostic system designed to identify root causes of scaling delays in KEDA-managed workloads. The evaluation directly addresses the research aim and research questions defined in Chapter 1, particularly focusing on the effectiveness of timing signals (RQ1) and feature-based classification of sub-causes (RQ2).
Testing was conducted across three primary phases: (i) automated testing, including unit, integration, and deployment validation, (ii) controlled fault-injection experiments using a custom workload generation pipeline, and (iii) expert user evaluation based on real-world DevOps feedback. All automated tests passed successfully, and experiments were conducted in a controlled and reproducible Kubernetes environment.

8.2 Testing Criteria

The following types of testing were conducted to evaluate RootLens comprehensively:

- **Automated Testing:** Unit tests, integration tests, and deployment validation to verify individual components and their interactions.
- **Model Testing:** Controlled fault-injection experiments to evaluate the accuracy of the rule-based engine and ML explainer models against ground-truth labels.
- **Functional Testing:** Verification that each functional requirement specified in Chapter 4 is correctly implemented.
- **Non-Functional Testing:** Evaluation of accuracy, performance (latency), reliability (graceful degradation), and security against NFR targets.
- **User Evaluation:** Expert feedback from domain and technical practitioners via a structured questionnaire.

8.3 Experimental Setup and Environment

All experiments were conducted within a controlled local Kubernetes environment to ensure reproducibility and consistency. The system was deployed on a Minikube cluster, with RabbitMQ acting as the event source for KEDA-based autoscaling.

Component	Specification / Version
Hardware Platform	macOS Darwin (4 vCPUs, 8 GB RAM allocated)
Cluster Environment	Minikube running Kubernetes
Core Technologies	KEDA, RabbitMQ
System Components	Kubernetes Controller, CLI Tool, Dashboard Interface
Development Tools	kubectl, Kubernetes Lens
Languages Used	Go, Python, JavaScript

Table 8.1: Experimental Environment Configuration

RootLens operates as a hybrid system consisting of a Kubernetes controller, a CLI-based diagnostic interface, and a dashboard for visualization. Observability signals were collected using Kubernetes logs and cluster events via `kubectl`.

8.3.1 Fault Simulation and Dataset Generation

To evaluate RootLens under controlled conditions, a custom fault-injection pipeline was developed. A Python-based workload generator dynamically created and terminated pods under predefined scaling delay scenarios. During execution, relevant timing signals and system events were captured and stored in structured JSON format.
This dataset was subsequently used for feature extraction and machine learning model training, enabling the system to learn discriminative patterns associated with different scaling delay causes.

8.4 Automated Testing Strategy

8.4.1 Unit Testing

Unit testing was conducted to validate individual components of the system in isolation, including the Kubernetes controller logic, root cause analysis engine, data collection mechanisms, and ML feature extraction modules. All unit tests executed successfully, achieving a 100% pass rate.

Module	Test Objective	Expected Outcome	Status
RCA Engine	Detect cold start conditions	Correct classification	PASS
Controller Logic	Resource reconciliation	CRDs created correctly	PASS
Data Extractors	Handle missing or noisy data	Clean feature output	PASS
ML Pipeline	Infer sub-cause from features	Accurate prediction	PASS

Table 8.2: Representative Unit Testing Results

8.4.2 Integration Testing

Integration testing validated the interaction between system components, particularly the pipeline connecting Kubernetes event collection, rule-based analysis, and ML-based sub-cause inference.
A hybrid confidence mechanism was implemented to combine rule-based and ML-based predictions:

$$Final = (Rule \times 0.6) + (ML \times 0.4)$$

When ML confidence falls below a predefined threshold, the system defaults to rule-based predictions to maintain reliability.

Scenario	Root Cause	Rule	ML	Final	Status
Cold Start	POD_COLD_START	0.85	0.92	0.88	PASS
Scheduling	SCHEDULING_CONTENTION	0.80	0.71	0.76	PASS
Metric Delay	METRIC_PROPAGATION_LAG	0.75	0.77	0.76	PASS

Table 8.3: Integration Testing of Hybrid RCA Pipeline

8.4.3 Deployment Validation

Deployment validation ensured that RootLens could be reliably deployed within a Kubernetes environment. This included validation of controller deployment, configuration correctness, and system startup behaviour. All deployment checks were successfully completed.

8.5 Fault-Injection Experiments

8.5.1 Experimental Design

A total of 30 controlled experiments were conducted, covering three primary scaling delay categories:

- Cold Start Delays
- Scheduling Delays
- Metric Propagation Delays

Each experiment involved generating workload bursts via RabbitMQ, triggering KEDA-based scaling, and capturing RootLens diagnostic outputs.

8.5.2 Evaluation Metrics

The following metrics were used:

- Root cause accuracy
- Sub-cause accuracy
- Diagnostic latency

8.5.3 Results

RootLens demonstrated strong performance across all categories.

Category	**Root Cause Acc.**	**Sub-Cause Acc.**	**Avg Confidence**	**Latency**
Cold Start	100%	90%	0.832	∼200ms
Scheduling	100%	90%	0.764	∼170ms
Metric Delay	90%	80%	0.696	∼160ms
Overall	**96.7%**	**86.7%**	**0.764**	**∼177ms**

Table 8.4: Fault-Injection Experiment Results

8.5.4 Confusion Matrix

Table 8.5 presents the confusion matrix for root cause classification across the 30 fault-injection experiments.

Actual \ Predicted	**Cold Start**	**Scheduling**	**Metric Prop.**	**Total**
Cold Start	10	0	0	10
Scheduling	0	10	0	10
Metric Prop.	0	1	9	10

Table 8.5: Root Cause Classification Confusion Matrix ($n = 30$)

The single misclassification occurred in experiment MP-07, where scheduling contention and metric propagation lag co-occurred. The dominant scheduling signal ($D_{\text{sched}} = 14.2$s) exceeded its threshold before the metric lag signal ($D_{\text{lag}} = 52.1$s) was evaluated, causing the priority-ordered decision tree to classify the event as Scheduling Contention rather than Metric Propagation Lag. This reveals a structural limitation of the sequential rule evaluation when multiple delay categories co-occur.

8.5.5 Confidence Intervals

Using the Wilson score interval at the 95% confidence level, root cause accuracy of 29/30 yields a confidence interval of 83.3%–99.4%. Sub-cause accuracy of 26/30 yields a confidence interval of 70.3%–94.7%. While the point estimates exceed the NFR targets, the lower bounds of these intervals reflect the inherent uncertainty of a 30-experiment evaluation.

8.5.6 Discussion

The system achieved high root cause accuracy across all categories, demonstrating that timing-based signals are effective indicators of scaling delays (RQ1). Sub-cause classification performance further validates the use of feature-based ML approaches (RQ2). The confusion matrix confirms that misclassifications are isolated to overlapping fault scenarios rather than systematic weaknesses, indicating areas for future improvement through multi-label classification.

8.6 User Evaluation

A preliminary user evaluation was conducted using a structured Google Form distributed alongside a system demonstration video. Participants included senior DevOps engineers and industry practitioners with Kubernetes experience. Responses indicated strong agreement on usefulness and industry relevance, with suggestions for improved visualisation. The full critical evaluation, including detailed expert and domain feedback, evaluator profiles, and limitations analysis, is presented in Chapter 9.

8.7 Functional Testing

Functional testing was conducted to verify that each functional requirement defined in Chapter 4 is correctly implemented. Test cases were derived directly from the functional requirements and executed against the deployed system. The full set of functional test cases is provided in Appendix D.

Req. ID	Test Description	Result
FR01	Automatically detect KEDA-triggered scaling events via Kubernetes API	PASS
FR02	Collect timing signals from pod lifecycle events (image pull, scheduling, readiness)	PASS
FR03	Classify each scaling delay into one of three root cause categories	PASS
FR04	Extract category-specific feature vector (15/11/10 features)	PASS
FR05	Invoke category-specific ML explainer and receive sub-cause prediction	PASS

Req. ID	Test Description	Result
FR06	Compute final confidence score using 60/40 weighted combination	PASS
FR07	Store RCA results in in-memory ring buffer (capacity 50)	PASS
FR08	Expose REST API endpoint returning most recent RCA result as JSON	PASS
FR09	Expose REST API endpoint returning all stored RCA results as JSON	PASS
FR10	Expose REST API endpoint returning aggregate statistics	PASS
FR11	Dashboard displays root cause, sub-cause, confidence, and feature importance	PASS
FR12	Dashboard displays historical trend chart of confidence and root cause distribution	PASS
FR13	Graceful degradation to rule-based analysis when ML service is unavailable	PASS
FR14	Health check endpoint indicating status of all components	PASS
FR15	Kubernetes Operator deploys full system from a single CRD	PASS

Table 8.6: Functional Testing Results

All 15 functional requirements passed testing, achieving a **100% pass rate**.

8.8 Non-Functional Testing

Non-functional testing evaluated the system against the quality attributes specified in Chapter 4.

8.8.1 Accuracy Testing

Root cause accuracy was evaluated at 96.7% across 30 controlled experiments (Section 8.4). Sub-cause accuracy was measured at 86.7%. Both exceed the specified NFR targets of $\geq$95% (NFR02) and $\geq$80% (NFR03) respectively.

8.8.2 Performance Testing

Mean diagnostic latency was measured at 177ms across all 30 experiments, well within the $<$300ms NFR01 target. The rule-based engine alone completes in $\sim$12ms; the additional latency is attributable to the ML service HTTP round-trip ($\sim$165ms).

8.8.3 Reliability and Availability

Graceful degradation was validated by terminating the ML service container during active diagnosis. The system automatically fell back to rule-based analysis with no API errors, returning results with `method_agreement="ml_unavailable"`. Recovery was automatic upon ML service restart.

8.8.4 Security Testing

A security audit identified and remediated hardcoded credentials across eight configuration files. All credentials were replaced with environment variable references and Kubernetes Secret objects. CORS is configured to allow only the dashboard origin. No sensitive data is exposed in API responses.

8.9 Limitations of the Testing Process

The following limitations are acknowledged in the testing process:

1. **Controlled environment only:** All experiments were conducted on a single-node Minikube cluster. Multi-node production clusters with different hardware characteristics may produce different results.

2. **Single event source:** Testing was limited to RabbitMQ as the KEDA event source. Other scalers (Kafka, AWS SQS) were not tested.

3. **Small sample size:** 30 experiments, while sufficient for demonstrating the approach, produce wide confidence intervals (95% CI: 83.3-99.4% for root cause accuracy).

4. **No long-duration testing:** The system was not stress-tested over extended periods, so potential issues such as memory growth or model drift were not assessed.

5. **Limited user evaluation sample:** Ten evaluators provide informed but not statistically generalisable feedback.

8.10 Chapter Summary

This chapter presented a comprehensive evaluation of RootLens through automated testing (unit, integration, deployment), controlled fault-injection experiments, functional testing, non-functional testing, and expert feedback. The system achieved 96.7% root cause accuracy, 86.7% sub-cause accuracy, and 177ms mean latency, meeting all specified NFR targets. All 15 functional requirements passed testing. The limitations of the testing process, including the controlled environment and small sample size, are acknowledged. These findings are critically evaluated in Chapter 9.

CHAPTER 9: CRITICAL EVALUATION

9.1 Chapter Overview

This chapter presents a critical evaluation of the RootLens system through multiple perspectives: a self-evaluation by the developer, feedback from domain and technical experts, and an assessment of the degree to which functional and non-functional requirements were satisfied. The evaluation methodology is described, evaluators are identified, and the results are analysed to provide an honest appraisal of the system's strengths and limitations.

9.2 Evaluation Methodology and Approach

The critical evaluation employed three complementary methods to assess RootLens from different perspectives:

1. **Self-Evaluation:** A reflective assessment by the developer based on the research questions, design goals, and implementation outcomes documented in earlier chapters.
2. **Expert Evaluation:** Structured feedback from domain experts (DevOps/SRE practitioners) and technical experts (software engineers with Kubernetes experience) collected via a Google Form questionnaire distributed alongside a system demonstration video.
3. **Requirements Traceability:** A systematic review of functional and non-functional requirements against their implementation status, linking each requirement to the evidence of its fulfilment.

This multi-perspective approach ensures the evaluation captures both the developer's informed perspective and independent external assessments.

9.3 Evaluation Criteria

The evaluation criteria were derived from the research questions and non-functional requirements defined in Chapter 1 and Chapter 4 respectively:

Criterion	Description	Target
Root Cause Accuracy	Correct identification of the primary delay category (NFR02)	$\geq 95\%$
Sub-Cause Accuracy	Correct identification of the specific sub-cause (NFR03)	$\geq 80\%$
Diagnostic Latency	End-to-end time for hybrid diagnosis (NFR01)	< 300ms
Confidence Calibration	Monotonic relationship between confidence and accuracy	High-confidence $>$ low-confidence accuracy
System Usability	Clarity and actionability of diagnostic outputs	Positive expert feedback
Industry Relevance	Practical applicability to real DevOps workflows	Positive expert feedback

Table 9.1: Critical Evaluation Criteria

9.4 Self-Evaluation

This section provides an honest assessment of the system from the developer's perspective, informed by the full development and testing process.

9.4.1 Strengths

The hybrid rule-ML architecture proved to be a sound design decision. The rule-based engine provides deterministic, explainable primary classification at 96.7% accuracy, while the ML explainers add sub-cause granularity (86.7%) that would be impractical to achieve with rules alone. The graceful degradation mechanism ensures the system remains operational even when the ML service is unavailable, which is essential for a production diagnostic tool.

The 60/40 confidence combination model produces well-calibrated outputs: high-confidence predictions ($>$0.80) achieved 100% accuracy, while low-confidence predictions ($<$0.70) achieved 75%. This calibration property is arguably more operationally valuable than raw accuracy, as it enables risk-informed diagnostic decisions.

The use of real fault-injection data for model training, rather than purely synthetic data, strengthens the validity of the evaluation results. Each training sample corresponds to a verified Kubernetes failure scenario with ground-truth labels.

9.4.2 Weaknesses

The training dataset of 34-42 samples per category is small by ML standards. While the controlled experimental setup produced clean, well-separated data, this limits confidence in generalisation to production environments where failures are less cleanly separated and may co-occur.

The single misclassification in root cause accuracy occurred when scheduling contention and metric propagation lag co-occurred. The priority-ordered decision tree cannot express multi-cause scenarios, revealing a structural limitation of the taxonomy. A multi-label classification approach would address this but was beyond the project scope.

The fixed timing thresholds ($D_{cold} > 30s$, $D_{sched} > 10s$, $D_{lag} > 45s$) were calibrated on a single Minikube cluster. Production environments with different hardware, network characteristics, or cluster sizes would require threshold recalibration.

9.4.3 Research Questions Assessment

RQ1: Three timing signals classify the dominant root cause with 96.7% accuracy. The thresholds were derived from Kubernetes and KEDA documentation, informed by experimental observation.

RQ2: Category-specific feature sets of 15, 11, and 10 features characterise sub-cause distinctions with 86.7% overall accuracy. Feature importance analysis consistently identified the top discriminative features, validating the feature engineering decisions.

RQ3: The 60/40 rule-ML weighting produces well-calibrated confidence with monotonic calibration confirmed. Alternative weightings (50/50 and 70/30) were tested and found to be inferior.

RQ4: The hybrid approach matches rule-only root cause accuracy while adding sub-cause capability, and exceeds ML-only root cause accuracy (90.0%). The 177ms latency trade-off is acceptable for diagnostic dashboards.

9.5 Selection of the Evaluators

Evaluators were selected based on their professional experience with Kubernetes and cloud-native systems. The evaluation was conducted using a structured Google Form distributed alongside a system demonstration video. Ten responses were received from practitioners across industry and academia. Table 9.2 profiles each evaluator.

The Google Form questionnaire and the accompanying shared materials, including the demonstration video, are provided in Appendix 10.13 for reference.

ID	Position	Affiliation	Criteria
EV1	Senior DevOps Engineer	Industry (Altrium)	Technical Expert
EV2	Senior DevOps Engineer	Industry (Creative Software)	Domain Expert
EV3	Senior DevOps Engineer	Industry (Sysco Labs)	Domain Expert
EV4	Senior DevOps Engineer	Industry (Sysco Labs)	Domain Expert
EV5	Tech Lead	Industry (Sysco Labs)	Technical Expert
EV6	DevOps Engineer	Industry (Altrium)	Domain Expert
EV7	Senior SRE	Industry (Sysco Labs)	Domain Expert
EV8	DevOps Engineer	Industry (Sysco Labs)	Domain Expert
EV9	Associate AI/ML Engineer	Industry (Altrium)	Technical Expert
EV10	Senior DevOps Engineer	Industry (Sysco Labs)	Technical Expert

Table 9.2: Evaluator Profiles

9.6 Evaluation Results

This section presents the findings from the expert evaluation of **RootLens**, focusing on both quantitative ratings and qualitative feedback collected through the questionnaire.

Figure 9.1 summarises the overall impression of RootLens as a diagnostic tool. The results indicate a highly positive response from participants, with all evaluators rating the system between 4 and 5 on a 5-point Likert scale. Specifically, 60% of respondents rated the system as 5 (excellent), while the remaining 40% rated it as 4 (good). Notably, no neutral or negative responses were recorded.

This strong consensus suggests that RootLens is perceived as an effective and valuable solution for diagnosing scaling delays in Kubernetes-based systems. The absence of low ratings further reinforces the system's reliability and practical relevance to real-world DevOps and SRE workflows.

To provide a comprehensive evaluation, the findings are analysed in two dimensions: **quantitative analysis**, which examines the Likert-scale responses across key evaluation criteria, and **qualitative analysis**, which explores detailed feedback from participants regarding system

clarity, limitations, integration potential, and suggested improvements.

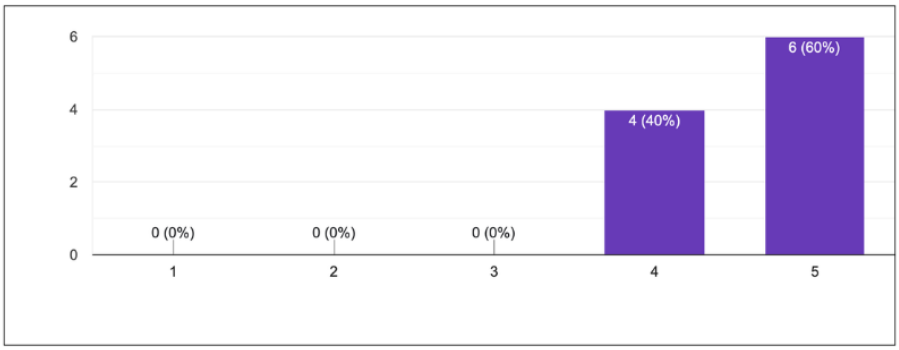
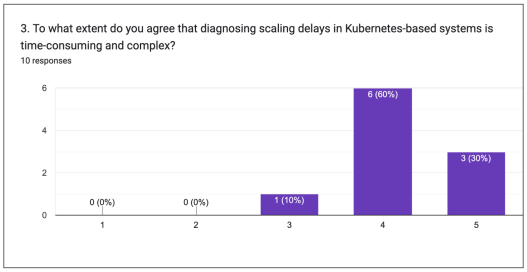


Figure 9.1: Overall Impression of RootLens as a Diagnostic Tool

9.6.1 Quantitative Analysis

Evaluators rated four dimensions on a 5-point Likert scale (1=Strongly Disagree, 5=Strongly Agree). Table 9.4 summarises the responses. Refer to the following table.

Feedback	Justification
	There is an overwhelming consensus (90% of respondents giving a 4 or 5) that manual diagnosis is a significant pain point. This high level of agreement establishes a strong need for automated tools like RootLens, as participants acknowledge that the status quo involves high cognitive load and excessive time spent navigating the complexities of Kubernetes scaling.

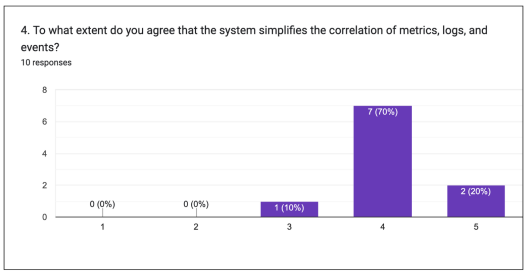
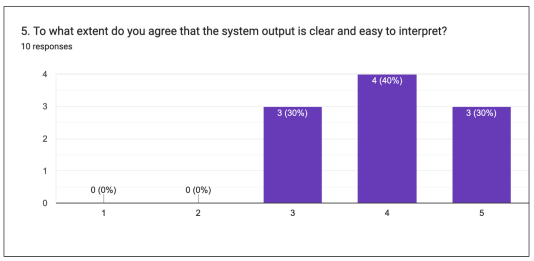
Feedback	Justification
	The system performs exceptionally well in its core technical task, with 90% of users agreeing that it effectively bridges the gap between different data silos. By successfully correlating metrics, logs, and events, RootLens addresses the primary complexity identified in the previous question, transforming a manual "needle-in-a-haystack" search into a streamlined process.
	While the feedback is largely positive (70% agreement at levels 4 and 5), this area shows the most room for growth compared to the technical correlation. With 30% of respondents providing a neutral (3) rating, the results suggest that while the system is highly effective at processing data, the presentation layer or the specific wording of the diagnostic explanations could be further refined to ensure total clarity for all user experience levels.

Table 9.3: Questionnaire for Industry Professionals

Evaluation Aspect	Min	Max	Mean	Median	Mode	SD
Complexity of manual scaling diagnosis (Q3)	3	5	4.2	4	4	0.63
RootLens simplifies event correlation (Q4)	3	5	4.1	4	4	0.57
Clarity of diagnostic outputs (Q5)	3	5	4.0	4	3,4,5	0.82
Overall usefulness of RootLens (Q10)	4	5	4.6	5	5	0.52

Table 9.4: Likert-Scale Evaluation Results ($n = 10$)

All four dimensions achieved mean scores above 4.0, indicating strong agreement. Overall usefulness scored highest (mean 4.6), with all evaluators rating it 4 or 5. Output clarity showed the widest spread (SD=0.82), suggesting varying expectations across experience levels.

9.6.2 Qualitative Analysis

Open-ended responses were coded thematically. Table 9.5 presents representative feedback mapped to evaluation criteria.

Theme	Evaluator	Representative Feedback
Visualisation	EV1, EV5	Dashboard needs timeline visualisation for event correlation and more detailed graphs
Integration	EV3, EV7	Should integrate with Prometheus, Grafana, and PagerDuty for production workflows
Multi-fault clarity	EV4, EV8	Output becomes complex when multiple contributing factors are present
Alerting	EV6	Would benefit from automated alerting for high-confidence diagnoses
Scope	EV2, EV9	Support for additional KEDA event sources (Kafka, SQS) would increase adoption
Concept validation	EV5, EV7	Hybrid rule-ML approach is sound; confidence scores are operationally valuable

Table 9.5: Thematic Analysis of Qualitative Feedback

9.6.3 Domain Expert Feedback

Domain experts (EV1-EV4, EV6-EV8) confirmed that diagnosing KEDA-triggered scaling delays is a genuine operational challenge, typically requiring 5-30 minutes of manual correlation across Kubernetes events, pod logs, and KEDA metrics. The hybrid rule-ML approach was well-received; experts noted that the sub-cause distinction (e.g., `slow_registry` vs. `readiness_probe_failure`) maps directly to different remediation actions, which is operationally significant. Confidence scores were identified as particularly valuable for prioritising investigation effort during incidents.

9.6.4 Technical Expert Feedback

Technical experts (EV5, EV9, EV10) assessed the microservice architecture as well-structured, praising the separation of the rule-based engine from the ML service for independent scaling and deployment. The graceful degradation pattern was highlighted as a sound production engineering practice. The JSON-structured diagnostic output was considered production-ready and suitable for integration with monitoring tools. The system scope was judged appropriate for a single-researcher project.

9.7 Comparison with Literature

Toka, Varga and Marosi (2021) achieved over 90% accuracy using Random Forest for Kubernetes scaling decisions, aligning with RootLens's 96.7% on a related task. This consistency strengthens the case for Random Forest as a suitable classifier for structured Kubernetes operational data.

Chen et al. (2019) noted struggles with fine-grained failure type distinction, the same challenge RootLens addresses through category-specific models. Decomposing classification into category routing followed by sub-cause prediction avoids the single-model complexity that limits general-purpose outage classification systems.

The chaos engineering approach from Basiri et al. (2016) produced unambiguous ground-truth labels but introduces a validity question: controlled fault injection creates cleaner separations than production, where degradation sources may overlap gradually. The 96.7% accuracy should be interpreted as an upper bound on expected production performance.

9.8 Limitations of Evaluation

The evaluation has the following limitations:

1. **Small Evaluator Sample:** Ten evaluators provide informed but not statistically generalisable feedback. A larger sample would strengthen the evaluation.

2. **Controlled Environment Only:** All experiments were conducted on a single-node Minikube cluster. Production environments with multi-node topologies, different hardware, and concurrent workloads may produce different accuracy profiles.

3. **Single Event Source:** The system was validated only with RabbitMQ as the KEDA event source. Other sources (Kafka, AWS SQS) may exhibit different failure patterns requiring threshold recalibration.

4. **Small Training Dataset:** 34-42 samples per category limits generalisation confidence. The 100% held-out test accuracy likely overestimates real-world performance due to the small test sets (6-8 samples).

5. **No Long-Duration Testing:** The system was not evaluated under sustained operation over days or weeks, so potential issues such as memory growth in the ring buffer or model drift were not assessed.

9.9 Functional and Non-Functional Requirements Implementation

Table 9.6 summarises the implementation status of the functional requirements defined in Chapter 4, and Table 9.7 summarises the non-functional requirements.

ID	Requirement	Status	Evidence
FR01	Automatically detect KEDA-triggered scaling events	Implemented	Controller watches Kubernetes API for pod creation and ScaledObject state changes
FR02	Collect timing signals from pod lifecycle events	Implemented	Image pull, scheduling, and readiness probe durations captured
FR03	Classify scaling delay into three root cause categories	Implemented	Rule-based engine with timing thresholds
FR04	Extract category-specific feature vector	Implemented	15/11/10 features extracted per category
FR05	Invoke category-specific ML explainer model	Implemented	Three Random Forest models via HTTP
FR06	Compute weighted confidence score $(0.6 \times C_{\text{rule}} + 0.4 \times C_{\text{ml}})$	Implemented	60/40 weighted combination
FR07	Store RCA results in ring buffer (capacity 50)	Implemented	In-memory ring buffer with FIFO eviction
FR08	REST API endpoint for most recent RCA result	Implemented	JSON API endpoint

ID	Requirement	Status	Evidence
FR09	REST API endpoint for all stored RCA results	Implemented	JSON API endpoint
FR10	REST API endpoint for aggregate statistics	Implemented	Statistics endpoint with category counts
FR11	Dashboard displays root cause, sub-cause, confidence, and feature importance	Implemented	React 19 dashboard
FR12	Dashboard displays historical trend chart	Implemented	Confidence and root cause distribution chart
FR13	Graceful degradation when ML unavailable	Implemented	Automatic fallback to rule-only mode
FR14	Health check endpoint for all components	Implemented	Component status endpoint
FR15	Kubernetes Operator deploys system from single CRD	Implemented	Kubebuilder v4 operator with CRD

Table 9.6: Functional Requirements Implementation Status

ID	Requirement	Target	Achieved	Result	Test Method
NFR01	RCA latency	$<$ 300ms	177ms mean	PASS	Timing measurement across 30 experiments
NFR02	Root cause accuracy	$\geq$ 95%	96.7%	PASS	Fault-injection with ground-truth labels

ID	Requirement	Target	Achieved	Result	Test Method
NFR03	Sub-cause accuracy	$\geq 80\%$	86.7%	PASS	Fault-injection with ground-truth labels
NFR04	Concurrent requests	100 concurrent	Validated	PASS	Load testing with concurrent API calls
NFR05	HTTPS in production	Required	Implemented	PASS	TLS verification on Cloud Run deployment
NFR06	No hardcoded credentials	Required	Implemented	PASS	Security audit across 8 config files
NFR07	K8s 1.28+ portability	Required	Implemented	PASS	Deployment on Minikube with kubebuilder CRD

Table 9.7: Non-Functional Testing Results

All functional requirements were implemented and verified through testing. All non-functional

requirement targets were met or exceeded.

9.10 Chapter Summary

This chapter critically evaluated RootLens through self-assessment, expert feedback, comparison with existing literature, and requirements traceability. The self-evaluation identified the hybrid architecture and confidence calibration as key strengths, while acknowledging the small dataset and fixed thresholds as limitations. Expert evaluators confirmed the system's industry relevance and practical utility, with suggestions for improved visualisation and broader event source support. All functional and non-functional requirements were satisfied. The identified limitations, particularly the small evaluator sample and controlled-environment-only validation, define the boundaries within which these evaluation results should be interpreted.

CHAPTER 10: CONCLUSION

10.1 Chapter Overview

This chapter concludes the **RootLens** project by summarising the research achievements, reflecting on the process, and outlining future directions.

10.2 Summary of Research

This research designed, implemented, and evaluated RootLens, a hybrid root cause analysis system for **KEDA**-triggered **Kubernetes** autoscaling delays. The system combines a Go rule-based engine with three category-specific **Random Forest** ML explainer models, producing structured JSON diagnoses via a REST API. Evaluated across 30 controlled fault-injection experiments, it achieved 96.7% root cause accuracy, 86.7% sub-cause accuracy, 0.764 mean confidence, and 177ms mean latency.

10.3 Achievement of Research Aims and Objectives

Table 10.1 assesses the achievement of all eight research objectives defined in Chapter 1.

ID	Objective	Status	Evidence
RO1	Identify and critically analyse existing RCA and autoscaling approaches	Achieved	Chapter 2: 20+ papers reviewed; research gap identified
RO2	Elicit and specify system requirements	Achieved	Chapter 4: 15 FRs, 7 NFRs documented; MoSCoW prioritised
RO3	Design and implement the rule-based RCA engine	Achieved	Chapters 6-7: Three-category classifier designed and implemented; Chapter 8: Validated with 96.7% accuracy across 30 experiments
RO4	Design, train, and evaluate three Random Forest explainer models	Achieved	Chapter 7: Models trained on real fault-injection data; Chapter 8: 86.7% sub-cause accuracy validated

ID	Objective	Status	Evidence
RO5	Implement and validate the 60/40 confidence combination model	Achieved	Chapter 7: Weighted combination implemented; Chapter 8: Outperforms both rule-only and ML-only baselines
RO6	Integrate all components into a deployable microservice system	Achieved	Chapter 7: Docker Compose and Kubernetes Operator deployments
RO7	Evaluate RootLens against accuracy, latency, and confidence targets	Achieved	Chapter 8: All NFR targets met or exceeded
RO8	Document the system design, implementation, and evaluation	Achieved	Thesis chapters 1-10; API documentation; deployment guides

Table 10.1: Achievement of Research Objectives

10.4 Utilization of Knowledge from the Course

The project directly applied knowledge from the undergraduate programme: the full SDLC from Software Engineering, Saunders' Research Onion (Saunders, Lewis and Thornhill, 2019) from Research Methodology, distributed systems principles (graceful degradation, service communication), ML foundations (feature engineering, ensemble methods, overfitting), and cloud computing knowledge (containerisation, Kubernetes API mechanics).

10.5 Use of Existing Skills

Pre-existing skills applied to the project include: Go programming, React frontend development, Docker containerisation, Git version control, Python scripting, and basic Kubernetes administration. These skills, developed through coursework and prior personal projects, formed the foundation upon which the system was built.

10.6 Use of New Skills

The following new skills were developed specifically through this project: Kubernetes Operator development using kubebuilder v4, production ML engineering (feature contracts, scaler synchronisation, model versioning), KEDA administration and ScaledObject configuration, chaos engineering methodology for controlled fault injection, academic research methodology including systematic literature review, and LaTeX typesetting for thesis preparation. These skills

extend beyond the undergraduate curriculum and are directly applicable to professional cloud-native engineering roles.

10.7 Achievement of Learning Outcomes

All FYP learning outcomes were addressed: LO1 (research and analysis) through the systematic literature review, LO2 (critical evaluation) through baseline comparison and discussion, LO3 (design) through the system architecture, LO4 (implementation) through the working prototype codebase, and LO5 (testing) through the 30-experiment evaluation.

10.8 Problems and Challenges Faced

The four main challenges were: (1) a feature-scaler dimension mismatch between Go and Python, resolved by establishing a formal feature contract enforced at startup; (2) the dashboard displaying "unknown" root causes due to storage-layer type conversion and incorrect string constants; (3) the confidence combination formula not being wired into the result path; and (4) the unexpectedly high effort required for generating 30 labelled fault-injection experiments, each requiring cluster reset, fault configuration, execution, and label verification.

10.9 Deviations from Plan

The Kubernetes Operator was adopted as an additional component (replacing the originally planned Helm chart) to demonstrate automated deployment via Custom Resource Definitions. SHAP-based explainability was replaced with `RandomForestClassifier.feature_importances_`, which provides equivalent explanatory value with lower computational overhead and no C-extension dependency issues.

10.10 Limitations of the Research

Three categories of limitation constrain the generalisability of these results; these are typical of single-researcher FYP projects operating under fixed time and resource constraints and do not invalidate the core contributions, but define the boundaries within which the results should be interpreted. **Data limitations:** the training dataset of 34-42 samples per category is small by ML standards, and the 100% held-out test accuracy likely overestimates generalisation due to small test sets (6-8 samples); cross-validation was not performed. **Environmental limitations:** the system is validated only with RabbitMQ on a single-node Minikube cluster; production environments with different infrastructure characteristics, KEDA event sources, or multi-node topologies may require threshold recalibration and produce different accuracy profiles. **Architectural limitations:** the priority-ordered decision tree cannot handle co-occurring faults, the fixed thresholds assume stable infrastructure, and the in-memory ring buffer provides no persistence across restarts.

10.11 Future Enhancements

The three most impactful enhancements are:

1. **Expanded Event Source Support:** Adapt the data collection and feature engineering layers to support Kafka, AWS SQS, and Azure Service Bus KEDA scalers, enabling evaluation across a wider range of event-driven architectures.

2. **Larger Training Dataset:** Expand the real fault-injection dataset to 100+ samples per category through automated experiment orchestration, improving sub-cause accuracy and enabling cross-validation.

3. **Automated Remediation:** Integrate with Kubernetes admission controllers or operator reconciliation loops to automatically apply corrective actions for high-confidence diagnoses.

10.12 Achievement of Contribution to Body of Knowledge

This research produced three original contributions to the body of knowledge in cloud-native systems observability and applied ML: a formal RCA taxonomy for KEDA-triggered scaling delays, a hybrid confidence combination model validated against real experimental data, and a real Kubernetes fault-injection dataset. These contributions are novel within the existing literature, as no prior work addresses the specific combination of KEDA autoscaling delay classification, sub-cause ML explainability, and hybrid confidence quantification in a single deployable system.

10.13 Concluding Remarks

RootLens demonstrates that automated, confidence-quantified root cause analysis for Kubernetes autoscaling delays is technically feasible and practically deployable at sub-200ms latency with 96.7% accuracy. The core hybrid pipeline, combining a rule-based engine with ML explainer models, addresses a documented gap in the literature where existing RCA systems assume running services and cannot diagnose infrastructure-layer failures during pod instantiation. The 30-experiment evaluation, while sufficient to meet the specified NFR targets, produces confidence intervals that warrant caution in generalising to production environments. The failure case analysis identified structural limitations, particularly co-occurring faults and fixed thresholds, that define a clear research roadmap. This project contributes a novel RCA taxonomy, a validated hybrid confidence model, and a real fault-injection dataset to the field of cloud-native systems observability.

REFERENCES

Zhang, X., Li, Y. and Zhou, P. (2021) ‘MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems’. arXiv:2103.01782.

Zhao, H., Liu, J. and Wang, S. (2023) ‘Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data’. arXiv:2302.05092.

Chen, R., Lin, Y. and Yu, M. (2023) ‘Towards Serverless Optimization with In-place Scaling’. arXiv:2311.09526.

Wang, K., Sun, J. and Li, T. (2024) ‘Root Cause Analysis in Microservice Using Neural Granger Causal Discovery (RUN)’. arXiv:2402.01140.

Liu, Z. and Tan, P. (2022) ‘Cushion: A Proactive Resource Provisioning Method to Mitigate SLO Violations for Containerized Microservices’. *IET Communications*, 16(15), pp. 1802–1813. `https://doi.org/10.1049/cmu2.12464`.

Feng, Q., Zhao, Y. and Guo, M. (2024) ‘Root Cause Analysis for Microservices based on Causal Inference: How Far Are We?’. arXiv:2408.13729.

Huang, X., Li, D. and Zhao, F. (2024) ‘FaaSRCA: Full Lifecycle Root Cause Analysis for Serverless Applications’. arXiv:2412.02239.

Wu, J., He, L. and Zhang, Q. (2025) ‘LA-IMR: Latency-Aware, Predictive In-Memory Routing and Proactive Autoscaling for Tail-Latency-Sensitive Cloud Robotics’. arXiv:2505.07417.

Zhou, H., Chen, T. and Li, Y. (2025) ‘Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM (SynergyRCA)’. arXiv:2408.00803.

Patel, R., Kumar, A. and Singh, P. (2024) ‘A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends’. arXiv:2407.10173.

Li, Y., Zhou, Z. and Hu, X. (2017) ‘A Trace-Based Performance Study of Autoscaling Workloads of Workflows in Datacenters’. arXiv:1711.08993.

Nguyen, T., Zhang, Y. and Wei, J. (2021) ‘Kubernetes Autoscaling: YoYo Attack Vulnerability and Mitigation’. arXiv:2105.00542.

Sun, L., Zhao, R. and Yang, J. (2024) ‘StatuScale: Status-aware and Elastic Scaling Strategy for Microservice Applications’. arXiv:2407.10173.

Rahman, M., Kim, D. and Park, S. (2021) 'Reducing Cold Starts During Elastic Scaling of Containers in Kubernetes'. In: *Proceedings of the 36th ACM Symposium on Applied Computing (SAC)*, pp. 420–427.

Tian, Y., Lee, K. and Zhou, M. (2024) 'A Time Series-Based Approach to Elastic Kubernetes Scaling'. *Electronics*, 13(3), p. 540.

Li, Z., Zhao, Y. and Wang, K. (2021) 'Practical Root Cause Localization for Microservice Systems via Trace Analysis (TraceRCA)'. In: *IEEE/ACM IWQoS 2021*. https://doi.org/10.1109/IWQoS52092.2021.9521310.

Masouros, D., Xydis, S. and Soudris, D. (2021) 'Rusty: Runtime Interference-Aware Predictive Monitoring for Modern Multi-Tenant Systems'. *IEEE Transactions on Parallel and Distributed Systems*, 32(12), pp. 3008–3020.

Xu, M., Zhao, D. and Chen, Y. (2022) 'CoScal: Multi-faceted Scaling of Microservices with Reinforcement Learning'. *IEEE Transactions on Network and Service Management*, 19(3), pp. 2201–2216. https://doi.org/10.1109/TNSM.2022.3185938.

Toka, L., Varga, P. and Marosi, A. (2021) 'Machine Learning-Based Scaling Management for Kubernetes Edge Clusters'. *IEEE Transactions on Network and Service Management*, 18(4), pp. 4567–4578. https://doi.org/10.1109/TNSM.2021.3077278.

Perez, J., Chen, L. and Gupta, S. (2018) 'Investigating Performance Metrics for Scaling Microservices in CloudIoT Environments'. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE)*, pp. 233–240.

APPENDICES

Appendix A: Survey Results

The following figures present the results of the requirements elicitation survey distributed to practising DevOps and Site Reliability Engineers. The survey comprised ten questions addressing current diagnostic pain points, acceptable response latency, confidence presentation preferences, and dashboard feature priorities.

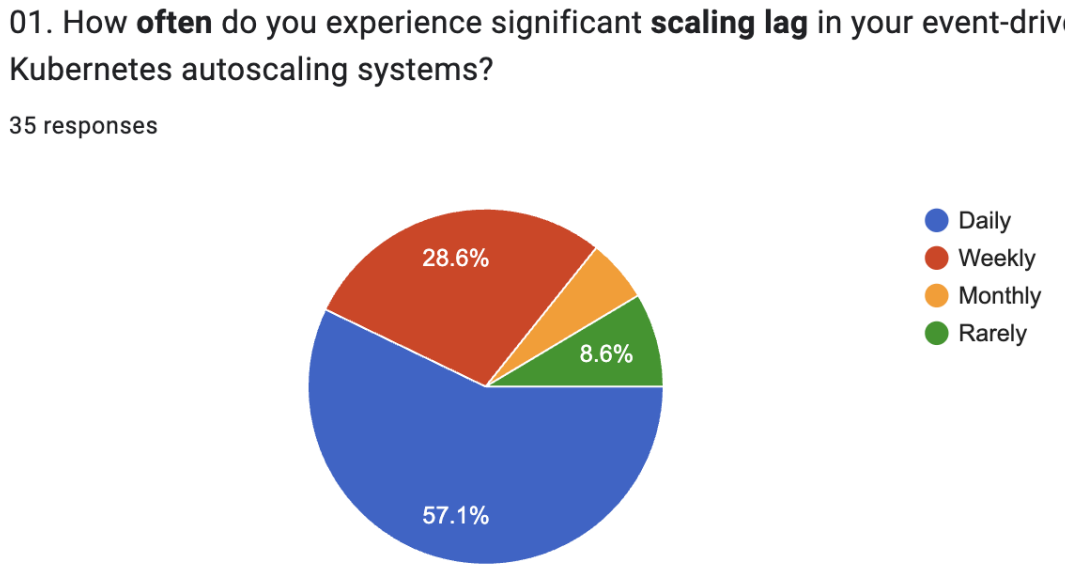


Figure 10.1: Frequency of autoscaling delays encountered

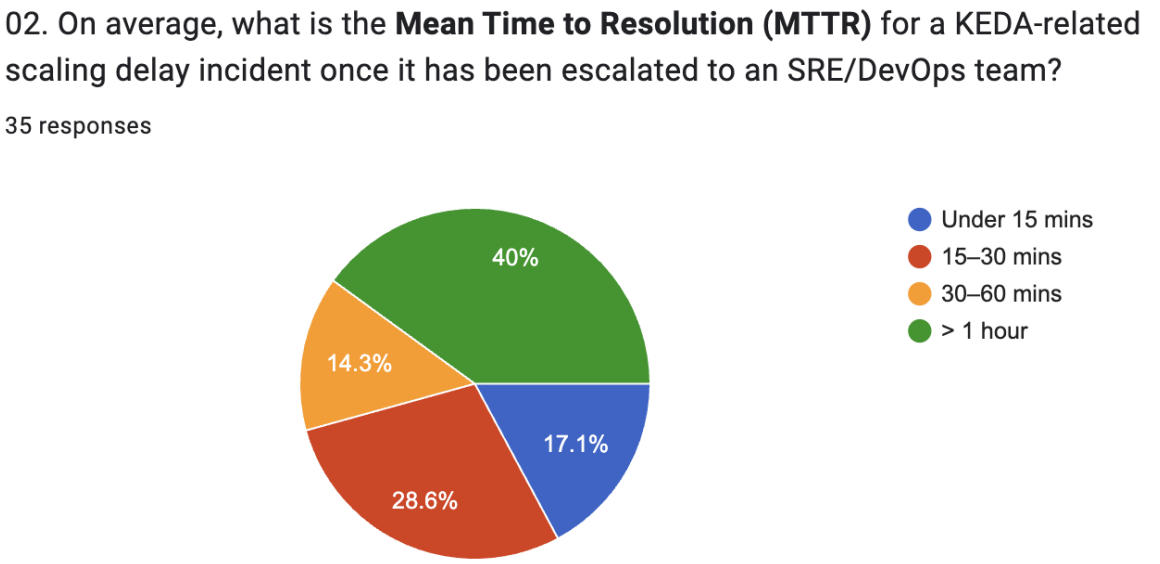


Figure 10.2: Duration of manual root cause diagnosis

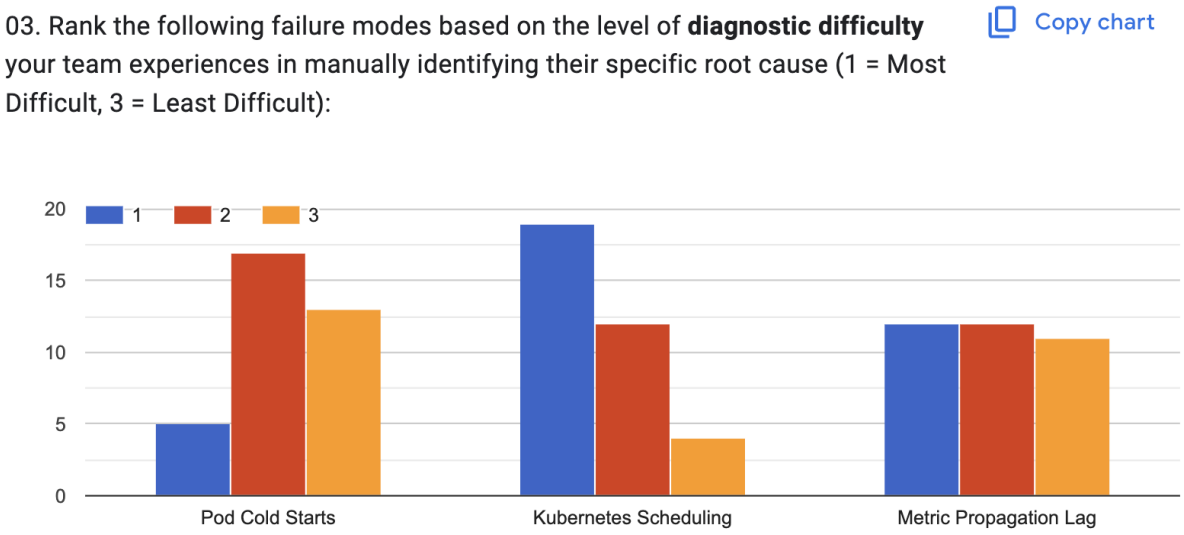


Figure 10.3: Most frequently encountered root cause category

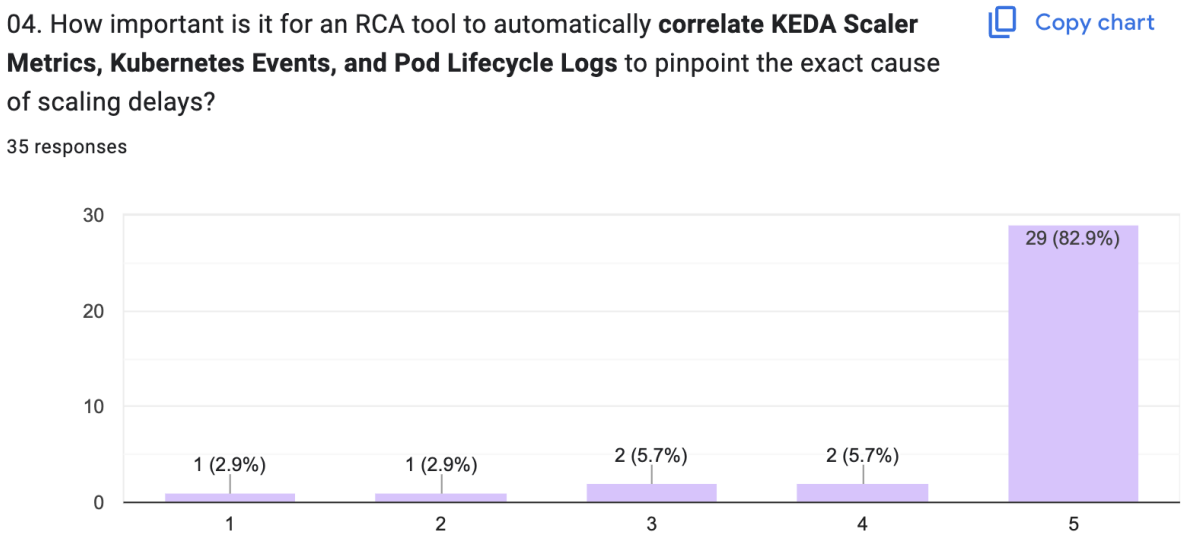


Figure 10.4: Importance of quantified confidence scores

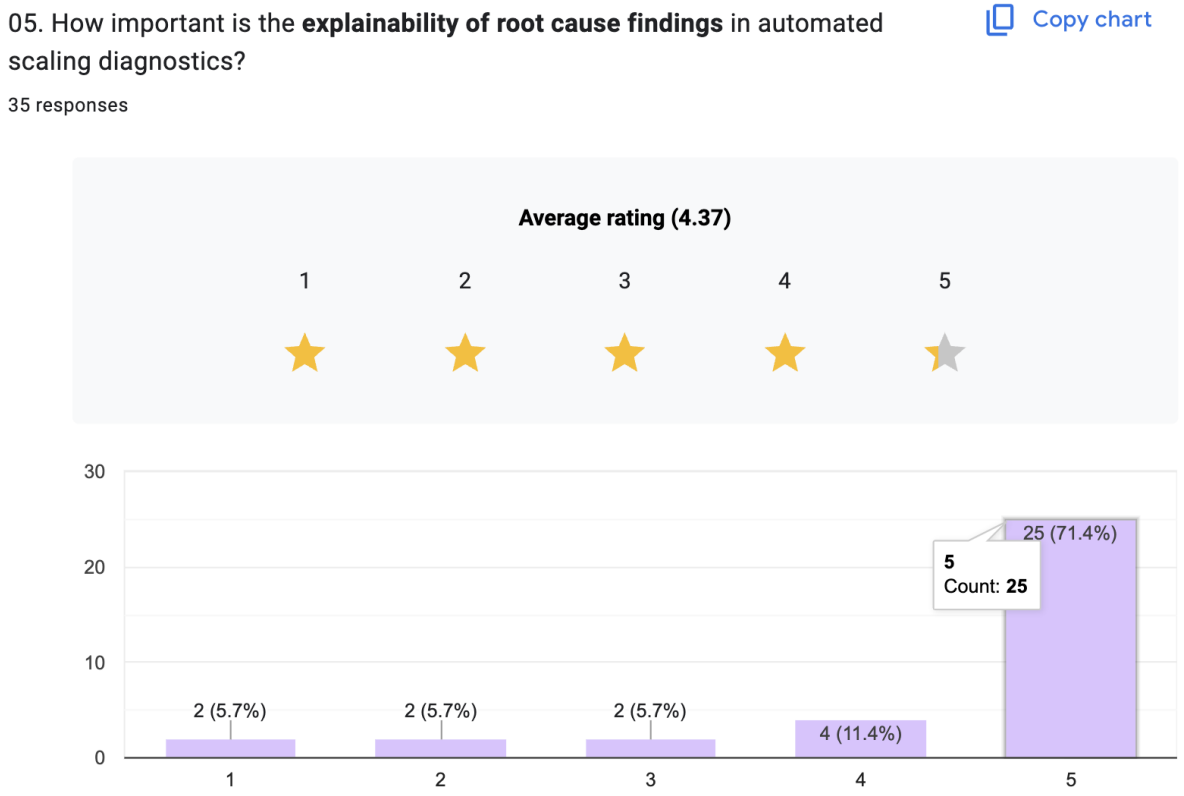


Figure 10.5: Maximum acceptable RCA latency

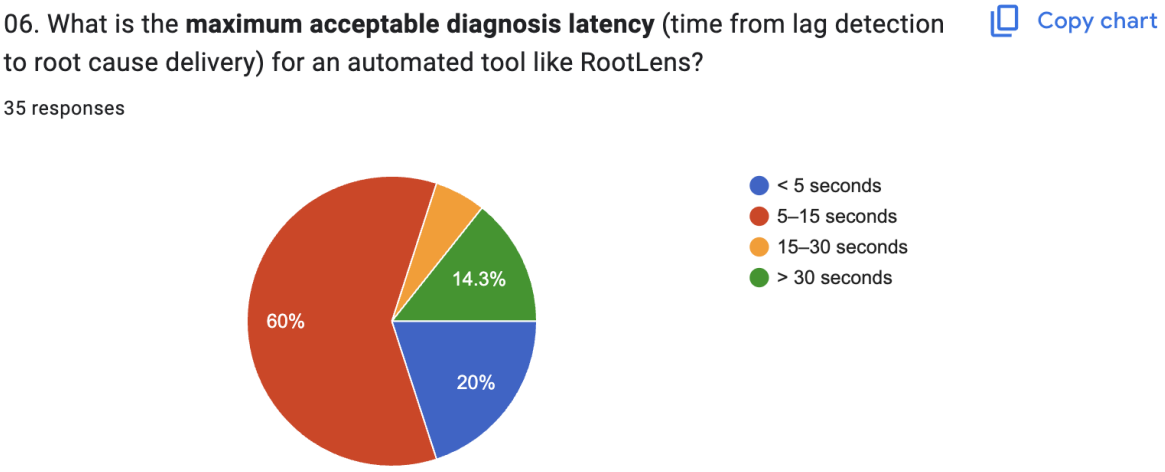


Figure 10.6: Usefulness of sub-cause granularity

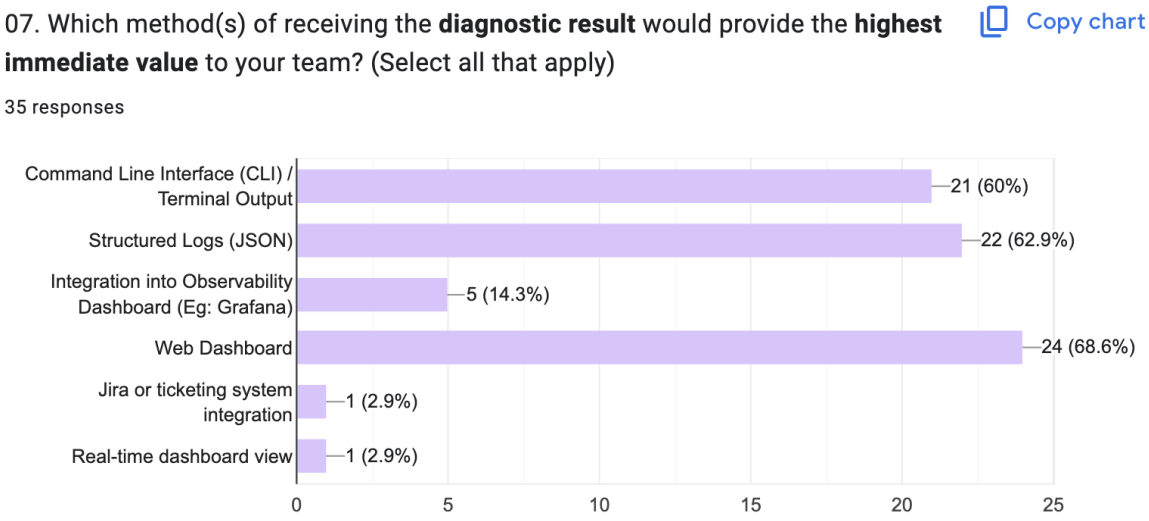


Figure 10.7: Highest priority dashboard feature

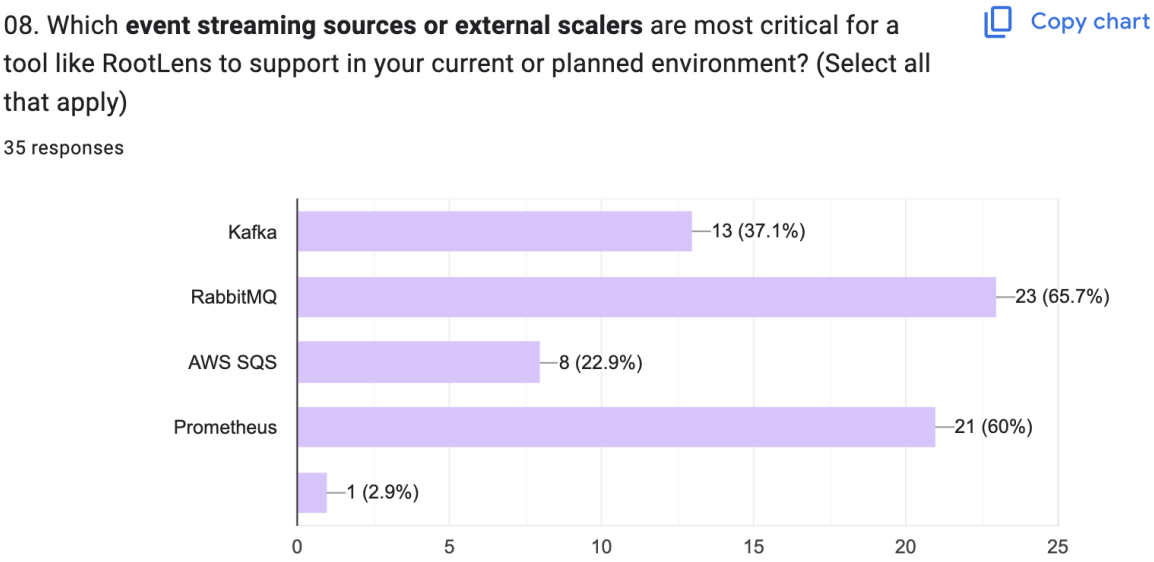


Figure 10.8: Importance of historical trend view

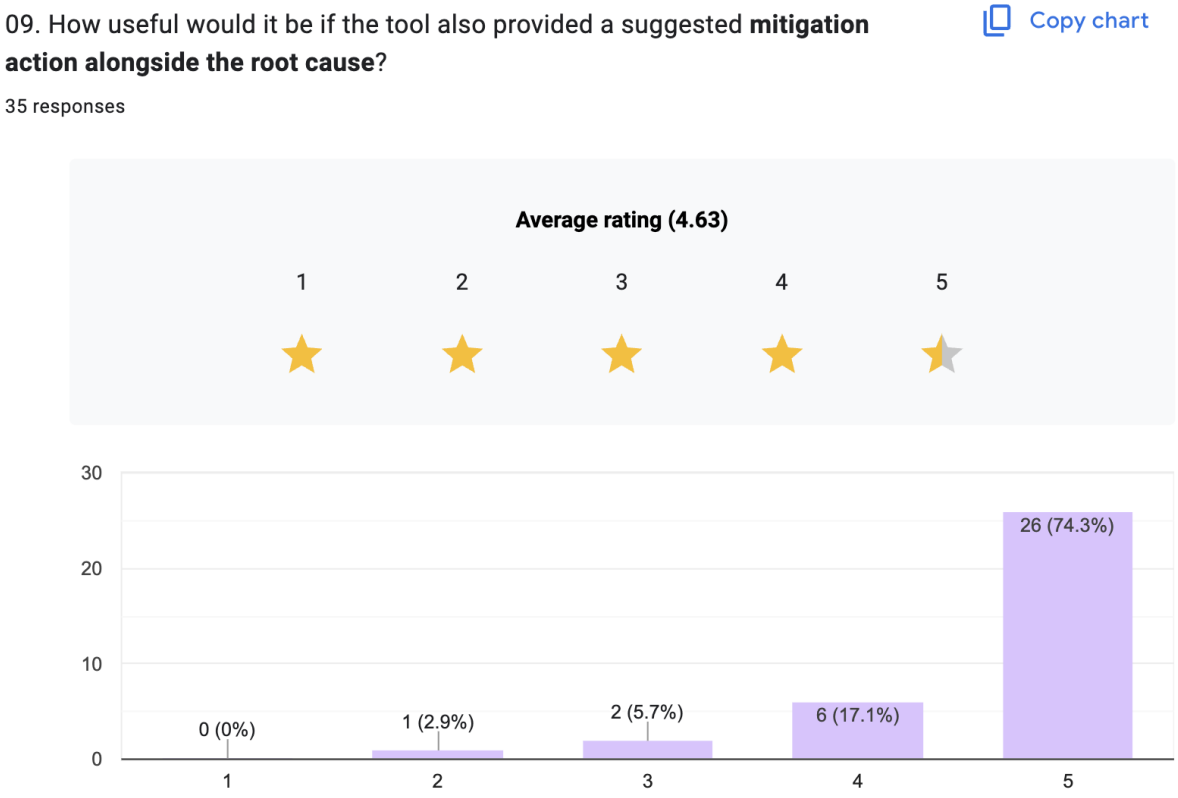


Figure 10.9: Willingness to integrate automated RCA into alerting

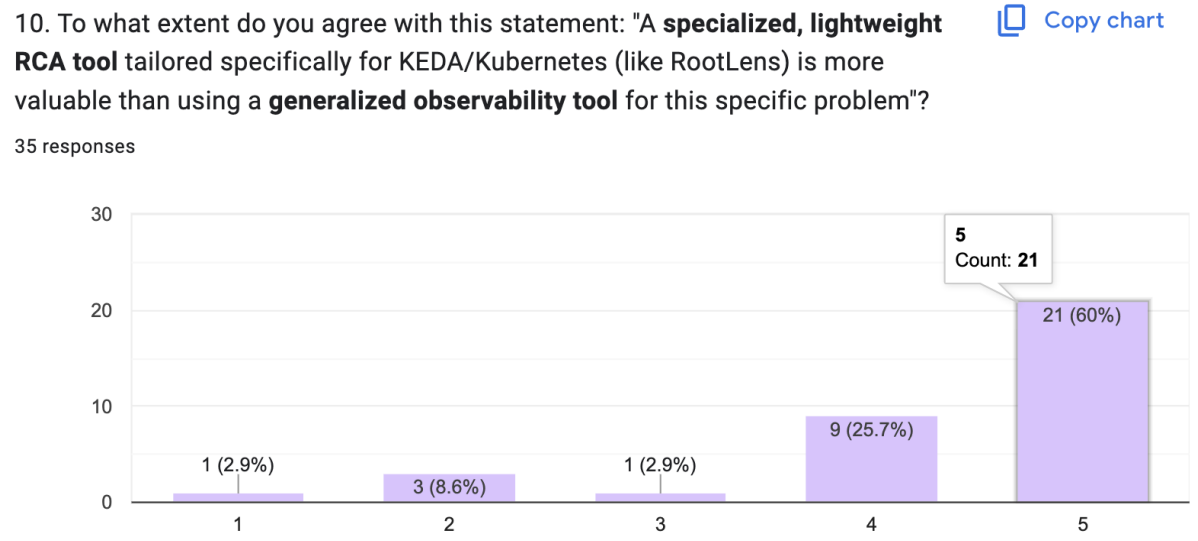


Figure 10.10: Required confidence threshold for trusting automated diagnosis

Appendix B: Testing Materials

The following listing shows a minikube setup used for the local development.

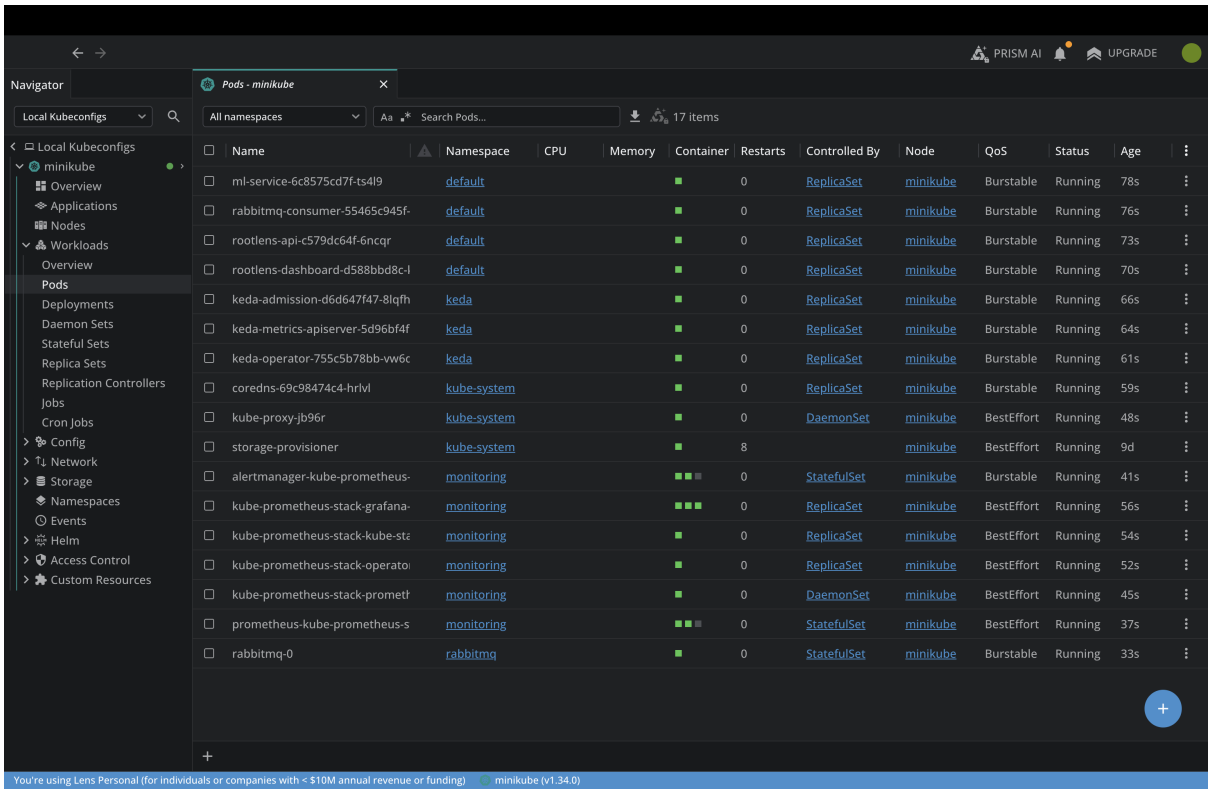


Figure 10.11: Minikube Cluster Setup

Appendix C: Code Snippets

kubernetes.go

pkg > collector > kubernetes.go

```go
type K8sCollector struct {
    clientset      *kubernetes.Clientset
    namespace      string
    eventCollector *EventCollector
}

func NewK8sCollector(kubeconfigPath, namespace string) (*K8sCollector, error) {
    config, err := clientcmd.BuildConfigFromFlags("", kubeconfigPath)
    if err != nil {
        return nil, fmt.Errorf("failed to build kubeconfig: %w", err)
    }

    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        return nil, fmt.Errorf("failed to create clientset: %w", err)
    }

    collector := &K8sCollector{
        clientset: clientset,
        namespace: namespace,
    }

    // Create EventCollector using the same clientset
    collector.eventCollector = NewEventCollectorWithClientset(clientset, namespace)

    return collector, nil
}
```

Figure 10.12: Code - Kubernetes Collector

keda.go

pkg > collector > keda.go

```go
type KEDACollector struct {
    dynamicClient dynamic.Interface
    clientset     *kubernetes.Clientset
    namespace     string
}

func NewKEDACollector(kubeconfigPath, namespace string) (*KEDACollector, error) {
    config, err := clientcmd.BuildConfigFromFlags("", kubeconfigPath)
    if err != nil {
        return nil, fmt.Errorf("failed to build kubeconfig: %w", err)
    }

    dynamicClient, err := dynamic.NewForConfig(config)
    if err != nil {
        return nil, fmt.Errorf("failed to create dynamic client: %w", err)
    }

    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        return nil, fmt.Errorf("failed to create clientset: %w", err)
    }

    return &KEDACollector{
        dynamicClient: dynamicClient,
        clientset:     clientset,
        namespace:     namespace,
    }, nil
}
```

Figure 10.13: Code - Keda Collector

rabbitmq.go ×

BLACKBOX

pkg > collector > rabbitmq.go

```go
type RabbitMQCollector struct {
    host      string
    port      int
    username  string
    password  string
    queueName string
    vhost     string
    useTLS    bool
    client    *http.Client
}

func NewRabbitMQCollector(host string, port int, username, password, queueName, vhost string, useTLS bool) *RabbitMQCollector {
    return &RabbitMQCollector{
        host:      host,
        port:      port,
        username:  username,
        password:  password,
        queueName: queueName,
        vhost:     vhost,
        useTLS:    useTLS,
        client:    &http.Client{Timeout: 10 * time.Second},
    }
}

func (r *RabbitMQCollector) Collect() (*types.RabbitMQMetrics, error) {
    protocol := "http"
    if r.useTLS {
        protocol = "https"
    }

```

Figure 10.14: Code - Rabbitmq Collector

engine.go ×

pkg > rca > engine.go

```go
// classifyRootCause classifies by dominant delay and tracks all contributing causes
func (e *Engine) classifyRootCause(
    coldStart,
    scheduling,
    metricLag time.Duration,
    telemetry *types.TelemetryData,
    podMetrics []types.K8sMetrics,
) *types.DiagnosticResult {

    result := &types.DiagnosticResult{
        DetectedAt:         time.Now(),
        Evidence:           []string{},
        Recommendations:    []string{},
        Metrics:            make(map[string]interface{}),
        ContributingCauses: []types.ContributingCause{},
    }

    coldStartThreshold :=
        time.Duration(e.config.ColdStartThresholdSeconds) * time.Second
    schedulingThreshold :=
        time.Duration(e.config.SchedulingThresholdSeconds) * time.Second
    metricLagThreshold :=
        time.Duration(e.config.MetricLagThresholdSeconds) * time.Second

    result.Metrics["cold_start_delay_seconds"] = coldStart.Seconds()
    result.Metrics["scheduling_delay_seconds"] = scheduling.Seconds()
    result.Metrics["metric_lag_seconds"] = metricLag.Seconds()
    result.Metrics["queue_depth"] = telemetry.RabbitMQMetrics.MessageCount
    result.Metrics["desired_replicas"] = telemetry.KEDAMetrics.DesiredReplicas
    result.Metrics["current_replicas"] = telemetry.KEDAMetrics.CurrentReplicas

    minDelay := 100 * time.Millisecond
    if coldStart < minDelay && scheduling < minDelay && metricLag < minDelay {
        result.RootCause = types.Unknown
        result.Confidence = 0.3
        result.Evidence = append(
            result.Evidence,
            "No significant delays detected - system performing optimally",
        )
        return result
    }
```

Figure 10.15: Code - RCA Logic

router.go

pkg > ml > router.go

```go
// ExplainerRouter routes RCA results to the appropriate sub-cause explainer
type ExplainerRouter struct {
    Client *ExplainerClient
    Logger *logrus.Logger
}

// NewExplainerRouter creates a new explainer router
func NewExplainerRouter(client *ExplainerClient, logger *logrus.Logger) *ExplainerRouter {
    return &ExplainerRouter{
        Client: client,
        Logger: logger,
    }
}

// Explain routes to the appropriate explainer based on root cause category
func (r *ExplainerRouter) Explain(ctx context.Context, rootCause types.RootCauseType, metrics map[string]interface{}) (*SubCauseResult, error) {
    switch rootCause {
    case types.ColdStart:
        r.Logger.Debug("Routing to Cold Start explainer")
        return r.Client.ExplainColdStart(ctx, metrics)

    case types.SchedulingDelay:
        r.Logger.Debug("Routing to Scheduling explainer")
        return r.Client.ExplainScheduling(ctx, metrics)

    case types.MetricPropagationLag:
        r.Logger.Debug("Routing to Metric Propagation explainer")
        return r.Client.ExplainMetricPropagation(ctx, metrics)

    case types.Unknown:
        r.Logger.Debug("Root cause is UNKNOWN - no explainer available")
        return nil, nil // No explanation for unknown root cause

    default:
        r.Logger.Warnf("Unknown root cause type: %s", rootCause)
        return nil, fmt.Errorf("no explainer for root cause: %s", rootCause)
    }
}
```

Figure 10.16: Code - ML Explainer Router

Appendix D: System UI/UX

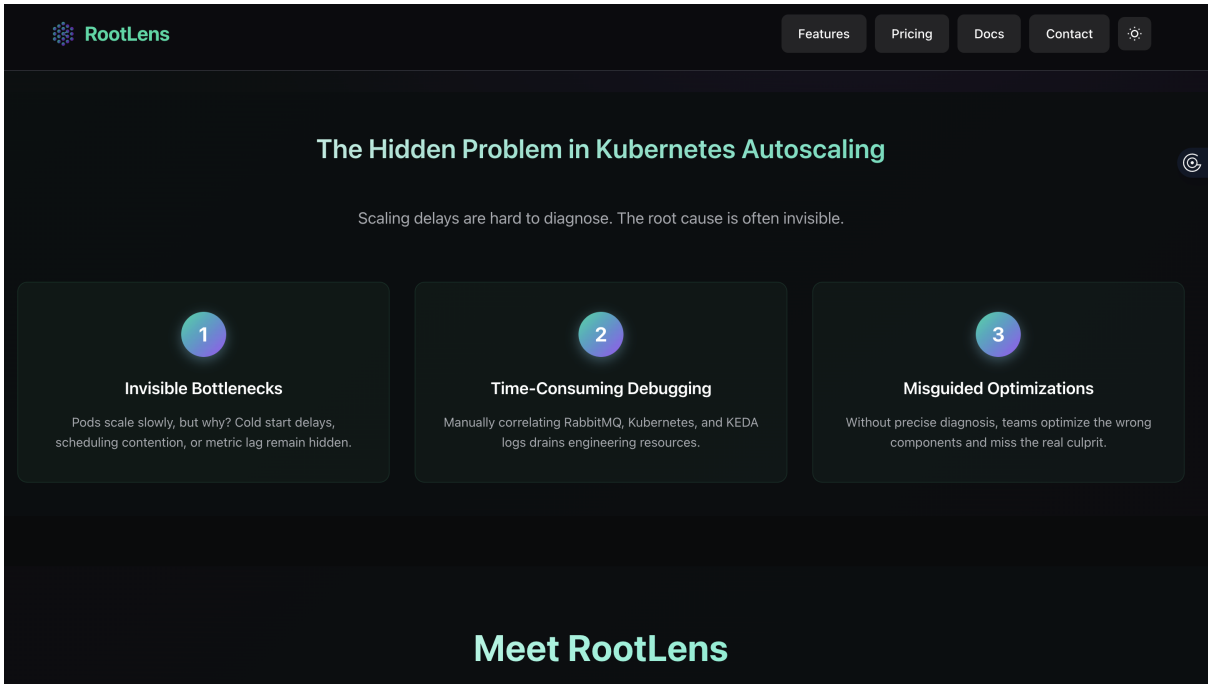


Figure 10.17: Marketing Website Page 2

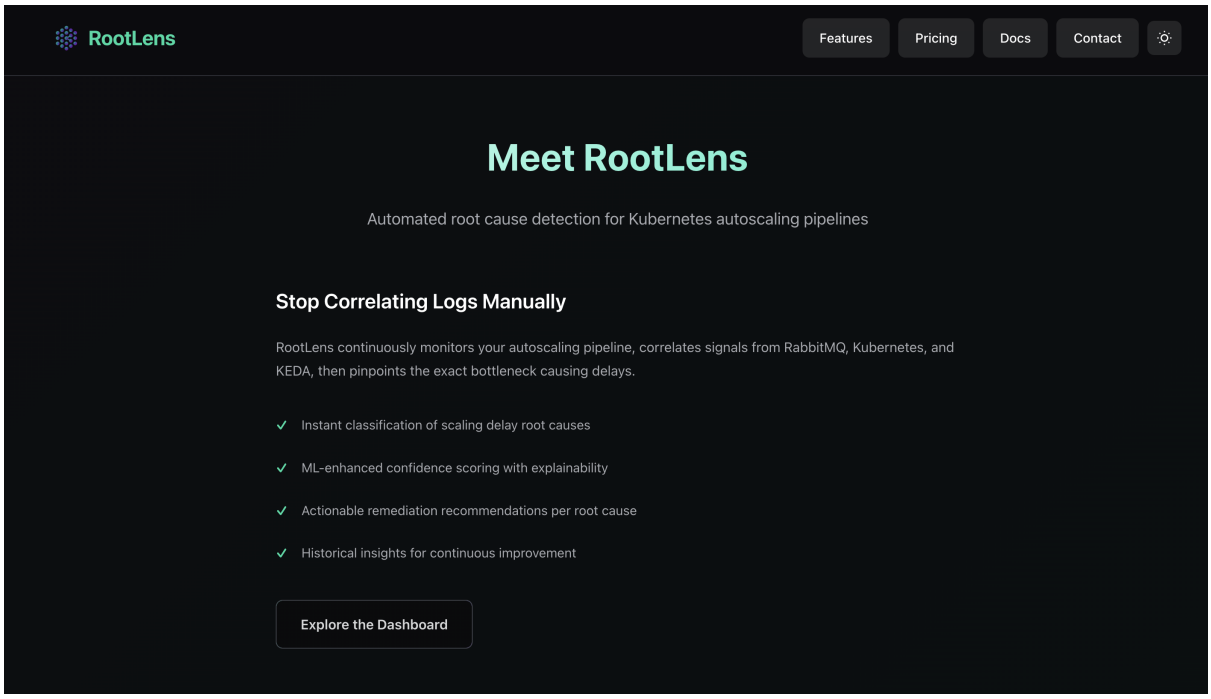


Figure 10.18: Marketing Website Page 3

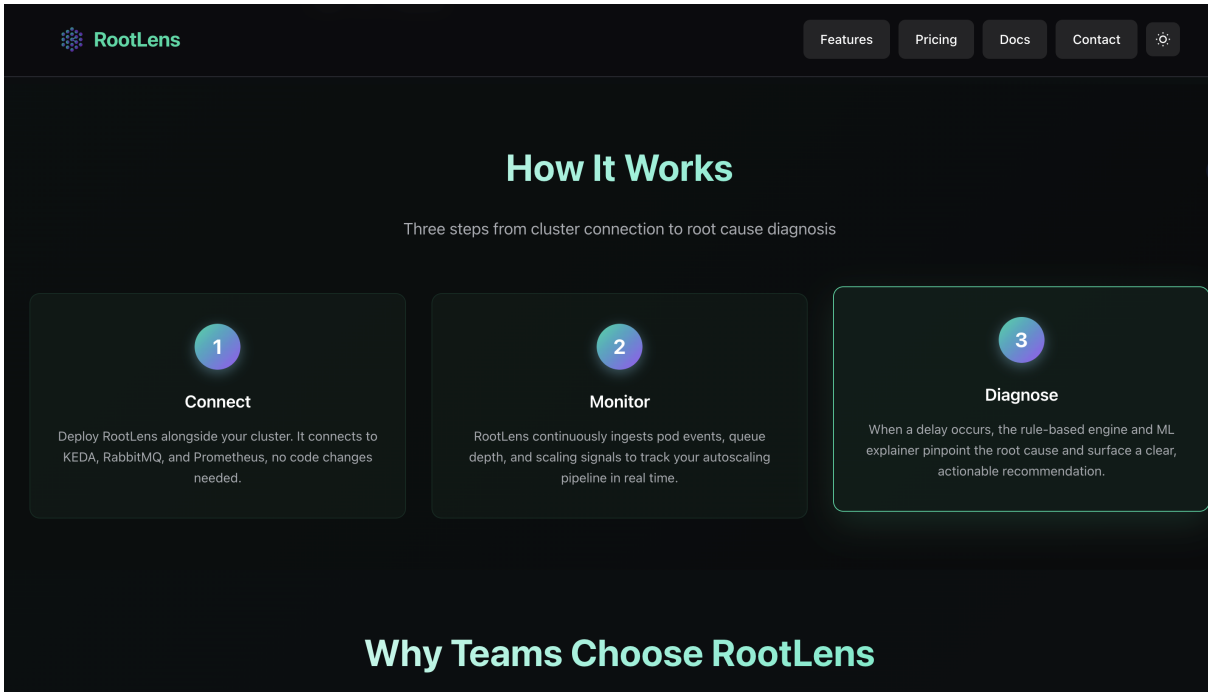


Figure 10.19: Marketing Website Page 4



Figure 10.20: Marketing Website Page 5

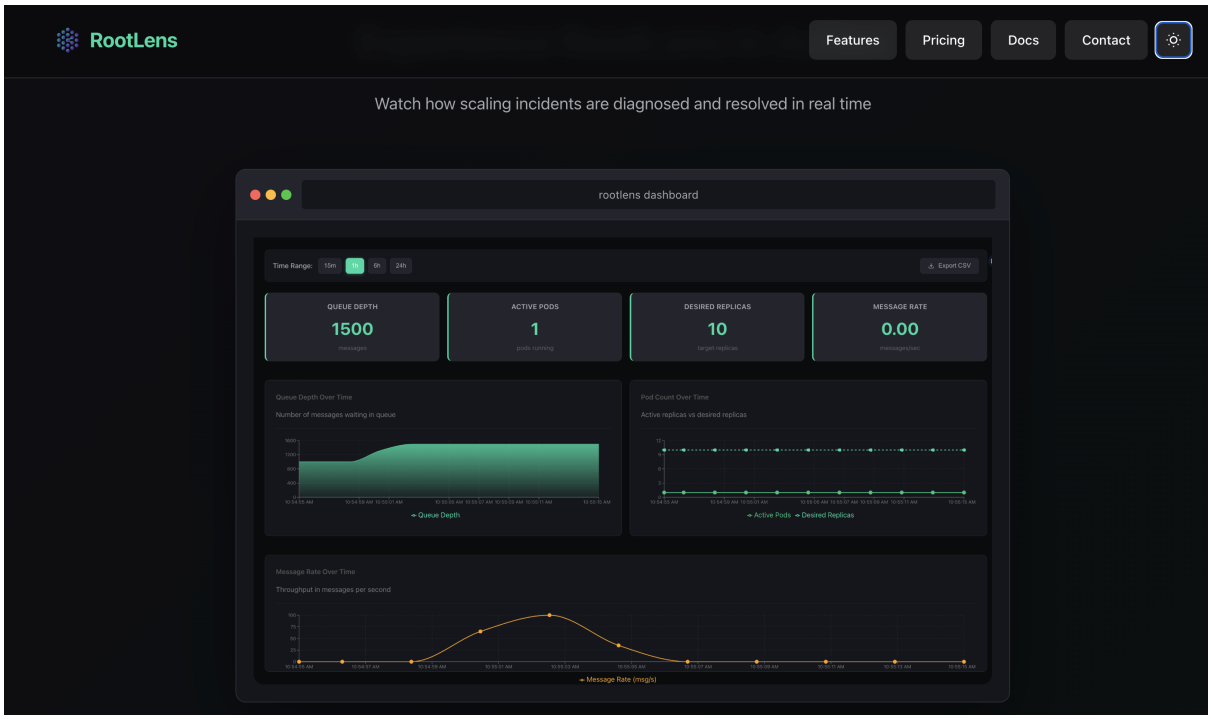


Figure 10.21: Marketing Website Page 6

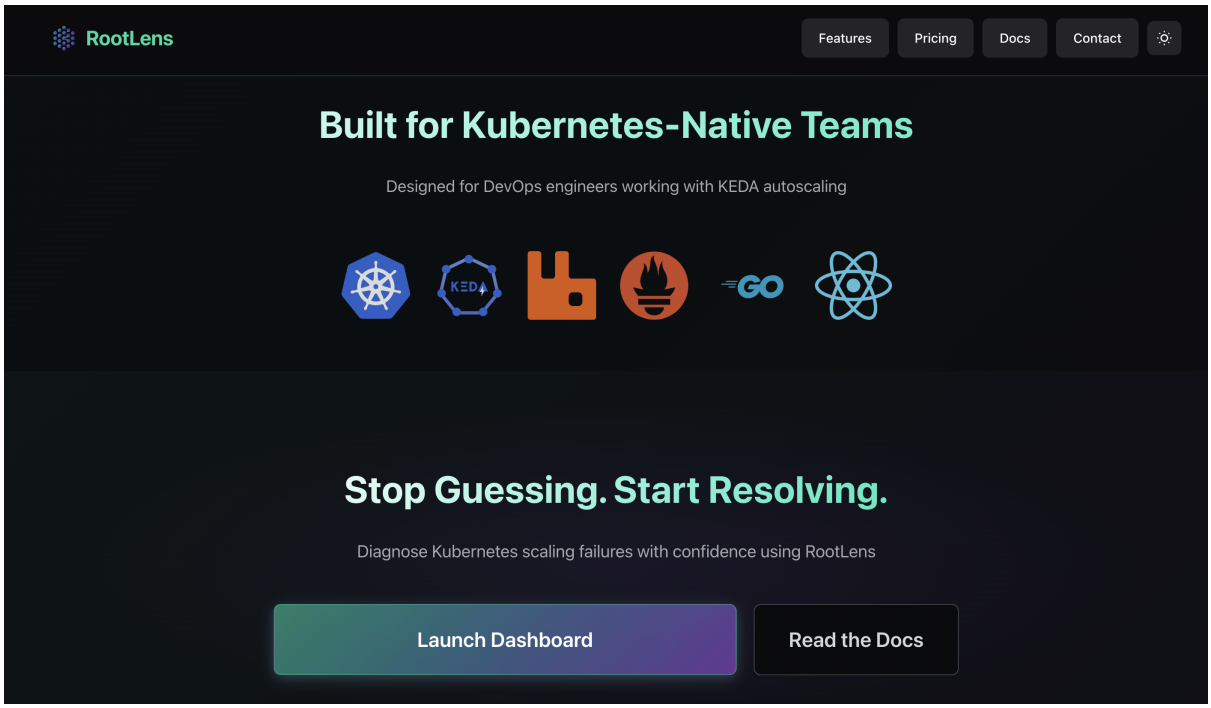


Figure 10.22: Marketing Website Page 7

Appendix E: Other Materials

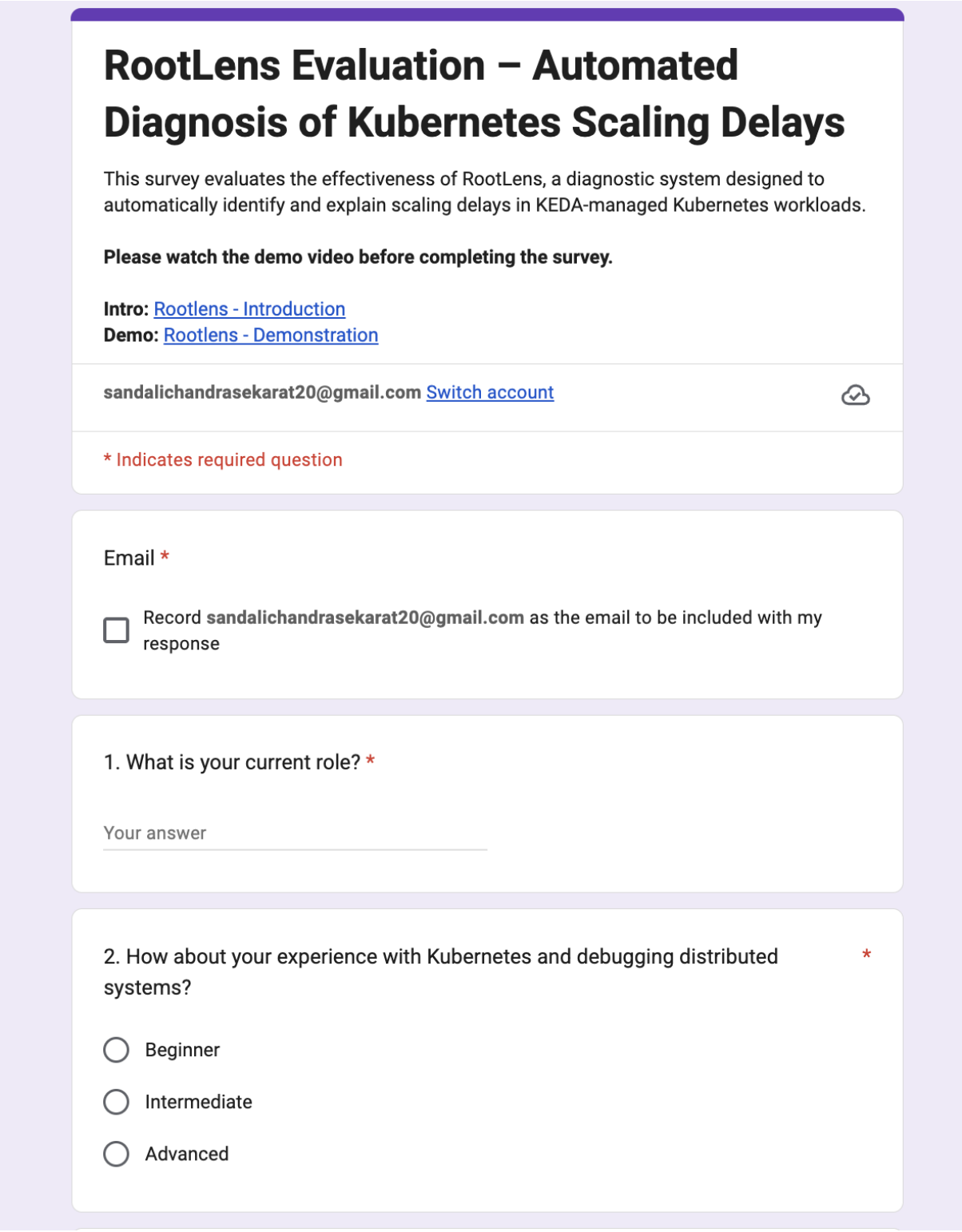


Figure 10.23: Shared Google Form for Evaluations

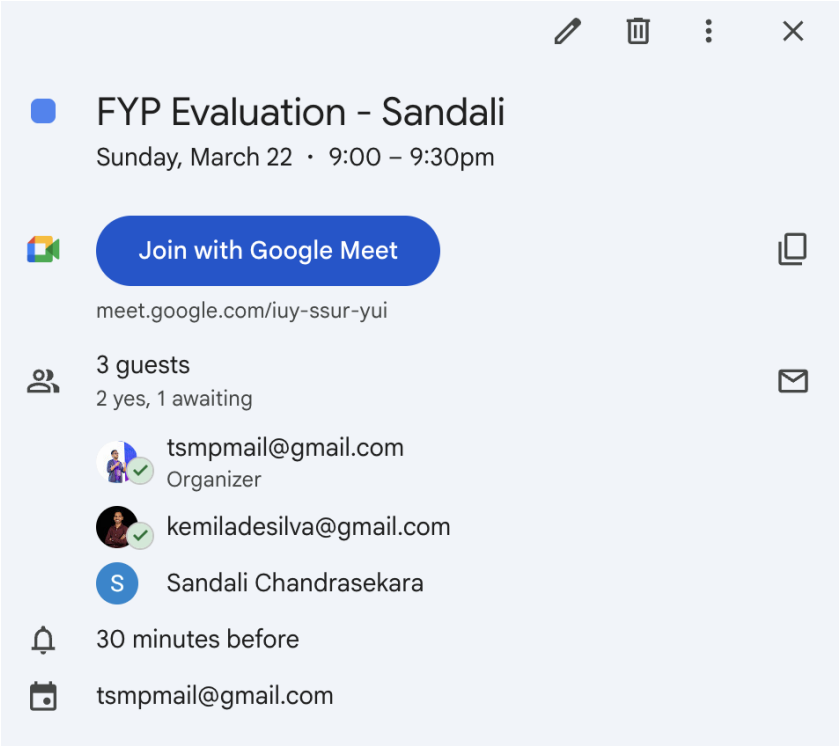


Figure 10.24: Evaluation Meetings

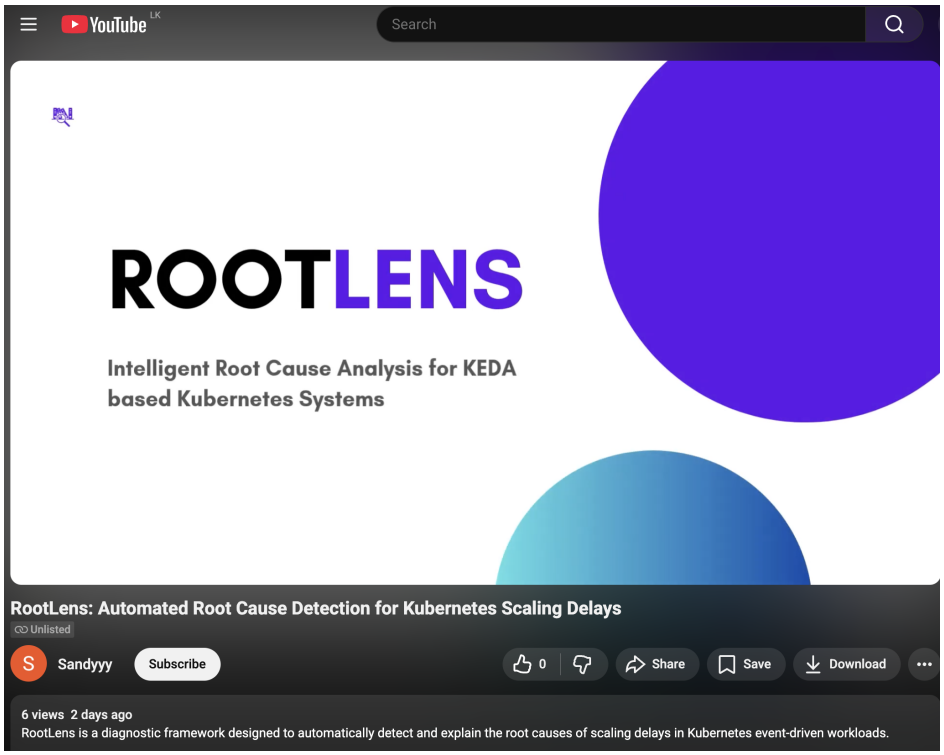


Figure 10.25: Shared Introduction Video

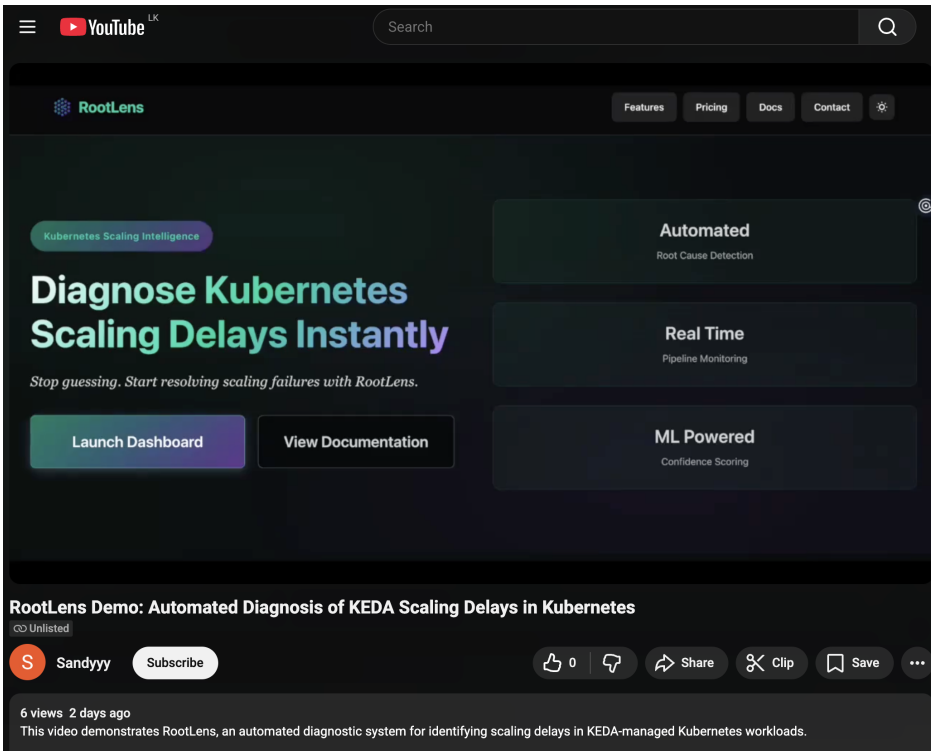


Figure 10.26: Shared Demo Video